

无线电干扰源定位与清除的覆盖保证与在线调度模型

摘要

本题的核心是：在源数未知、测向误差有界、辐射方向未知的条件下，让机器狗自主完成“发现一定位一清除”，并给出**可证明全覆盖、可认证清除、可执行调度**的闭环方案。全文按四层展开：覆盖保证（每个源必被收到）→ 布局下界（保证下用最少测站）→ 在线调度（怎样走、怎样测最省时间）→ 有界误差定位与认证清除（清除前用几何证书确认定位足够准），并始终以**清除率 100% 为硬约束**。四项核心结果：140 局演练与 6 次正式测试清除率均为 100%；第三问证明**七站达到距离覆盖的理论下界**；第四问把测站压到 **22 站**并取得 1 m 网格连续域证书；合并平均定位清除时间 239.0 s/ 源（第三问）与 498.8 s/ 源（第四问）。

针对问题一，把示向度读数化为前向楔形之交，给出区域直径算法；由 Jung 定理证明以 D 为直径的圆一般不能覆盖定位区域（三站算例 $D = 40\text{ m}$ 、 $R_* = 23.094\text{ m} > 20\text{ m}$ ），故全文认证清除判据取 $R_* \leq 20\text{ m}$ 。

针对问题二，导出无需已知源距与接收半径的保证接收区域 C_{safe} （三个 1000 m 圆盘之交），并证明两次测向的分辨极限——存在相距 41 m 的两源产生相同观测，故光学覆盖兜底不可省略。

针对问题三，证明“圆心加正六边形”七站布局最坏覆盖距离 $988.5114\text{ m} < 1000\text{ m}$ ，并由 Bezdek 界证明七站是保证发现的最小站数；官方演练 70 局、917 个源全部清除。

针对问题四，给出方向覆盖的**充要条件**（源落在近邻测站凸包内），据此把测站压到 22 站，并以 1018.68 万点、最小余量 1.7331 m 的 1 m 网格证书完成连续域验证；官方演练 70 局、887 个源全部清除。

140 局演练还表明**整局总时间几乎完全由固定成本决定，与源数无关**，平均时间随源数下降主要是分母效应；三次正式测试全部清除，**合并平均 239.0 与 498.8 s/ 源**。

关键词：交会定位；有界误差可行域；覆盖保证；凸包方向覆盖；在线调度；最小包围圆

解答总览

问题	核心方法	关键结果	节
一	前向楔形交会 + 直径算法	$D = 40\text{ m}$ 时 $R_* = 23.09\text{ m} > 20\text{ m}$, 判据取 $R_* \leq 20\text{ m}$	四
二	保证接收区 C_{safe} + 分辨极限	相距 41 m 的两源不可分辨, 光学兜底不可省	五
三	七站覆盖 + 混合贪心调度	$d_{\text{max}} = 988.51\text{ m} < 1000\text{ m}$, 七站为最小	六
四	凸包方向覆盖 + 网格证书	22 站, 1018.68 万点通过, 最小余量 1.7331 m	七
演练	官方模拟器逐局核验	140 局 100% 清除; 239.0 / 498.8 s / 源	八

复现: 源码哈希与样本筛选规则见第八节, 运行方式与日志清单见附录 A。

一、问题重述

1.1 问题背景

无线电干扰源的定位与清除是频谱管理的关键任务, 其中最后的精准定位须由技术人员携带便携式测向机徒步完成, 效率低且风险大。本题要求搭载测向机、光学探测仪与激光枪的机器狗, 在半径 $R = 1800\text{ m}$ 、以原点为心的圆形区域内自动完成发现一定位一清除, 并使总耗时尽可能短。

区域中干扰源个数未知 (第三、四问为 $10 \sim 16$ 个), 每源占用 $1 \sim 20$ 中互不相同且不变的一个频道, 其位置、有效接收半径 ($1000 \sim 1500\text{ m}$) 与第四问的定向方向都无法直接读取, 机器狗只能通过 `/measure` 与 `/clear` 交互。因此问题本质是**部分可观测的在线决策问题**。

1.2 需要解决的问题

问题一: 已知若干检测点坐标及某干扰源关于这些点的示向度, 给出用交会定位法计算多边形定位区域直径的算法, 并回答以该直径为直径的圆能否覆盖此定位区域。

问题二: 已知某全向干扰源在一个检测点处的示向度, 给出第二个检测点的选择策略, 使定位效果较好, 并给出第二个检测点的候选区域。

问题三: 区域中的源全部为全向源, 总数在 $10 \sim 16$ 之间但具体未知。机器狗自 $(0, 0)$ 出发、测向机初始频道为 1, 在确保全部清除的前提下使完成时间尽可能短。需给出策略与算法, 通过模拟器演练计算“被清除干扰源个数的比例”与“平均定位清除时间”, 并报告三次正式测试的结果。

问题四: 区域中既有全向源又有定向源, 总数仍为 $10 \sim 16$, 定向源的个数与方向均未知。定向源的有效覆盖角度范围为定向方向两侧各 90° (含边界), 范围之外没有信号辐射。其余条件与问题三相同。

二、题设约束、模型假设与符号说明

2.1 题设硬约束与计时规则

题目与附件给出的规则构成模型的全部硬约束, 汇总于表 1。其中三条对建模影响最大。

其一, 测向误差有界且确定。误差只保证落在 $[-1^\circ, 1^\circ]$ 内, 且同一地点重复检测读

数不变，故不能假设零均值或高斯分布，也不宜使用最小二乘、卡尔曼滤波等依赖概率模型的估计器；本文采用集员定位，只维护“真源可能在哪里”的集合。

其二，清除判据是距离。光学定位与激光清除只要求清除点到源不超过 20 m，与辐射方向无关；因此即使某定向源不再发出信号，只要还持有一次方位约束，仍可用光学覆盖清除。

其三，动作耗时结构固定。移动 5 m/s、切频 1 s、检测 5 s、清除失败 3 s、成功 5 s。这组常数决定了全部成本权衡：一次额外检测约相当于 25 ~ 30 m 的移动。

表 1 题目给定的物理规则与计时常数

项目	规则	取值	项目	规则	取值
目标区域	圆盘，东为 x 正向	$R = 1800\text{ m}$	移动	直线，可出区域	5 m/s
源个数	不通过接口返回	10 ~ 16	检测	须停止移动	5 s
频道	一源一频道，不变	1 ~ 20	切换频道	任意两频道之间	1 s
示向度误差	同点重复检测固定	$\pm 1^\circ$	清除成功 / 失败	与辐射方向无关	5 s / 3 s
有效接收半径	每源一值，未知	1000 ~ 1500 m	近场	返回 near	5 m
定向覆盖	闭半平面，含边界	180°	清除半径	确定性阈值	20 m
虚拟时间上限	防止死循环	360000 s	程序运行时间	两个限制取早者	$\leq 1200\text{ s}$

2.2 模型假设

本文的全部几何保证都建立在下列假设之上，其失效后果在第九节中逐条检查。

假设 1：静态环境。一次任务过程中，干扰源的位置、频道、有效接收半径与定向方向均不随时间变化；机器狗可沿任意直线自由行走，不考虑障碍、地形与高度差。

假设 2：确定性阈值。接收、近场与清除均为确定性阈值，不存在概率性漏检或误清除；不同源的信号互不影响。

假设 3：有界测向误差。读数误差有界： $|\theta - \theta^{\text{true}}| \leq \varepsilon = 1^\circ$ ；算法统一取保守界 $\bar{\varepsilon} = 1.005^\circ$ 以吸收取整偏差，且不假设误差的分布、独立性或零均值。

假设 4：信息边界。策略只能使用接口返回值，不得读取源的个数、位置、频道集合、接收半径或定向方向；本地实验真值仅用于事后核验。

假设 5：计时模型。虚拟时间按附件规则累加：/measure 为“移动距离 /5 + 切频 0/1 +5”秒，/clear 为“移动距离 /5 +3（失败）或 +5（成功）”秒；/clear 不改变当前频道，/enter、/exit 不耗时。

假设 6：本地实验生成器，仅用于离线比较。本地配对实验中源数在 10 ~ 16 均匀抽取、位置按面积均匀、接收半径在 [1000, 1500] 均匀、第四问每源以 0.5 概率为定向源；误差由“种子、频道与坐标”哈希确定。这些分布是本文假设，本地数值只用于版本之间的配对比较，不作为对官方成绩的预测。

表 2 通用符号（左）与各问专用符号（右）

符号	意义	符号	意义
$\mathcal{D}$	目标圆盘, $R = 1800$ m	v	速度 5 m/s
g_i, c_i	源 i 的位置、频道	t_m, t_s	检测 5 s、切频 1 s
ρ_i, d_i	其接收半径、定向方向	t_f, t_c	清除失败 3 s、成功 5 s
$\rho_{\min}, \rho_{\max}$	1000 m、1500 m	r_c, r_n	清除 20 m、近场 5 m
$\varepsilon, \bar{\varepsilon}$	误差界 1° 、 1.005°	L, N_m, N_s	移动距离、检测、切频次数
$\mathbf{u}(\alpha)$	$(\cos \alpha, \sin \alpha)^\top$	N_f, N_c	清除失败、成功次数
$[\mathbf{a}, \mathbf{b}]$	叉积 $a_x b_y - a_y b_x$	$T, \bar{T}$	总时间; $\bar{T} = T/N_c$
s_k, θ_k	第 k 个检测点及其读数	$\mathcal{S}$	搜索站集合
W_k	前向楔形	$\mathcal{N}_\rho(g)$	距 $g \leq \rho$ 的测站集
P	定位区域（可行域）	n	该局真实源数
D, R_*, c_*	直径、最小包围圆半径与圆心		

2.3 符号说明

全文内部计算一律使用弧度, 接口输入输出保持度与米。方位角采用题目约定: 东为 0° 、逆时针为正、取值范围 $[0^\circ, 360^\circ)$ 。术语上, 正文简称的“测站”与题面的“检测点”同义（机器狗停靠并测量的位置）; $\mathcal{S}$ 为测站集合, 亦称“搜索站集合”。

三、 问题分析与总体框架

3.1 目标函数的精确形式

按假设 5, 把一局任务中的全部动作按类型归并, 整局虚拟总时间可以写成一个精确的线性表达式:

$$T = \underbrace{\frac{L}{v}}_{\text{移动}} + \underbrace{t_m N_m}_{\text{检测}} + \underbrace{t_s N_s}_{\text{切频}} + \underbrace{t_f N_f}_{\text{光学失败}} + \underbrace{t_c N_c}_{\text{成功清除}} = \frac{L}{5} + 5N_m + N_s + 3N_f + 5N_c. \quad (1)$$

这里 $t_c = 5$ s 已经包含了成功那一次的光学精确定位 3 s 与激光清除 2 s, 计算时只计一次。题目要求的两个统计量是

$$\text{清除比例} = \frac{N_c}{n}, \quad \bar{T} = \frac{T}{N_c}, \quad (2)$$

其中 n 为该局真实源数。于是优化问题为

$$\min_{\pi} \mathbb{E}[T(\pi)] \quad \text{s.t.} \quad N_c(\pi) = n \text{ 以概率 } 1, \quad (3)$$

π 是一个只依赖历史观测的在线策略（假设 4）。本文把约束收紧为“以概率 1”，这是所有布局设计的出发点：以清除率 100% 为硬约束、以时间为优化目标，覆盖保证优先（该取舍的定量代价在第九节中另行讨论）。

两种平均口径，全文一律标注。一批 M 局测试有两种求平均的方式，它们是不同的量：

$$\bar{T}_{\text{案例}} = \frac{1}{M} \sum_{i=1}^M \frac{T_i}{N_i}, \quad \bar{T}_{\text{合并}} = \frac{\sum_i T_i}{\sum_i N_i}. \quad (4)$$

前者每局等权，回答“随机抽一局的平均每源耗时是多少”；后者按源数加权，回答“累计处理每个源的总体成本是多少”。源多的局 $\bar{T}$ 偏小，所以合并口径通常略低于案例口径。两种口径各有用途，但比较两个版本时必须用同一口径。本文所有表格均在表题或表注中注明采用哪一种，跨表引用数字时也一并标注。

由 (2) 可见， $\bar{T}$ 的分母是 n 。下面的第 3.2 节将说明， T 的主体是与 n 关系很弱的固定成本，因此 $\bar{T}$ 随 n 下降主要是分母效应。这一点在解释三次正式测试成绩时至关重要。

3.2 时间构成与固定成本

对官方演练（第三问 70 局、第四问 70 局）的时间构成按动作类型分解，两问构成高度一致：移动约占 75%、检测约占 20%，二者合计约 95%，真正用于清除的时间不到 1%——瓶颈是“把机器狗移到该去的地方”。整局总时间对源数 n 的线性回归为

$$T_{Q3} \approx 2355 + 72n, \quad T_{Q4} \approx 6848 - 40n \quad (\text{s}), \quad (5)$$

斜率很小（第四问甚至为负），两个截距即与 n 基本无关的固定成本——巡站路线与对空频道的排除性扫描。因此平均定位清除时间 $\bar{T} \approx T_0/n + k$ 随源数下降主要是分母效应， $n = 10$ 与 $n = 16$ 实测可相差近一半。由此得到两条结论：其一，主要优化空间来自固定成本，即测站布局（第四问每减一站约省 260 s）；其二，减少测站受覆盖保证的硬约束，这正是第六节与第七节的核心问题。完整的时间构成表、回归与分层数据见第 G 节与第八节。

3.3 四层求解框架

综合上述分析，本文把四个问题统一在同一框架下，总体框架见图 1，执行流程即算法 2。四层分别对应“发现—省时—省路—保对”：覆盖保证（发现）用测站集保证任何源都会被收到，并兼作终止证明；布局下界（省时）在覆盖成立下最少化测站数与巡回长度；在线调度（省路）把搜索与目标处理放进同一任务池贪心选择；有界误差定位与认证清除（保对）用集员可行域与最小包围圆保证清除。每一层对应的数学保证、代码实现与逐例验收见表 3。用一句话概括，四层依次回答：会不会漏？最少要去几个地方？下一步去哪？什么时候应该清除？这样，后续的定理、布局与调度策略都能挂到四层中的某一层，读者无需再判断“这一段在解决什么”。另外，图 1 的左侧四列为理论功能划分，中央纵向流程为算法实际执行顺序，两者含义独立。

3.4 四问的核心难点

四问各自的核心困难可概括为：问题一要在有界角度误差下严格表示定位区域，并给出只依赖可行域的清除判据；问题二要让第二测点既保证收到信号又改善交会角，并说清两次测向的能力边界；问题三要在不知道源数与位置时证明“已全部发现”，并在

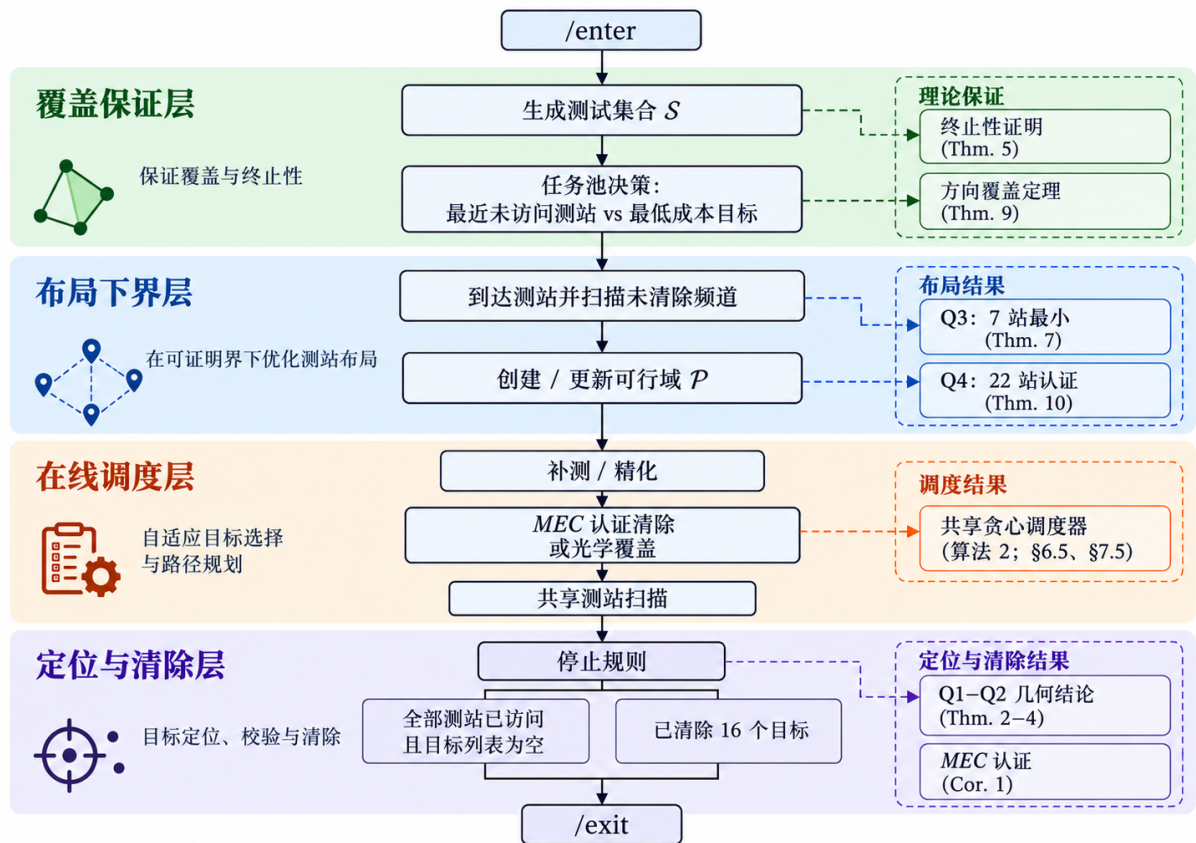


图 1 总体求解框架：覆盖保证、布局下界、在线调度与定位清除四层

表 3 数学保证、代码实现与逐例验收的对应

数学保证	代码实现	逐例验收
终止性 (定理 5)	走遍所有覆盖测站后才允许退出	逐例核对清除数 = 真实源数
方向覆盖 (定理 9)	运行时 20 m 粗网格凸包检查	$h = 1\text{ m}$ 网格证书 1018.68 万点全部通过
认证清除 (推论 1)	仅当 $R_* \leq r_c - 10^{-6}$ 才在圆心清除	清除点到真源距离 $\leq 20\text{ m}$
包含性不变量 ((22))	目标圆外切、圆盘排除取凸包外近似	每时刻可行域与包围圆均含真源

表 4 在线决策流程的判定与动作

状态	判断条件	后续动作
首测成功	返回 direction	由方位约束建立可行域，进入定位
首测无信号	全向源返回 no_signal	以圆盘排除该站邻域；定向源改用其他测站
观测不足	可行域无界或仅 1 条方位	走到第二测点或补测一次
可行域过大	$R_* > r_c$	圆心补测精化，仍不足则光学覆盖
认证通过	$R_* \leq r_c - 10^{-6}$	在最小包围圆圆心执行 /clear
清除失败	接口拒绝或超时	限次重试，否则安全退出并留档

保证全清下压缩固定成本；问题四中定向源使“距离覆盖”失效，需要新的方向覆盖条件。四问与四层的对应是：问题一提供定位证书（第四节），问题二给出第二测点选择（第五节），问题三完成全向源的覆盖与路径（第六节），问题四给出定向源的方向覆盖与搜索布局（第七节）。

3.5 四个问题之间的关系

需要强调，第三、四问是部分可观测的在线搜索控制问题：在走完所有测站之前，算法不知道还有几个源、在哪里，因此必须先回答“未发现源可能存在于哪些区域”和“何时可以安全结束”，才能借鉴路径优化的思想。

核心贡献. 全文的主要贡献可凝练为四点：**其一**，建立“发现一定位一清除”的全覆盖保证框架，把终止条件建立在连续域几何覆盖上；**其二**，证明七站布局达到距离覆盖的理论下界（6站不可能）；**其三**，用凸包充要条件刻画定向源的方向覆盖，并给出22站的连续域证书；**其四**，由时间构成诊断认定固定成本主导，据此把优化落到测站布局上。

四、 问题一：交会定位区域的直径算法与覆盖性判定

本问回答两个问题：如何用带误差的方位读数严格刻画出源的可能位置，以及据此判断能否一次清除。

4.1 从示向度读数到线性约束

设在检测点 s_k 处测得某干扰源的示向度为 θ_k ，误差界为 ε 。由假设3，真源 g 相对 s_k 的真实方位角落在 $[\theta_k - \varepsilon, \theta_k + \varepsilon]$ 内，因此 g 位于以 s_k 为顶点、张角 2ε 的前向楔形

$$W_k = \left\{ x \in \mathbb{R}^2 \mid [\mathbf{u}(\theta_k - \varepsilon), x - s_k] \geq 0, [\mathbf{u}(\theta_k + \varepsilon), x - s_k] \leq 0 \right\} \quad (6)$$

之中，其中 $[\mathbf{a}, \mathbf{b}] = a_x b_y - a_y b_x$ 。定位区域即

$$P = \bigcap_{k=1}^K W_k. \quad (7)$$

引理1：两条不等式已自动选出前向分支。当 $2\varepsilon < 180^\circ$ 时，(6) 恰好刻画以 $\mathbf{u}(\theta_k)$ 为轴、半角为 ε 的单侧锥，而不包含背向方向。

（证明见附录H。）

引理1是一个必须处理的细节：若把示向度处理成一条“带宽为 2ε 的双向直线带”，则背向的一整块区域会被错误地保留，在多站交会时可能产生虚假的交集。

把(6)的两条约束写成行向量形式， K 个检测点共给出 $m = 2K$ 条线性不等式，于是

$$P = \{x \in \mathbb{R}^2 : \mathbf{A}x \leq \mathbf{b}\}, \quad \mathbf{A} \in \mathbb{R}^{m \times 2}, \mathbf{b} \in \mathbb{R}^m. \quad (8)$$

P 是闭凸集，但不一定有界，也可能为空或退化。

4.2 直径的计算：算法与正确性

定理1：直径在顶点上取到。设 P 为有界闭凸多边形，顶点集为 $V(P) = \{v_1, \dots, v_k\}$ ，则 $\text{diam}(P) = \max_{i < j} \|v_i - v_j\|$ 。

（证明见附录H。）

由定理 1，求直径只需枚举顶点对。据此给出算法 1。它的关键是先把四类情形弄清楚，再计算：直接对约束做顶点枚举而不判空、不判无界，会在读数互相矛盾或测站张角不足时输出毫无意义的数字。

算法 1 有界误差交会定位区域的直径算法

输入： 检测点 $s_1, \dots, s_K$ ，示向度 $\theta_1, \dots, \theta_K$ ，误差界 ε

输出： 定位区域 P 的状态（空 / 无界 / 有界）、顶点表 V 与直径 D

- 1: 由 (6) 组装 $A \in \mathbb{R}^{2K \times 2}$, $b \in \mathbb{R}^{2K}$
 - 2: 解可行性规划 $\min\{0 : Ax \leq b\}$ ；不可行则返回空集并报错
 - 3: **对** $c \in \{(1, 0), (-1, 0), (0, 1), (0, -1)\}$ **循环**
 - 4: 解 $\min\{c^\top x : Ax \leq b\}$ ；无下界则返回无界， $D = +\infty$
 - 5: **循环结束**
 - 6: 枚举约束对 (i, j) ，解 $a_i^\top x = b_i$, $a_j^\top x = b_j$ 得交点，保留满足 $Ax \leq b + \tau \mathbf{1}$ 且按容差互异的点，得 V
 - 7: 以 V 的重心为极点按极角排序， $D \leftarrow \max_{v_i, v_j \in V} \|v_i - v_j\|$ （线段与单点自然退化）
 - 8: **返回** 有界， V ， D
-

算法的复杂度为 $O(m^3 + k^2)$ ；四类退化（空、无界、线段、单点）的处理、五次线性规划的用途与替代实现见附录 H。

4.3 以定位区域直径为直径的圆能否覆盖定位区域

定理 2 :覆盖性的充要条件与紧界. 设 $P \subset \mathbb{R}^2$ 为有界闭凸集, $D = \text{diam}(P)$, $R_* = R_*(P)$ 为最小包围圆半径。则

$$\frac{D}{2} \leq R_* \leq \frac{D}{\sqrt{3}}, \quad (9)$$

右端为 Jung 不等式，两端均可取等。于是

- 存在以 D 为直径的圆覆盖 P ，当且仅当 $R_* = D/2$ ；
- 一般情况下只能保证：以 $\frac{2D}{\sqrt{3}} \approx 1.1547 D$ 为直径的圆必能覆盖 P ，且该系数不可改进。

证明. 左端：取 $p, q \in P$ 使 $\|p - q\| = D$ ，任何包含 P 的圆都包含 p, q ，故其半径不小于 $D/2$ 。右端即平面情形的 Jung 定理。取等： P 为线段时 $R_* = D/2$ ； P 为边长 D 的正三角形时 $R_* = D/\sqrt{3}$ 。第一条由 $R_* \geq D/2$ 立得。□

结论：一般不能覆盖. 由定理 2，只要 $R_* > D/2$ ，以 D 为直径的圆无论圆心放在哪里都无法覆盖 P 。这是可以由真实检测点与示向度产生的具体反例：

例 1 : 三站交会给出 Jung 界上的正三角形。取三个检测点与读数

$$\begin{aligned} s_1 &= (-1100, 0), & \theta_1 &= 1^\circ; \\ s_2 &= (590, -952.6279), & \theta_2 &= 121^\circ; \\ s_3 &= (570, 987.2690), & \theta_3 &= 241^\circ, \end{aligned}$$

误差界 $\varepsilon = 1^\circ$ 。按算法 1 计算, P 为有界三角形, 顶点为 $(0, 0)$ 、 $(40, 0)$ 、 $(20, 34.641016)$, 即边长 40 m 的正三角形。此时

$$D = 40.000000 \text{ m}, \quad R_* = 23.094011 \text{ m}, \quad \frac{D}{2} = 20 \text{ m},$$

$R_*/D = 0.577350 = 1/\sqrt{3}$, 恰好取到 Jung 不等式的等号。以 D 为直径的圆 (半径 20 m) 不能覆盖该三角形, 缺口为 3.094011 m, 见图 2。

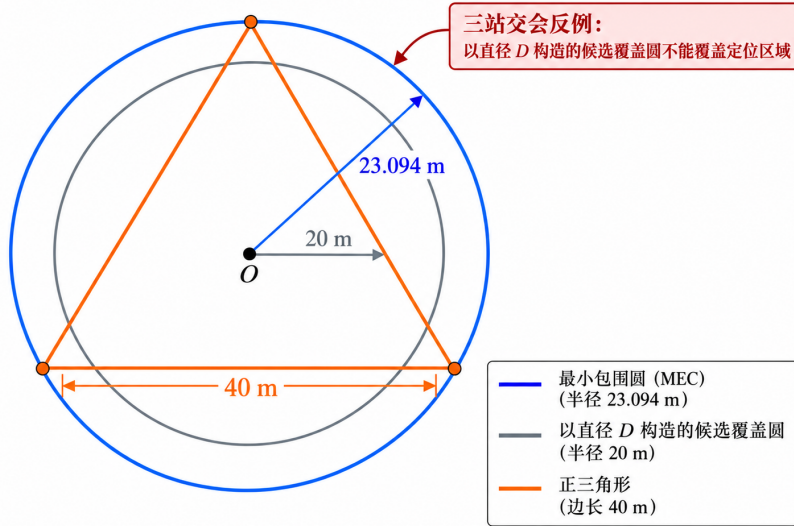


图 2 三站交会反例: $D = 40 \text{ m}$ 时以 D 为直径的圆盖不住三个角

什么时候可以覆盖: 两站交会的情形。 题目附图所示的两站交会四边形, 在两站到源的距离远大于区域尺度时趋于平行四边形, 对这一类区域同直径圆总是够用的:

引理 2 : 平行四边形的最小包围圆。 设 P 为平行四边形, 则 $R_*(P) = \text{diam}(P)/2$ (证明与数值算例见附录 H)。该结论只用到向量内积的符号, 对交会角不加限制。

由于清除成功的条件是清除点到源不超过 $r_c = 20 \text{ m}$, 而算法只知道 $g \in P$, 故有:

推论 1 : 认证清除判据。 在 $c_*(P)$ 处执行一次 /clear 必定成功, 当且仅当 $R_*(P) \leq r_c = 20 \text{ m}$; 若仅已知直径, 则 $D \leq \sqrt{3} r_c = 20\sqrt{3} \approx 34.641 \text{ m}$ 是充分条件, 且该系数不可放宽到 $2r_c = 40 \text{ m}$ 。

因此, 工程上常见的“定位区域直径小于 $2 \times 20 = 40 \text{ m}$ 就能一次清除”的说法是错误的: 例 1 中 $D = 40 \text{ m}$ 却存在距圆心 23.09 m 的可能位置。本文一律以 $R_*(P) \leq 20 \text{ m} - 10^{-6}$ 作为认证条件。

本问小结。 算法 1 已在四类退化上与解析结果核对。本问结论: 认证清除判据应取 $R_*(P) \leq 20 \text{ m}$; 本文维护源所在的可行域 P ($g \in P$), 满足该条件才清除。

五、 问题二：第二检测点的选择策略与候选区域

本问回答：第二检测点选在哪里，既保证一定收到信号，又能把定位区域压到最小。

5.1 问题的形式化与两个必须同时满足的要求

设首个检测点为 s_1 ，测得某全向源的示向度为 θ_1 。建立以 s_1 为原点、 $\mathbf{u}(\theta_1)$ 为 x 轴正向、 $\mathbf{u}(\theta_1 + 90^\circ)$ 为 y 轴正向的局部坐标系。记真源在局部坐标下为

$$g = r(\cos \delta, \sin \delta), \quad |\delta| \leq \varepsilon, \quad 0 \leq r \leq \rho \leq \rho_{\max} = 1500 \text{ m}, \quad (10)$$

其中 r 与该源的真实接收半径 ρ 都未知，只知道 $\rho \geq \rho_{\min} = 1000 \text{ m}$ ，且因首站收到了信号必有 $r \leq \rho$ 。第二检测点记为 $q = (a, b)$ ，其在原坐标下为 $q = s_1 + a \mathbf{u}(\theta_1) + b \mathbf{u}(\theta_1 + 90^\circ)$ 。

一个“好”的第二检测点必须同时满足两个要求：

要求 A（可用性）：在 q 处必须确实能收到该源的信号，否则这次移动与检测不能提供任何新的方位约束。由于 r 与 ρ 未知，这必须是一个无需未知量的充分条件。

要求 B（有效性）：两条视线的交会必须足够“横”，使得定位区域 $P = W_1 \cap W_2$ 足够小。

二者与移动代价 $\|q\|/v$ 构成三方权衡。下面分别给出要求 A 的精确刻画（定理 3）与要求 B 的定量刻画（引理 3、(14)），再由二者导出选点策略。

5.2 要求 A：保证接收的候选区域

定理 3：保证接收的候选区域。若第二检测点 $q = (a, b)$ 满足

$$C_{\text{safe}} : \begin{cases} a^2 + b^2 \leq \rho_{\min}^2, \\ a^2 + b^2 \leq 2\rho_{\min}(a \cos \varepsilon - |b| \sin \varepsilon), \end{cases} \quad (11)$$

则无论真实源距 r 与真实接收半径 ρ 取何值，在 q 处一定能收到该全向源的信号。等价地，

$$C_{\text{safe}} = B(\mathbf{0}, \rho_{\min}) \cap B(\rho_{\min} \mathbf{u}(\varepsilon), \rho_{\min}) \cap B(\rho_{\min} \mathbf{u}(-\varepsilon), \rho_{\min}), \quad (12)$$

即三个半径为 $\rho_{\min}$ 的圆盘之交。

（证明见附录 I。）

推论 2：候选区域的极值点。 C_{safe} 中横向偏移的最大值为 $|b|_{\max} = \rho_{\min} \sin 59^\circ = 857.167 \text{ m}$ （在 $a = 515.038 \text{ m}$ 处取到），前向偏移的最大值为 $a_{\max} = 1000 \text{ m}$ 。证明见附录 I。

图 3 给出了 C_{safe} 的形状及其与定位精度、移动代价的关系。

关于候选区域的三点说明。其一， C_{safe} 是充分条件，实际可用点可能更多，但那依赖未知的 r 与 ρ ，不能作为策略依据。其二，它允许 q 落在目标圆之外（附件允许）。其三，原点 $q = \mathbf{0}$ 虽满足约束但必须排除——同点重复检测不提供新约束，且交会角与点线距离公式在该点同时退化为零；实现中要求 $\|q - s_1\| \geq q_{\min} = 100 \text{ m}$ ，该门槛与共享测站规则复用。

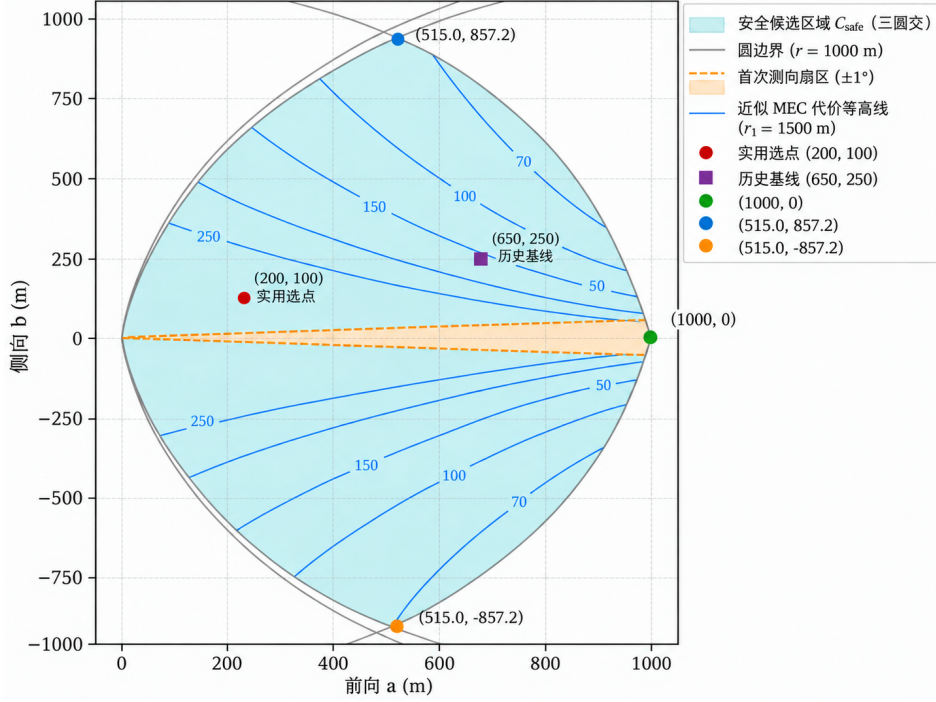


图 3 保证接收区的形状与等代价线（越靠近原点，移动越省）

5.3 要求 B：交会角与定位区域尺度

引理 3：交会角由横向偏移单独决定。设 ψ 为两条视线在真源 g 处的夹角， L 为过 s_1 与 g 的直线，则

$$\sin \psi = \frac{\text{dist}(q, L)}{\|q - g\|}, \quad \text{dist}(q, L) = |b \cos \delta - a \sin \delta| \geq |b| \cos \varepsilon - a \sin \varepsilon. \quad (13)$$

（证明见附录 I。）

引理 3 给出了本问最重要的结构性结论：交会角的下界只由横向偏移 $|b|$ （相对于第二段视距 $r_2 = \|q - g\|$ ）决定，与前进距离 a 无关。“沿着示向度方向一直往前走”对改善交会角毫无帮助，只能缩短 r_2 。

在小角一阶近似下，两条楔形在 g 附近可视为等宽条带，其交为平行四边形（引理 2），由此得到定位尺度的一阶估计

$$R_* \propto \frac{r_1 r_2}{\text{dist}(q, L)}, \quad (14)$$

即 R_* 大致正比于源距、反比于横向偏移。该式把楔形当成等宽条带、只在交会角不特别尖锐时准确，故仅用于解释趋势；其推导、适用范围以及作为结论依据的精确枚举见附录 I。

5.4 两次测向不足以保证认证清除

进一步考察两次测向能否保证认证清除：即使把第二次测向挑得更讲究， R_* 仍压不进 20 m，光学覆盖不可省略。这一点可以严格证明，无需任何近似公式。

定理 4：两次测向的分辨极限。存在两个相距 41 m 的源位置，使得无论第二检测点取在 C_{safe} 中的哪一点，都存在一组容许的测向误差，令两个源产生完全相同的两次观测。

由于 $41\text{ m} > 2r_c = 40\text{ m}$ ，不存在任何一点能同时位于两源的 20 m 清除范围内。因此，仅凭两次测向无法保证一次认证清除。

证明. 取首检测点 s_1 为原点，并考虑两个候选源

$$g_1 = (1500, 0), \quad g_2 = (1459, 0), \quad \|g_1 - g_2\| = 41\text{ m},$$

二者的接收半径都取 $\rho_{\max} = 1500\text{ m}$ 。它们相对 s_1 的真实方位角都是 0° ，故首次读数 $\theta_1 = 0^\circ$ 对两者同时相容（误差取零即可）。

设第二检测点为 $q \in \mathcal{C}_{\text{safe}} \subseteq B(\mathbf{0}, \rho_{\min})$ ，即 $\|q\| \leq 1000\text{ m}$ 。记 $\alpha(q) = \angle g_1 q g_2$ 为两源在 q 处的张角。由于 g_1, g_2 都在 x 轴上，直接计算给出：在圆周 $\|q\| = \rho_q$ 上 α 的最大值为

$$\alpha_{\max}(\rho_q) = \arctan \frac{\|g_1 - g_2\| \cdot \rho_q}{\sqrt{(\|g_1\|^2 - \rho_q^2)(\|g_2\|^2 - \rho_q^2)}}, \quad (15)$$

且 $\alpha_{\max}$ 关于 ρ_q 单调增，故在整个圆盘 $\|q\| \leq 1000$ 上的上确界在边界取到。代入数值，

$$\alpha_{\max}(1000) = \arctan \frac{41 \times 1000}{\sqrt{(1500^2 - 1000^2)(1459^2 - 1000^2)}} = 1.976940^\circ < 2\varepsilon = 2^\circ.$$

于是对任意 q ，取第二次读数 θ_2 为两条真实方位角的中点，则两个假设各自所需的误差都不超过 $\alpha_{\max}/2 = 0.988^\circ \leq \varepsilon$ ，均为容许误差。又因 $\|q - g_i\| \leq \|q\| + \|g_i\|$ 且实际计算给出两段视距均不超过 $700\text{ m} < \rho_{\max}$ ，两个源在 q 处都确实可被接收。

因此 g_1 与 g_2 产生同一组观测 $\{(s_1, \theta_1), (q, \theta_2)\}$ ，任何只依赖观测的算法都无法区分二者。若某个清除点 x 能保证成功，则必须同时有 $\|x - g_1\| \leq 20$ 与 $\|x - g_2\| \leq 20$ ，从而 $\|g_1 - g_2\| \leq 40 < 41$ ，矛盾。 $\square$

(15) 还给出一个刻画：两次测向能分辨的最小间距随源距变差，在 $\|q\| = 1000\text{ m}$ 下 $r_1 = 1500\text{ m}$ 时约为 41.5 m ，恰在 $r_1 \approx 1485\text{ m}$ 处越过 $2r_c = 40\text{ m}$ ——即只有最远的一段会“原理上不够用”。

定理 4 有三个直接后果。**其一，光学覆盖兜底不可省略：**任何“两次测向再清除”的流水线都不完备，必须保留对可行域有连续覆盖证明的清除点列。**其二，不宜盲目追求正交交会：**远源反正要进入光学覆盖，为降低 R_* 而多走数百米往往不划算。**其三，近源应优先获得第二次测向：**由一阶估计 $R_* \propto r_1 r_2$ ，对近源两次测向常常够用——以 $(200, 100)$ 为第二点时，首测距离在 380 m 以内基本可一次认证清除（代表点与完整候选表见附录 I）。

5.5 精确枚举验证与选点策略

为避免只依赖一阶近似，本文对 12 个候选点做了精确枚举（源距 21 个值、两测偏差各 3 个方向，共 189 个组合，逐一按算法 1 计算精确交会多边形的直径并取样本内最坏值）。完整表格与三类数字的口径说明见附录 I；结论与定理 4 一致——表现最好的候选所达最坏直径仍远大于 $r_c = 20\text{ m}$ 。

第二检测点的选择按下列顺序（综合前述要求 A、B 与移动代价）：

1. **可行性优先：** $q \in \mathcal{C}_{\text{safe}}$ ((11))，保证一定收得到信号；

2. 左右镜像取近: b 取正负两个镜像位置中综合代价更小的一侧;
3. 以完成任务所需时间为目标: 最小化

$$J(q) = \underbrace{\frac{\|q - p_{\text{cur}}\|}{v}}_{\text{走到第二点}} + \underbrace{t_m + t_s}_{\text{检测与切频}} + \underbrace{\hat{C}_{\text{clear}}(P(q))}_{\text{此后清除该源的估计代价}}, \quad (16)$$

其中 $\hat{C}_{\text{clear}}$ 为“若 $R_* \leq r_c$ 则走到圆心清除, 否则按光学覆盖点列走一遍”的保守估计。

整局视角下的第二测点选择. 在单目标更优的 (750, 450) 与较短的 (200, ± 100) 之间, 正式方案取后者: 远源反正要走光学覆盖, 多走 650 m (130 s) 换来的扫描节省有限; 多目标下同一次移动可服务多个频道 (第 6.5 节); 走远还会带来后续折返。固定种子的配对实验也显示自适应选点无改善 (见附录 I), 故保留固定 (200, ± 100), 把“是否再测一次”交给第 6.4 节的补测精化与第 7.5 节的扫描成本门控。

本问小结. 保证接收、分辨极限与精确枚举三者结论一致: 两次测向不足以保证认证清除, 光学兜底不可省略。正式方案取固定 (200, ± 100), 是否再测交给在线层。

六、 问题三: 全向干扰源的自动搜索、定位与清除

6.1 三层结构与终止证明

第三问真正困难的是“怎样证明已经没有源了”。本文把策略分成发现一定位一清除三层, 其中发现层承担终止证明的责任。

称测站集 $\mathcal{S}$ 对 $\mathcal{D}$ 具有距离覆盖, 若

$$d_{\max}(\mathcal{S}) := \max_{x \in \mathcal{D}} \min_{s \in \mathcal{S}} \|x - s\| \leq \rho_{\min} = 1000 \text{ m}. \quad (17)$$

定理 5 : 终止的正确性. 设 $\mathcal{S}$ 具有距离覆盖。若算法在每个 $s \in \mathcal{S}$ 处都对每个尚未成功清除的频道检测过一次, 并清除了由此发现的所有目标, 则 $\mathcal{D}$ 内不存在未被清除的全向干扰源。

证明. 反设频道 c 上还存在未清除源 $g \in \mathcal{D}$ 。由 (17) 存在 $s \in \mathcal{S}$ 使 $\|s - g\| \leq 1000 \leq \rho$ 。全向源覆盖角为 360° , 故在 s 处检测频道 c 必定返回 direction 或 near, 于是 g 会被发现并进入目标列表, 与“所有发现的目标均已清除”矛盾。□

定理 5 说明: 合法的停止理由只有两个——(i) 覆盖测站全部访问完毕且目标列表清空; (ii) 已成功清除 16 个源, 达到题设数量上界。“连续若干次没有收到信号”“已经清了 10 个”“20 个频道各扫过一遍”都不是有效的终止证明。

6.2 发现层：七站达到距离覆盖的理论下界

最坏覆盖距离的解析式. 由第 3.2 节，固定成本与测站数成正比，因此应在 (17) 的约束下尽量减少站数、缩短巡回。本文采用“圆心 + 半径 r 的正六边形”共 7 站的布局

$$\mathcal{S}_7 = \{0\} \cup \{r \mathbf{u}(60^\circ k) : k = 0, 1, \dots, 5\}. \quad (18)$$

定理 6： 七站布局的最坏覆盖距离. 对 $0 < r \leq \sqrt{3}R$,

$$d_{\max}(\mathcal{S}_7) = \max \left\{ \frac{r}{\sqrt{3}}, \sqrt{R^2 + r^2 - \sqrt{3} R r} \right\}. \quad (19)$$

证明. 设 $x \in \mathcal{D}$, $t = \|x\|$ 。六个环点均匀分布，故存在一个环点 s_k 使 x 与 s_k 的极角差 ϕ 满足 $|\phi| \leq 30^\circ$ ，于是 $\|x - s_k\| = \sqrt{t^2 + r^2 - 2tr \cos \phi} \leq \sqrt{t^2 + r^2 - \sqrt{3} tr} =: g(t)$ 。又圆心站给出距离 t ，故最近站距离不超过 $f(t) = \min\{t, g(t)\}$ 。

比较两支： $t \leq g(t) \iff t^2 \leq t^2 + r^2 - \sqrt{3} tr \iff t \leq r/\sqrt{3}$ 。因此 $f(t) = t$ 于 $[0, r/\sqrt{3}]$, $f(t) = g(t)$ 于 $[r/\sqrt{3}, R]$ 。前段最大值为 $r/\sqrt{3}$ 。后段 g 是关于 t 的二次根式， $g'(t) \propto 2t - \sqrt{3}r$ ，在 $t = \sqrt{3}r/2$ 取极小，故在闭区间 $[r/\sqrt{3}, R]$ 上的最大值在端点取到，为 $\max\{g(r/\sqrt{3}), g(R)\} = \max\{r/\sqrt{3}, \sqrt{R^2 + r^2 - \sqrt{3} R r}\}$ 。合并即得 (19)。上界可达： x 取 $t = r/\sqrt{3}$ 且极角恰在两环点之间（Voronoi 三重点），或 x 取目标圆边界上两环点角平分方向的点。□

推论 3： 环半径的可行区间与最优值. 取 $R = 1800 \text{ m}$ 、 $\rho_{\min} = 1000 \text{ m}$ ， $d_{\max} \leq \rho_{\min}$ 当且仅当

$$\begin{aligned} r &\in [r_-, r_+], \\ r_- &= \frac{\sqrt{3}R - \sqrt{3R^2 - 4(R^2 - \rho_{\min}^2)}}{2} = 1122.956 \text{ m}, \\ r_+ &= \sqrt{3}\rho_{\min} = 1732.051 \text{ m}. \end{aligned} \quad (20)$$

$d_{\max}(r)$ 在 $r^* = \frac{\sqrt{3}}{2}R = 1558.846 \text{ m}$ 处取最小值 $R/2 = 900 \text{ m}$ 。

为什么不取覆盖最优的 r^* . 覆盖是约束，时间才是目标。 r 越小，六个环点越靠内，巡站路线越短，而且发现目标后的第二测点、补测点与清除点也都更靠近主路线。因此本文在可行区间内取靠近下界的 $r = 1150 \text{ m}$ ，此时

$$d_{\max} = \sqrt{1800^2 + 1150^2 - \sqrt{3} \times 1800 \times 1150} = 988.5114 \text{ m} < 1000 \text{ m}, \quad (21)$$

覆盖余量 11.4886 m 。5 m 网格加边界采样的数值核对（61445 个位置）与该解析值一致。程序在启动时用 (19) 重新校验，不满足覆盖条件即拒绝运行，避免误改参数导致保证失效。

图 4 给出了该布局的覆盖示意与 $d_{\max}(r)$ 曲线。实验上， r 取 1250, 1300, ..., 1450 的扫描全部慢于 1150；而 $r < 1122.956$ 会直接破坏覆盖保证。也就是说， $r = 1150 \text{ m}$ 已经贴着可行下界，这一维参数上不再有可挖掘的空间。

七站是保证发现的最小站数.

定理 7：站数下界. 在 $R = 1800\text{ m}$ 、 $\rho_{\min} = 1000\text{ m}$ 的条件下，任何具有距离覆盖的测站集至少包含 7 个站；(18) 取 $r \in [r_-, r_+]$ 时达到该下界。

证明. (17) 等价于用 $|S|$ 个半径 $\rho_{\min}$ 的闭圆盘覆盖半径 R 的圆盘，即覆盖半径比为 $R/\rho_{\min} = 1.8$ 。Bezdek 证明了 6 个单位圆盘所能覆盖的圆的最大半径为 $1.7988... < 1.8[1, 2, 11]$ ，故 $|S| \leq 6$ 不可能；而 S_7 在 $r \in [r_-, r_+]$ 时满足覆盖，故 7 可达。□

为了从反面确认这条下界是紧的，并顺带回答“少一个站到底能省多少”，本文另外实现了一个不保证覆盖的六站实验入口，逐布局结果见附录 F 的表 15。

对六站布局做过 Nelder–Mead 极小极大优化，最坏未覆盖距离最低只能压到约 1003 m （已接近理论值 1000.7 m ），仍然越过 1000 m 这条线。更值得注意的是收益侧：去掉一个站只省下约 6 s/源 的扫描，却因为失去圆心站而使首次发现更远、清除路程更长，移动反而增加约 4 s/源 ，净收益仅 $1 \sim 2\text{ s/源}$ ，代价是引入约万分之二的单源漏检概率。在“清除率是硬约束”的准则下这笔交换不划算，正式方案因此坚持七站；这也说明第三问的发现层已经贴着几何下界。

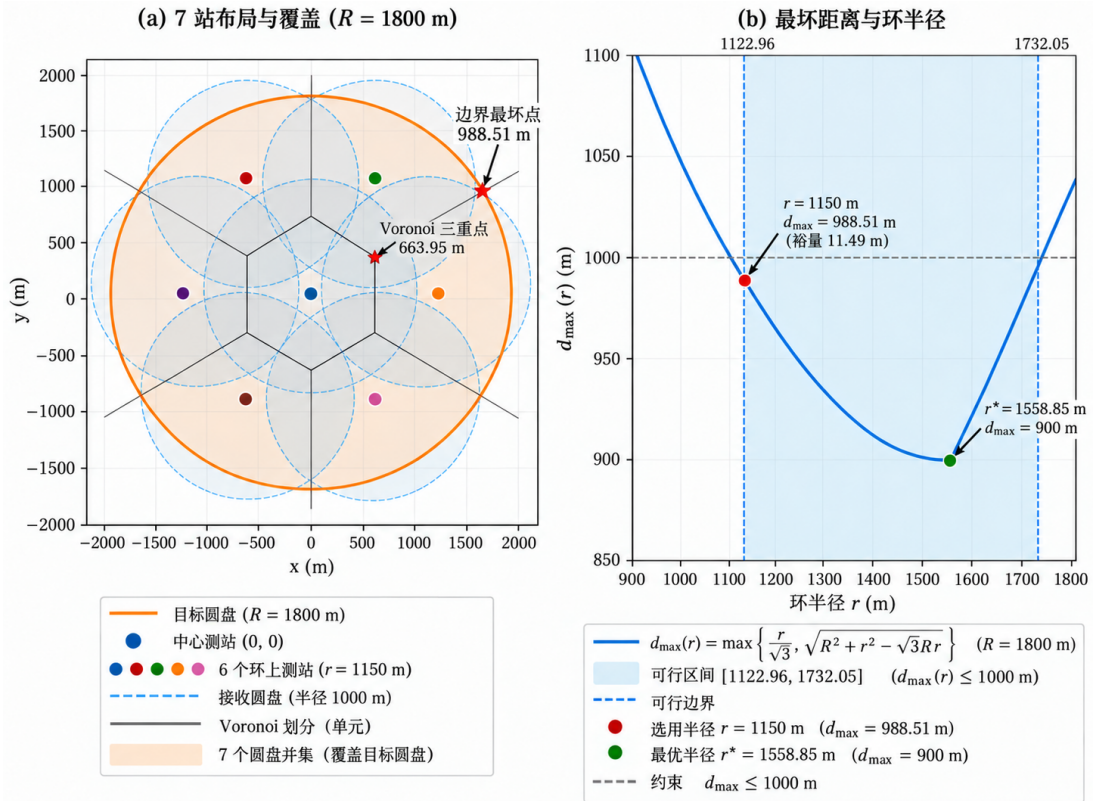


图 4 七站布局的覆盖证明与环半径的可行区间

6.3 定位层：有界误差可行域

对每个已发现频道 c 维护一个凸可行域 P_c ，其不变量是

$$(\text{包含性不变量}) \quad g_c \in P_c \text{ 恒成立.} \quad (22)$$

初始化. 首次收到 `direction` 时, 以检测点 p 为顶点构造包含真实扇区的三角形外包

$$P_c^{(0)} = \text{conv}\{p, p + \rho_{\max} \mathbf{u}(\theta) - w\mathbf{n}, p + \rho_{\max} \mathbf{u}(\theta) + w\mathbf{n}\}, \quad w = \rho_{\max} \tan \bar{\varepsilon}, \quad \mathbf{n} \perp \mathbf{u}(\theta), \quad (23)$$

它外包了半径 $\rho_{\max}$ 、半角 $\bar{\varepsilon}$ 的扇形, 因此无需猜测源距。

新方位裁剪. 每获得一条新的有效示向度, 按 (6) 生成两条半平面并求交: $P_c \leftarrow P_c \cap W$ 。若结果为空, 说明模型假设与观测相矛盾, 程序报错并保留日志后停止。

目标圆外包. 所有源都在 $\mathcal{D}$ 内, 故可再与目标圆求交。为保持多边形表示, 本文用 48 条支撑半平面 $\mathbf{n}_j^\top x \leq R + 10^{-7}$ 作为圆的外切近似。这里必须外切而不能内接: 内接多边形会切掉靠近圆边界的一圈真实位置, 可能让边界源脱离 P_c , 破坏 (22)。

无信号的距离排除 (仅第三问). 这是第三问相对第四问独有的信息优势。

定理 8 : 无信号排除. 第三问中所有源都是全向源。若在点 s 检测频道 c 返回 `no_signal`, 而频道 c 上仍存在未清除的源 g_c , 则 $\|s - g_c\| > \rho_c \geq \rho_{\min}$, 即 $g_c \notin B(s, \rho_{\min})$ 。

于是可令 $P_c \leftarrow \text{conv}(P_c \setminus B^\circ(s, \rho_{\min}))$ 。注意 $P_c \setminus B^\circ$ 一般非凸, 本文取其凸包作为保守外近似: 这可能重新包含一小部分已被排除的区域 (约束变松), 但绝不会误删真源。实现中排除半径取 $\rho_{\min} - 10^{-4}$, 为浮点计算留余量。

一次必须记录的失败. 该约束首轮验证时曾有一例证书失效: 整局源已全部清除, 程序内部却报出“可行域排除了真源”, 根因是浮点判断把一个边界交点误判为落在圆盘内部, 丢弃了边界顶点。修复与回归测试见附录 C。它带来一条方法论主张: “**结果全部清除**”替代不了对中间几何不变量的检查。图 5 展示了可行域在上述四类约束下逐步收缩、直至可以认证清除的完整过程。

6.4 清除层: 认证清除、光学兜底与补测精化

认证清除. 由推论 1, 当 $R_*(P_c) \leq r_c - 10^{-6}$ 时, 在最小包围圆圆心 $c_*(P_c)$ 处执行一次 `/clear` 必定成功。最小包围圆按“1、2 或 3 个支撑顶点”穷举候选圆心并逐一校验, 凸多边形只需检查顶点。

光学覆盖兜底. 若 $R_* > r_c$, 用边长 $h_{\text{opt}} = 25 \text{ m}$ 的方格覆盖 P_c , 保留所有与之相交的格, 在各格中心依次尝试清除。正确性来自

$$\frac{h_{\text{opt}}}{\sqrt{2}} = \frac{25}{\sqrt{2}} = 17.6777 \text{ m} < 20 \text{ m} = r_c, \quad (24)$$

即格内任意点到格心的距离都在清除半径内; 真源必在 P_c 内而 P_c 被这些格覆盖, 故必有一格成功。遍历走按列蛇形路径, 并按当前位置选择从哪一端进入。

补测精化. 认证清除与光学覆盖之间存在一段过渡区间: R_* 只比 20 m 略大 (比如 60 m), 却可能要在上百米的网格上逐点试。为此加入圆心补测——当 $r_c < R_* \leq 300 \text{ m}$ 时先走到 $c_*(P_c)$ 再测一次, 这段移动与后续清除路径高度重合, 边际成本很低; 若仍不够, 就沿新方位的垂线横移 60 m 补测第二次, 此后才进入光学网格。100 例留出验证:

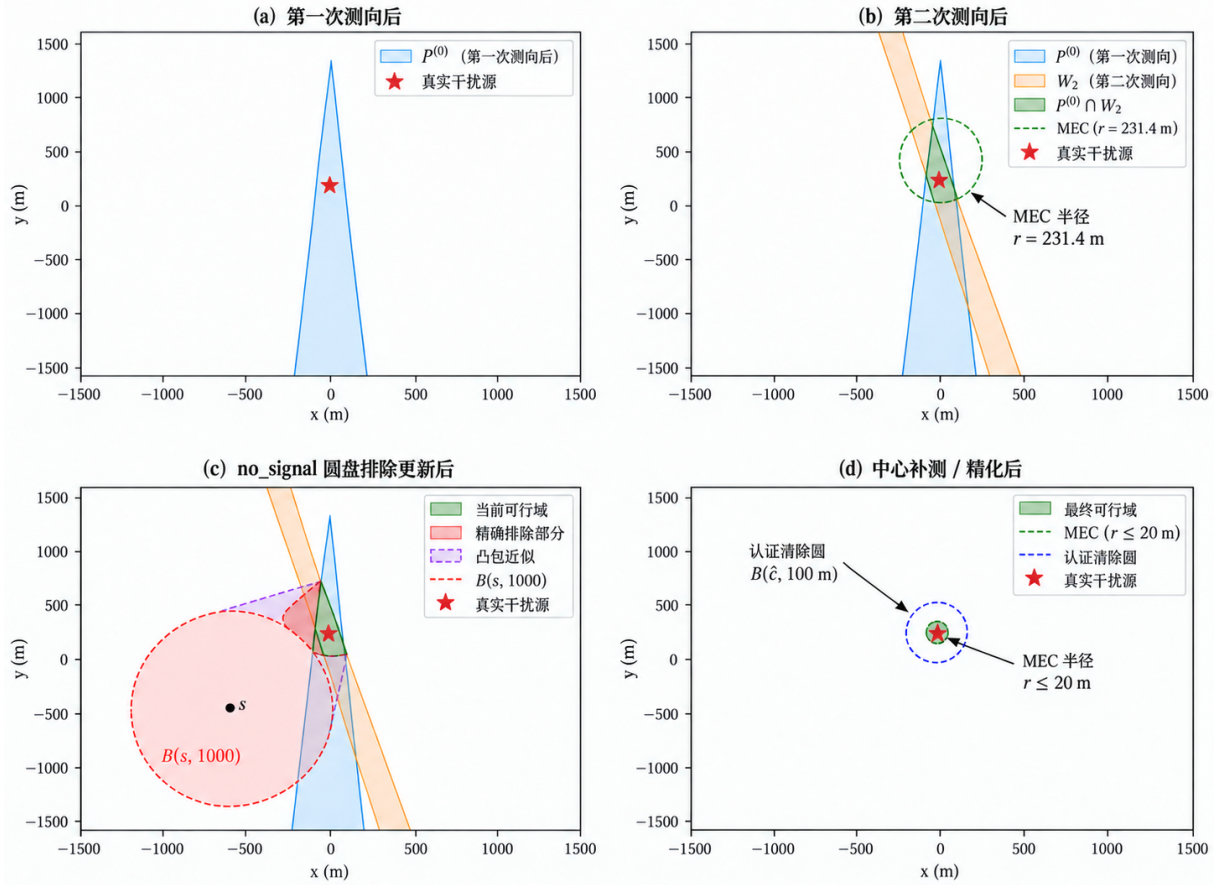


图 5 有界误差可行域的收缩过程

平均 $262.6 \rightarrow 260.0$ s/源，每源失败清除次数由 3.00 降到 0.38，64 例更快、36 例更慢、最大单例退步 30.9 s/源。

6.5 调度层：搜索与目标处理的混合贪心

早期版本采用“到一个站 → 扫完 → 处理本站发现的全部目标 → 再去下一站”的分批流程，会产生大量折返。当前版本把搜索任务与目标任务放进同一个任务池，每一步比较：

- 去最近未访问测站的代价： $\|s^* - p_{\text{cur}}\|$ ；
- 处理待清除目标 c 的代价：

$$J(c) = \begin{cases} \|q_c - p_{\text{cur}}\| + \|q_c - c_*(P_c)\|, & \text{还需补一条方位,} \\ \|x_c - p_{\text{cur}}\|, & \text{已可直接处理,} \end{cases} \quad (25)$$

其中 q_c 为第二测点， x_c 为 near 点或最小包围圆圆心 $c_*(P_c)$ ；取二者中较小者执行。这样机器狗可以“顺路”经过新的测站，也可以在靠近时提前把近处目标处理掉。目标状态跨测站保留，不再每到一站就清空。

共享测站。 一次移动应尽量服务多个目标：到达第二测点或完成清除后，算法顺带检查其他待处理频道，若当前位置对该频道确有观测价值就补测一次。是否“有价值”由一组阈值界定（见附录 G）；这些阈值都是选择动作的启发式，不提供接收保证，也不参

与正确性论证。

6.6 算法总览

把上述四层合在一起，得到算法 2。

算法 2 第三问：七站覆盖 + 混合贪心的在线搜索、定位与清除

输入： 目标圆半径 R ，接收半径下界 $\rho_{\min}$ ，误差界 $\bar{\varepsilon}$ ，环半径 $r = 1150\text{ m}$

输出： 全部干扰源被清除

- 1: 调用 /enter; 由 (18) 生成 $\mathcal{S}_7$ 并按 (19) 校验覆盖，不满足则拒绝运行
 - 2: **当** $\mathcal{S}_7 \neq \emptyset$ **或** $\mathcal{Q}$ 中仍有未清除频道**时循环**
 - 3: $c^* \leftarrow \arg \min_{c \in \mathcal{Q} \setminus \mathcal{C}_{\text{done}}} J(c)$ ((25)), $s^* \leftarrow \arg \min_{s \in \mathcal{S}_7} \|s - p_{\text{cur}}\|$
 - 4: **若** $\|s^* - p_{\text{cur}}\| < J(c^*)$ **则**
 - 5: 走到 s^* 并从 $\mathcal{S}_7$ 移除，对每个未清除频道依次检测（当前频道优先）
 - 6: 更新或新建各频道可行域 P_c （新建时施加定理 8 与目标圆约束）
 - 7: **否则**
 - 8: 必要时走到第二测点 $q_{c^*} = \text{origin} + 200 \mathbf{u}(\theta) \pm 100 \mathbf{n}$ 检测，并做共享测站扫描
 - 9: 圆心补测精化；若 $R_*(P_{c^*}) \leq r_c - 10^{-6}$ 则在圆心清除，否则按 25 m 方格蛇形遍历逐格尝试
 - 10: **判断结束**
 - 11: **若** $|\mathcal{C}_{\text{done}}| = 16$ **则 break** ▷ 题设数量上界
 - 12: **判断结束**
 - 13: **循环结束**
 - 14: 调用 /exit
-

虚拟时间与运行时间都有充裕余量。 按最坏情形可以证明整局虚拟时间不超过 $6.2 \times 10^4\text{ s}$ ，远小于题设上限 $3.6 \times 10^5\text{ s}$ （推导见附录 D），因此算法不会因死循环被强制终止。实际演练中整局虚拟时间为 $3293 \pm 294\text{ s}$ ，现实程序运行时间仅 $0.4 \sim 2.2\text{ s}$ ，相对 1200 s 的限制有三个数量级的余量。

6.7 实验结果与消融

图 6 给出一局完整任务的行走轨迹，可直观看到巡站与目标处理如何交错进行。所有本地实验均使用与开发阶段不重叠的新种子做留出验证，核验项目见第九节。

第三问的改进逐轮累积：单次测向改为两次、动态处理顺序与共享测站、25 站网格改为 7 站解析覆盖、环半径 1150 加混合贪心、圆心补测精化，其中收益最大的是测站布局（47.04%），与固定成本分析一致；完整逐轮配对数据与四条被否决路线（两步前瞻、非凸可行域、自适应第二测点、主动搜索）的记录见附录 F 的表 20 与附录 E 的表 13。总体而言，用少量额外测量换取大量移动节省是本文一贯的取舍。

6.8 成本基准与剩余优化空间

把覆盖保证策略难以回避的三项开销（走遍七站、全站排除空频道、每源一次第二测点往返与一次清除）相加，得到一个用于自我衡量的成本基准。按源数分层，官方演练的分层均值比该基准高 3% ~ 7%。需要强调这是一个经验构造，用于判断“现有方案是否已贴近一条合理的成本底线”。完整表格与推导见附录 G。结合定理 7（站数不

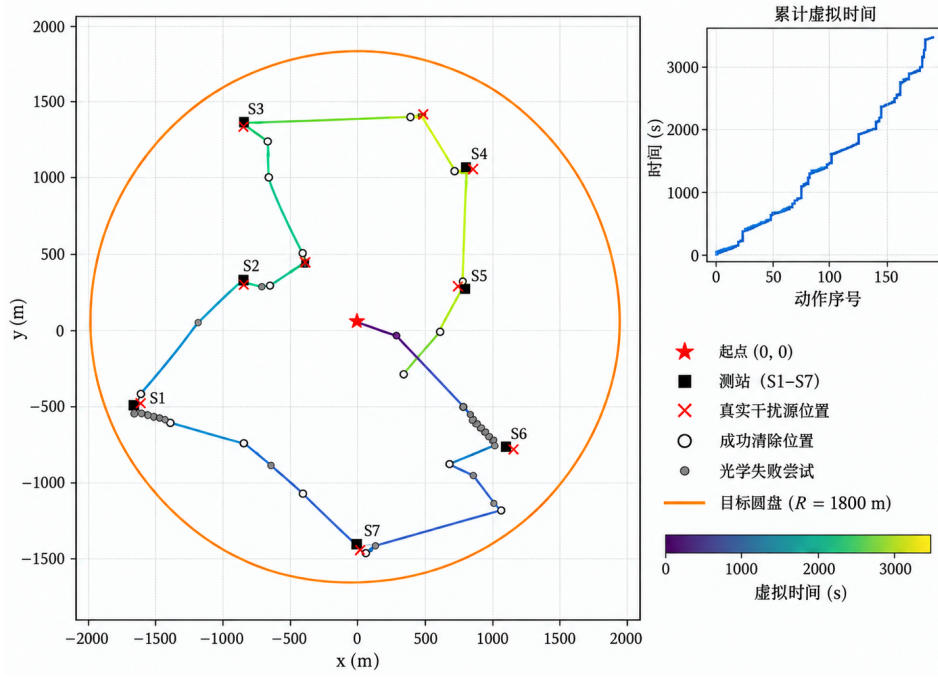


图 6 第三问典型局的行走轨迹与时间累积

能再减)与推论 3 (环半径已贴可行下界), 本文认为第三问的优化重点已从“搜索覆盖”转向“定位精度与光学覆盖开销”。

本问小结. 七站布局的解析式与 5 m 网格核对一致, 70 局演练全部清除, 发现层已贴近 Bezdek 几何下界。

七、 问题四：混合全向源与定向源的检测与清除

7.1 第三问的覆盖条件为什么不能照搬

第四问中每个源可能是定向源, 其有效覆盖范围是以定向方向 d 为轴、张角 180° 的闭半平面。于是“可见”的判据从单纯的距离变成距离与方向的合取:

$$\text{测站 } s \text{ 能发现位于 } g \text{、朝向 } d \text{ 的源} \iff \|s - g\| \leq \rho \text{ 且 } d^\top(s - g) \geq 0. \quad (26)$$

这带来两个本质变化。其一,“附近有一个站”不再足够: 离源最近的那个站完全可能落在天线背面。其二, `no_signal` 不再携带距离信息——在 s 处收不到频道 c , 可能是距离超限, 也可能只是背向, 因此定理 8 的圆盘排除在第四问不再适用, 必须在实现上严格隔离。这是两问算法最重要的一处不对称。为方便叙述, 记

$$\mathcal{N}_\rho(g) = \{s \in \mathcal{S} \mid \|s - g\| \leq \rho\}. \quad (27)$$

7.2 方向覆盖的充要条件

定理 9 : 凸包等价条件. 位于 g 的源对任意未知朝向 d 都存在可见测站, 当且仅当

$$g \in \text{conv } \mathcal{N}_\rho(g). \quad (28)$$

证明. (充分性) 设 $g \in \text{conv } \mathcal{N}_\rho(g)$, 则存在 $s_1, \dots, s_k \in \mathcal{N}_\rho(g)$ 与 $\lambda_i \geq 0$, $\sum \lambda_i = 1$, 使 $g = \sum \lambda_i s_i$ 。于是对任意单位向量 d ,

$$\sum_i \lambda_i d^\top (s_i - g) = d^\top \left(\sum_i \lambda_i s_i - g \right) = 0.$$

一组和为零的加权项不可能全部严格为负, 故存在 i 使 $d^\top (s_i - g) \geq 0$; 又 $\|s_i - g\| \leq \rho$, 由 (26) 该站可见。

(必要性) 设 $g \notin \text{conv } \mathcal{N}_\rho(g)$ 。 $\text{conv } \mathcal{N}_\rho(g)$ 是紧凸集, 由凸集分离定理存在单位向量 d 与 $\alpha > 0$ 使 $d^\top (s - g) \leq -\alpha < 0$ 对一切 $s \in \mathcal{N}_\rho(g)$ 成立。取该 d 为定向方向, 则所有距离足够近的站都落在天线背面, 而距离超过 ρ 的站本来就收不到, 故无可见站。 $\square$

取 $\rho = \rho_{\min} = 1000 \text{ m}$, (28) 对区域内每一点成立即给出保证发现的**充要条件**。

与三角剖分条件的比较. 早期版本 (49 站方格、27 站三角网格、25 站双环) 用的是一个充分条件: 把区域剖分成直径小于 $\rho_{\min}$ 的单元, 源必落在某个单元内, 而单元顶点既在接收范围内, 又由凸组合关系保证至少一个位于前发射半平面。这要求相邻站之间的距离都小于 $\rho_{\min}$, 站数因此下不去。定理 9 放松了这一点: $\mathcal{N}_\rho(g)$ 中的站**不必两两相邻**, 只要 g 落在它们的凸包内即可。正是这一差别让测站数从 25 减到 22——当前布局中内环 (990, 0) 与外环 (1616, 933) 相距 $1123.6 \text{ m} > 1000 \text{ m}$, 任何基于剖分的论证都覆盖不了它, 而凸包条件依然成立。

7.3 连续域的网格证书

(28) 是对连续域中每一点的要求, 不能靠有限采样“验证”。本文用一个带余量的网格证书把连续域问题化为有限检查。

定理 10 : 网格证书. 取网格步长 h , 令 $\delta = h/\sqrt{2}$ 。若对覆盖 $\mathcal{D}$ 外扩 δ 的每一个网格点 g 都有

$$B(g, \delta) \subseteq \text{conv } \mathcal{N}_{\rho-\delta}(g), \quad (29)$$

则 $\mathcal{D}$ 中任意一点 p 都满足 $p \in \text{conv } \mathcal{N}_\rho(p)$ 。

证明. 任取 $p \in \mathcal{D}$, 设 g 为距 p 最近的网格点, 则 $\|p - g\| \leq h/\sqrt{2} = \delta$ 。其一, 对任意 $s \in \mathcal{N}_{\rho-\delta}(g)$, 有 $\|s - p\| \leq \|s - g\| + \|g - p\| \leq (\rho - \delta) + \delta = \rho$, 故 $\mathcal{N}_{\rho-\delta}(g) \subseteq \mathcal{N}_\rho(p)$, 从而 $\text{conv } \mathcal{N}_{\rho-\delta}(g) \subseteq \text{conv } \mathcal{N}_\rho(p)$ 。其二, $p \in B(g, \delta) \subseteq \text{conv } \mathcal{N}_{\rho-\delta}(g)$ 。二者合并即得 $p \in \text{conv } \mathcal{N}_\rho(p)$ 。 $\square$

(29) 中“包含一个半径 δ 的圆盘”这一要求等价于 g 到凸包各支撑超平面的距离都不小于 δ , 即余量 $\text{slack}(g) \geq \delta$, 可由凸包的超平面表示直接算出 (负值表示 g 在凸包外)。

认证结果. 布局搜索中考察过的全部候选 (含各自的余量与是否通过证书) 列于附录 F 的表 16, 最终采用的布局见表 5 与图 7。对该 22 站布局取 $h = 1 \text{ m}$ 、 $\delta = 0.7071 \text{ m}$, 检查 10 186 813 个网格点, 全部通过, **最小余量为 $1.7331 \text{ m} > \delta$** (最坏点位于 $(-1738, -471)$),

计算耗时约 12 s。运行时程序还会用 20 m 粗网格再做一次凸包检查（余量 2.417 m），测站列表不满足覆盖时拒绝运行。

7.4 22 站布局：连续域证书与减站边界

7.4.1 连续域证书与结论范围

结论的范围。可行性是严格的——表 5 给出的具体布局通过定理 10 意义下的连续域证书，对任意位置与朝向的源都保证发现；**不可行性则是受限的**——本文在“圆心 + 正内环 + 正外环”布局族内、配合局部搜索，找不到任何 ≤ 21 站的布局通过同一证书。需强调：这不排除某个不规则 21 站布局仍可行，故 22 是本文找到并认证过的最小站数（其最优性尚未证明）。

7.4.2 减站边界与工程取舍

减站的边界。三条引理说明减站会在三处碰壁：外环少于 12 站时边界弧上出现凸包无法覆盖的点；去掉圆心站会使朝外源不可见；内环少于 9 站时证书在 $(-100, 100)$ 附近出现负余量。其中只有第一条是解析结论，后两条是特定布局上的数值验证（陈述与证明见附录 J）。

表 5 第四问采用的 22 站认证布局

组成	站数	半径/m	角度
圆心站	1	0	—
内环	9	990	$0^\circ, 40^\circ, \dots, 320^\circ$
外环	12	1866	$0^\circ, 30^\circ, \dots, 330^\circ$

最近邻巡回长度 17940.2 m（约 3588 s）；相邻内环站间距 677.1 m；相邻外环站间距 966.0 m。

工程决策：22 站与 25 站的取舍。综合以上，本文以 22 站为默认：它在题设参数下通过严格连续域证书，且巡站路径更短；25 站解析布局作为保守备用保留。程序在启动时用 20 m 粗网格复核，若当前参数下证书不满足（例如接收半径下界被调低到 990 m，22 站约 6 ~ 10 m 的余量消失），则拒绝使用 22 站并回退到 25 站。题目明确允许机器狗走出目标区域，故外环站与区域外第二测点都合规，且与 2000000 m 的坐标上限相距甚远。

7.5 在线层：选择性测量、有界等待与认证条带覆盖

第四问的定位层与第三问共用第 6.3 节的可行域递推与目标圆外包，但去掉定理 8 的圆盘排除。在此基础上加入下列针对定向源的改进。

选择性重测。到达一个测站后，未发现的频道必须全部检测（这是终止证明的要求，不可省略）；但对已发现的频道，重复测量常常价值很低。设当前站为 s 、目标可行域为 P_c ，只有同时满足

$$\text{dist}(s, P_c) \leq 950 \text{ m} \quad \text{且} \quad \left(\sin \hat{\psi} \geq 0.3 \quad \text{或} \quad \min_{p \in P_c} \|s - p\| \geq 300 \text{ m} \right) \quad (30)$$

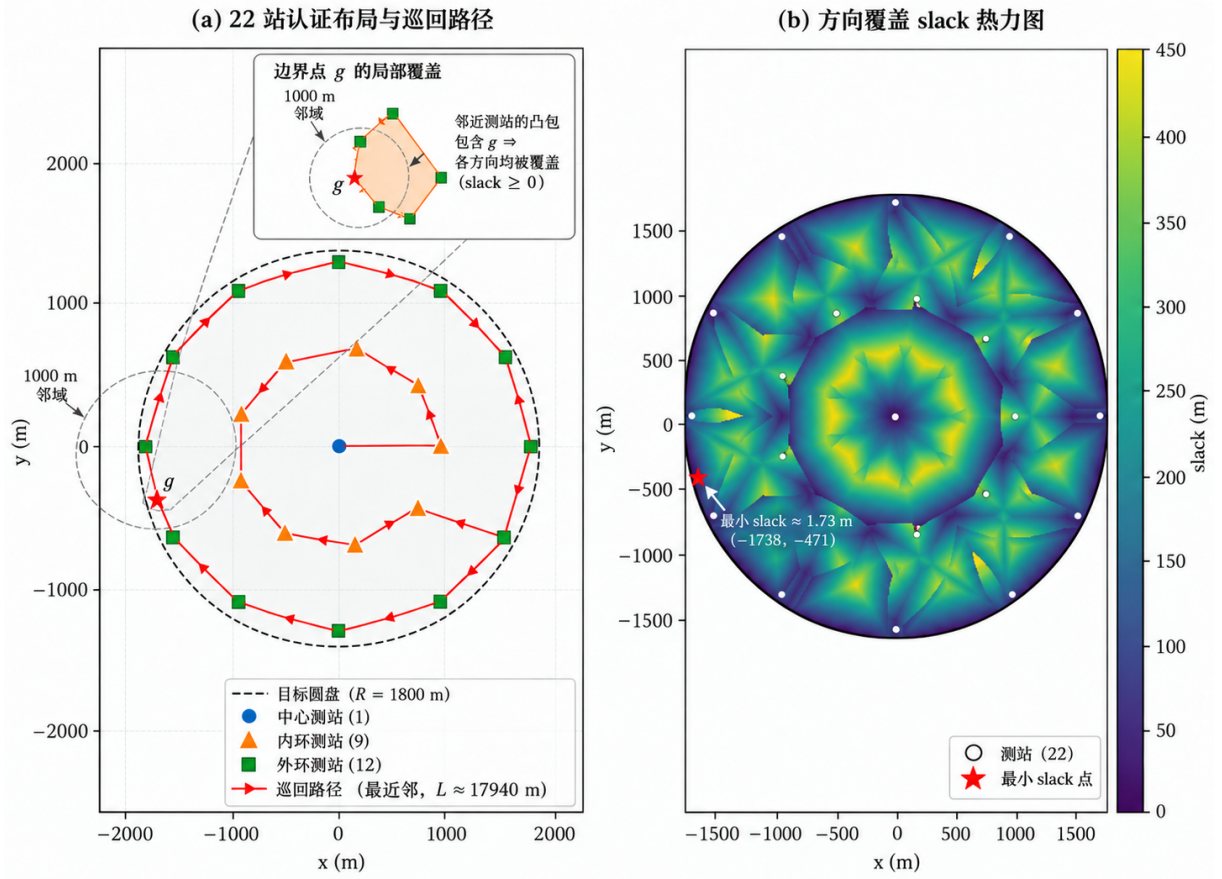


图 7 22 站布局的方向覆盖证书

时才重测，其中 $\hat{\psi}$ 是以 $c_*(P_c)$ 为顶点、由已有采样点与 s 张成的估计交会角， P_c 为已有采样点集。已经可以认证清除或已收到 near 的目标直接跳过。

有界等待与圆心补测。 若某目标只有一条方位线，而后续巡站路线上正好有测站能提供第二条方位线（距可行域不超过 950m），就可以推迟处理，等这次顺手可达的观测，省下一趟专程往返。为防止无限等待与错误等待，设了三重限制：只等一次；首次发现点距当前位置超过 1400m 时不等；在目标圆之外首次发现的目标不等——外环站能首次发现它，多半意味着该源朝外发射，后续站往往全部落在天线背面。

补测精化与第三问同理，只是阈值更紧： $R_* \in (20, 150] \text{ m}$ 时先走到 $c_*(P_c)$ 补测一次，若仍不够则沿新方位的垂线横移 60m 再补一次。

认证条带覆盖。 25m 方格兜底（(24)）只利用了清除圆的内接正方形，面积利用率为 $2/\pi \approx 63.7\%$ 。本文改用沿首测方向的条带矩形覆盖：把 P_c 沿 $\mathbf{u}(\theta)$ 切成宽 w 的条，再在每条内沿横向切成高

$$h(w) = 2\sqrt{r'^2 - \left(\frac{w}{2}\right)^2}, \quad r' = r_c - 0.1 = 19.9 \text{ m} \quad (31)$$

的矩形，取矩形中心为清除点。此时矩形四角到中心的距离恰为 $\sqrt{(w/2)^2 + (h/2)^2} = r' < r_c$ ，由凸性整个矩形都落在清除圆内，覆盖性得证。 w 在 $\{1.25, 1.5, 1.7, 1.9\} \times r'$ 中取；例如 $w = 1.5r' = 29.85 \text{ m}$ 时 $h = 26.32 \text{ m}$ ，单点覆盖面积 785.7 m^2 ，比 25m 方格的

625 m² 高出 25.7%。两种覆盖的对比见图 8。

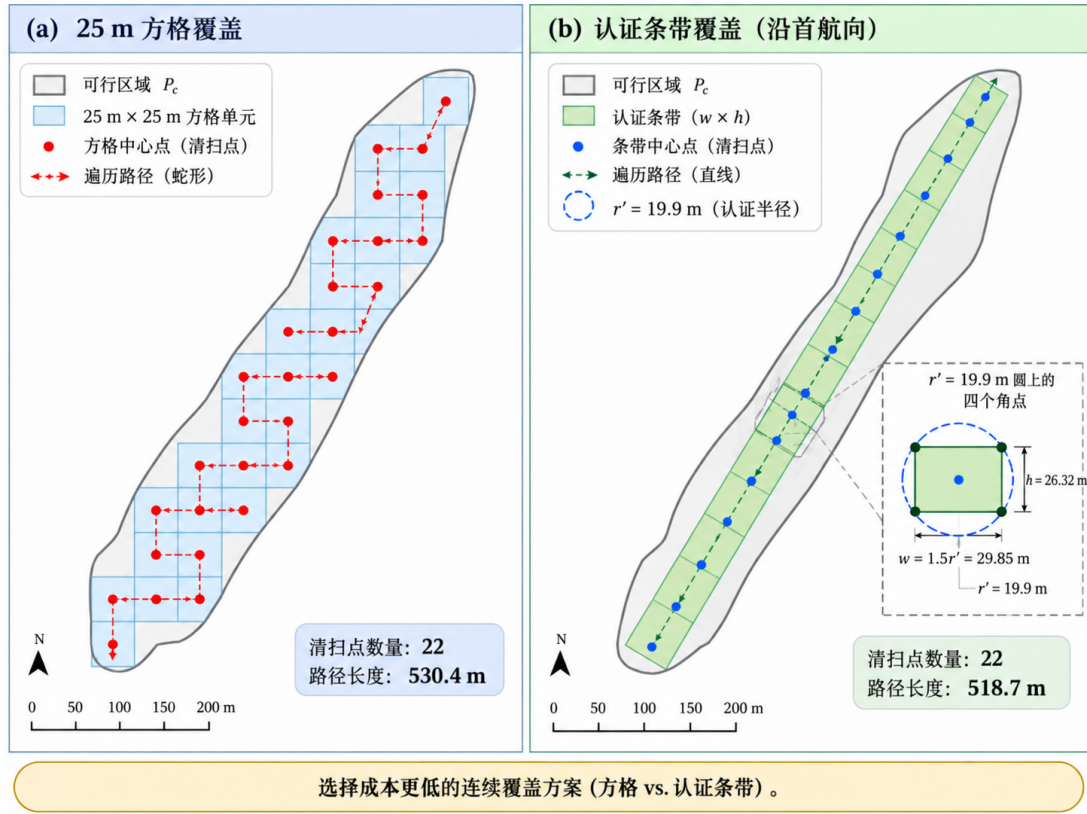


图 8 两种具有连续覆盖证明的光学清除点列

扫描成本门控与即时清除。 条带覆盖有时更慢：条带越宽，列数越少但每列越长，路径未必更短。因此算法同时生成方格路线与各个 w 对应的条带路线，按

$$\hat{C}_{\text{route}} = \frac{\|x_1 - p_{\text{cur}}\| + \sum_i \|x_{i+1} - x_i\|}{v} + |\{x_i\}| \cdot t_f \quad (32)$$

取其中最小者。同样的比较也用在补测决策上：若估计的光学覆盖成本已不高于“走到补测点再测一次”的成本，就直接进入光学清除，跳过价值不足的补测。最后，收到 near 时在当前测站立即清除，而不等本站扫描结束——真源就在 5m 之内，任何推迟只会增加往返。

7.6 实验结果

第四问历次结构性改进的配对实验见附录 F 的表 19，最后一次布局升级（25 → 22 站）的留出验证见表 6。

表 6 显示 22 站在五类场景的均值与 P_{95} 上都优于 25 站，最困难的“边界向外”“全部定向”改善最明显（30.8 与 31.2s/源）；但逐例表现有涨有跌，个别案例因发现顺序改变而变慢。本文保留的是“均值与分位数改善、清除率保持 100%”这一较弱但站得住的结论（典型局轨迹与逐例配对见图 9）。

表 6 25 站 → 22 站的留出验证（新种子 2026100000 起，与全部开发种子不重叠；220 例全部清除）

场景	案例数	25 站均值	22 站均值	25 站 P ₉₅	22 站 P ₉₅	22 站更快
随机	100	494.5	478.4	675.5	659.6	73/100
边界向外	30	588.7	557.9	725.5	683.8	24/30
最小接收半径	30	517.0	502.7	677.0	611.1	20/30
全部定向	30	606.1	574.9	776.5	726.3	26/30
圆心聚集	30	450.9	431.1	598.2	568.8	28/30

按源数分层（随机集）：10 源 666 → 642，13 源 491 → 475，16 源 361 → 346（虚拟秒/源）。

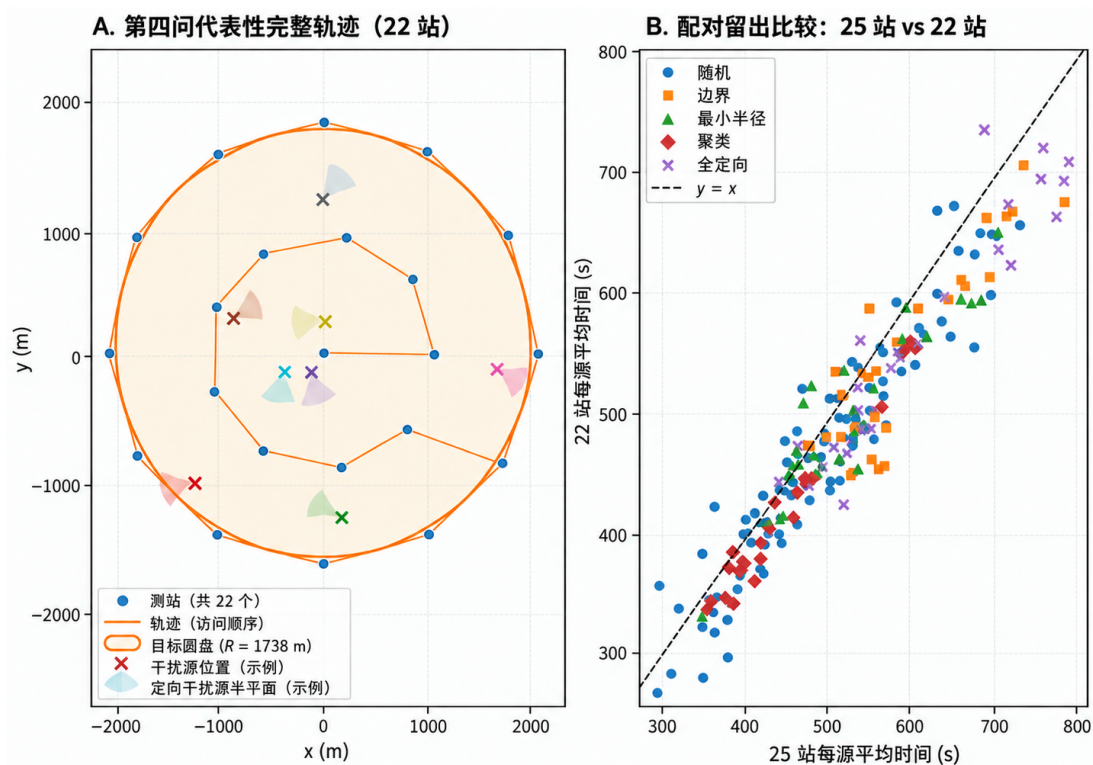


图 9 第四问典型局的行走轨迹与 25 站 / 22 站逐例配对

7.7 被否决的改进路线

本文另试过七条改进路线（绕行代价阈值调度、价值门控裁剪重复测量、运行时动态跳站等），均完成完整配对实验但未获稳定收益，完整记录见附录 E 的表 14。反复出现的教训是：只盯着某个局部指标做优化，往往会在别处付出更大代价。

7.8 成本基准与“放弃保证”的代价

把巡站、全站扫描与到达清除三项相加，得到第四问的成本基准；按源数分层，官方演练比该基准高 6% ~ 11%。需要强调这是一个经验构造，用于判断“现有方案是否已接近一条合理的成本底线”。若放弃覆盖保证，用 40 万个随机定向源做蒙特卡洛估计：16 站时每局期望漏检 0.14 个，约七局就有一局清除率低于 100%；按“清除率是硬约束”的准则，这些方案一律不采用。完整的基准公式、分层表与风险表见附录 G。

本问小结. 22 站布局通过细 / 粗两次凸包检查, 220 例留出与 70 局演练全部清除。需强调: 22 是本文找到并认证过的最小站数 (其最优性尚未证明)。

八、 演练测试与正式测试结果

8.1 测试环境与客户端

机器狗程序用 Python 实现, 通过 HTTP+JSON 与官方模拟器通信。客户端遵守附件 2 的全部要求: 以参赛队号作为 `robot_id`, 每个新动作使用新的 `request_id`, 串行发送、逐次等待响应, 网络中断后重试时复用原请求内容与原 `request_id`; 结果未知的动作标记为 `uncertain` 并禁止发出不同的后续动作, 以免重复计时或状态错位。客户端按 `/enter` 返回的剩余时长设置现实截止时间, 并同步记录逐动作日志、配置哈希与定位证书, 供事后核对。

8.2 演练样本的选取规则

演练测试不限次数, 因此日志中混有不同算法版本的记录。为避免把旧版本的成绩算进当前方案, 本文按三条规则筛选样本, 规则在看到成绩之前就已固定:

- (1) 该局状态为 `completed` 且由机器狗主动调用 `/exit` 结束——中途失败、超时或被手工中止的记录一律剔除;
- (2) 该局所用的策略源码哈希与当前提交版本一致 (第三问 `9e97786d`, 第四问 `45e328e8`);
- (3) 该局所用的配置与当前默认配置逐项一致。

按此规则, 第三问保留 70 局、第四问保留 70 局; 另有 25 局第三问记录因补测精化的参数与当前配置不同而被排除, 排除清单与原因随支撑材料一并给出。没有任何一局因为成绩不好而被剔除。

8.3 演练测试结果

演练结束后模拟器界面会显示本局干扰源总数, 逐局核对表明它与机器狗的成功清除数完全一致, 故

$$\text{被清除干扰源个数的比例} = \frac{N_c}{n} = 100\% \quad (\text{全部140局}). \quad (33)$$

汇总结果见表 7, 按源数分层的结果见表 8。逐局明细 (含每局的时间分项与动作数) 见支撑材料 `q3_runs.csv` 与 `q4_runs.csv`。

两张表合起来印证了第 3.2 节的判断。整局虚拟总时间的变异系数只有 6% ~ 9%, 而平均定位清除时间在 $n = 10$ 与 $n = 16$ 之间相差 84s/源 (第三问) 与 264s/源 (第四问) ——波动几乎全部来自分母。第四问 70 局中有 3 局在访问完 19 或 21 站时就清满 16 个源而提前退出, 这也是其总时间随源数轻微下降的原因。

8.4 本地模型与官方模拟器的一致性

本文所有版本比较都在本地合成模拟器上完成, 因此本地环境与官方环境的差距直接关系到这些结论能否外推。把第三问补测精化的 100 例留出验证与第四问的本地分层

表 7 官方模拟器演练测试汇总（时间单位：虚拟秒）

指标	问题三	问题四	指标	问题三	问题四
有效局数	70	70	平均 s/源（案例口径）	255.6	511.2
清除源总数	917	887	平均 s/源（合并口径）	251.4	500.2
清除比例	100%	100%	标准差 / 中位数	33.7 / 251.8	83.7 / 515.8
整局总时间均值	3293	6338	四分位区间	230.9 ~ 280.1	445.7 ~ 563.9
标准差 / 变异系数	294 / 8.9%	398 / 6.3%	P ₉₅	305.7	644.6
动作数	127 ~ 201	187 ~ 390	最好 / 最差	185.2 / 316.6	340.6 / 681.7
程序运行时间/s	0.4 ~ 2.2	0.7 ~ 1.1	清满 16 源提前结束	5 局	3 局

表 8 演练成绩按源数分层（平均定位清除时间，案例口径，虚拟秒/源）

源数 n	10	11	12	13	14	15	16
问题三：局数	7	13	8	14	7	8	13
均值	302.3	281.7	271.9	258.0	248.3	219.1	218.3
问题四：局数	10	11	13	12	10	10	4
均值	639.3	575.5	537.1	494.7	445.3	418.8	375.2

预测分别与官方演练的同源数结果对照，结果见附录 G 的表 26。

两问的分层对照都落在同一量级：第三问的最大偏差为 5.2%，第四问为 8.4%。第四问存在一个系统性趋势：源数越多，官方环境比本地慢得越明显，偏差从 -0.4% 升到 +8.4%；一个合理的解释是官方案例中源的空间分布比本地采用的面积均匀分布更分散，使多目标之间顺路共享观测的机会变少。第三问没有这样清晰的单调趋势（ $n = 10$ 时官方比本地快 5.2%， $n = 13, 16$ 的偏差都在 $\pm 2\%$ 以内），这与它各分层的样本量仍偏小有关。不过第四问 $n = 16$ 只有 4 局，上述趋势本身也可能部分来自抽样噪声。

因此本文对这一比较只作如下有限的表述：在已有 70 局官方演练覆盖的源数范围内，本地模型与官方模拟器给出的耗时量级一致。

图 10 把两问 140 局演练成绩按源数与时间构成做了汇总。

8.5 正式测试结果

三次正式测试的结果分别按题目表 1 的格式列于表 9 与表 10。

表 9 问题三正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
EUEH-33YM-VZW3-J36Y	11	382.95	0.56
XYWG-S6D9-PWZK-N5JT	16	178.28	0.66
NJHZ-XTXK-52DV-7CG4	16	200.83	0.58

三局合计清除 43 个源，案例口径均值 254.0 s/ 源、合并口径均值 239.0 s/ 源。

三次正式测试均在清除全部源后主动退出，第三、四问的合并平均分别为 239.0 与 498.8 s/ 源，程序运行时间 0.56 ~ 0.75 s，均远低于 1200 s 的限制。三次成绩的离散与前面的预期一致：第三问抽到 11 个源的一局为 382.95 s/ 源，而两局 16 个源分别只有

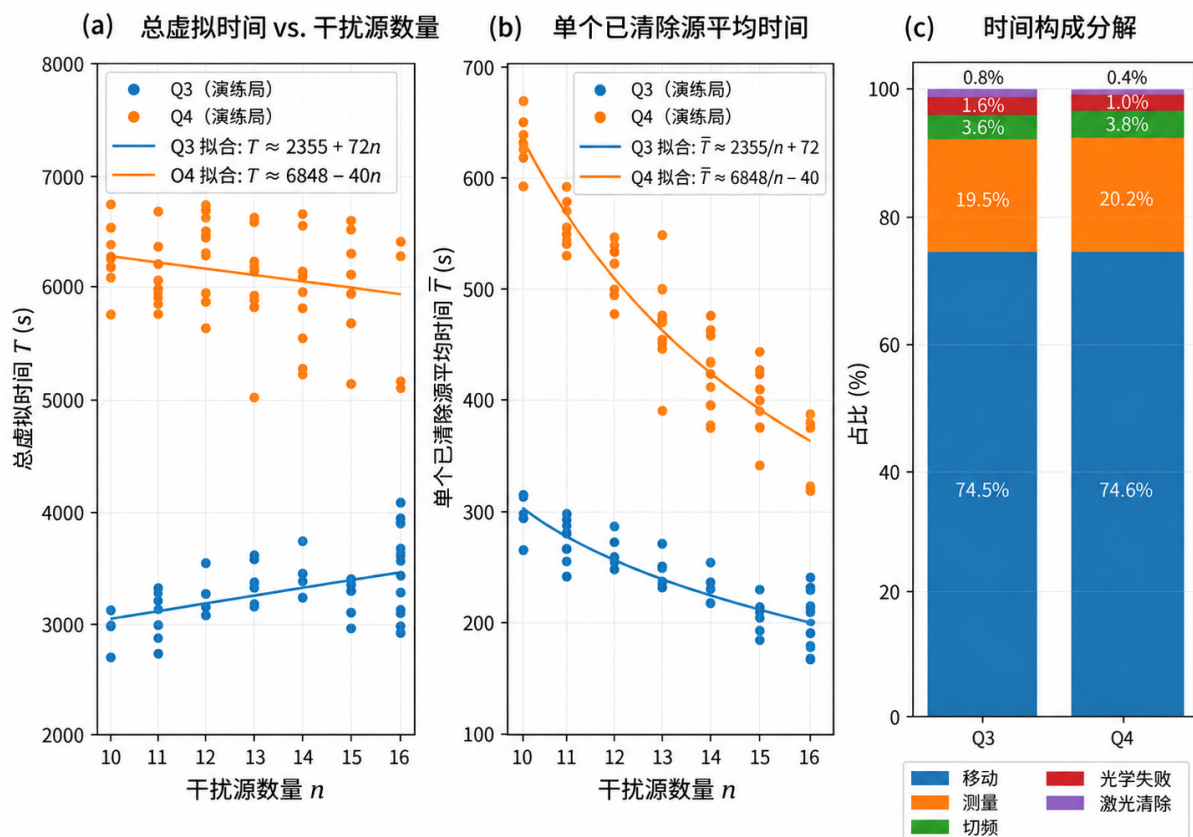


图 10 整局总时间随源数几乎不变，平均每源时间下降主要来自分母效应

表 10 问题四正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
YNET-EJBF-YMMZ-YPXU	11	514.01	0.74
3AJJ-CVE4-ZXMQ-PEKC	10	660.63	0.69
D6Z4-8ZFR-M9XA-ZWV3	15	379.64	0.75

三局合计清除 36 个源，案例口径均值 518.1 s/源、合并口径均值 498.8 s/源。

178.28 与 200.83 s/源；第四问 10、11、15 个源的三局依次为 660.63、514.01、379.64 s/源，随源数单调下降。这正是第 3.2 节“固定成本主导、平均时间随分母变化”的直接体现，因此三次成绩的差异主要反映抽到的源数。

8.6 成绩的正确解读

三次正式测试是三个独立随机案例，样本量极小，因此有必要先说清楚成绩该怎么读。**第一，三次成绩的差异主要由抽到的源数决定**：演练中整局虚拟总时间的变异系数只有 6%~9%，而平均定位清除时间在 $n=10$ 与 $n=16$ 之间相差近一半（表 8），因此本文同时报告两个指标，解读时优先看整局总时间。**第二，两种平均口径不能混用**：案例口径与合并口径的定义见 (4)，两者相差约 2%，本文所有对比都在同一口径下进行并在表注中标明。**第三，程序运行时间反映的是接口往返开销**：它在演练中只有 0.4~2.2 s（第三问）与 0.7~1.1 s（第四问），相对 1200 s 的限制有三个数量级余量，主要由 HTTP 往返次数决定，与虚拟时间没有可比性。

九、模型的检验、灵敏度分析与评价

9.1 正确性检验的内容

本文的每一次实验都包含逐例核验：清除数是否等于真实源数；每次成功清除时清除点到真源的距离是否不超过 20 m；每一时刻的可行多边形与其最小包围圆是否确实包含真源（即 (22)）；逐条动作重新累加的时间与日志时钟之差是否在 10^{-6} s 以内；测站布局是否在连续域上满足覆盖条件；各类边界与退化几何是否处理正确；候选版本有无退步案例；以及失败运行是否被独立保留。

其中第三项尤其关键。第 C 节记录的那次失败正是“清除数看似完全正确、几何证书却已经失效”的情形，它说明**结果正确并不意味着不做检查**。

9.2 灵敏度分析

对测向误差界的灵敏度。 改变实际误差幅度、同时把算法的误差界同步放宽为“幅度 + 0.005° ”，每档使用相同的种子，结果见附录 F 的表 17。该组实验做在早期的“两次测向”基线上，只用于观察趋势，其绝对数值不代表当前版本。

误差幅度加倍只带来 2%（问题三）与 5%（问题四）的时间增长，说明在误差界被正确告知的前提下，方案对误差幅度不敏感。原因是：可行域的尺度与 ε 成正比，但总时间的主体是与 ε 无关的巡站与扫描固定成本。

对误差的失配是危险的。 需要强调的是，这种“不敏感”并不意味着方案对误差界的设定可以放松；相反，误差界的失配可能导致可行域被错误裁剪。附录 F 的表 17 只说明“正确知道更大误差界”时的灵敏度。本文专门构造了真实偏差达 1.5° 而算法只容许 1.005° 的算例，结果出现正的半平面违约 12.958908 m——真源被排出了可行域，包含性不变量被破坏。这正是算法采用略宽于 1° 的保守界 $\varepsilon = 1.005^\circ$ 的原因：吸收“误差先产生、再按两位小数取整”可能带来的额外偏差。一旦正式测试中出现空交集或“认证清除失败”，正确的做法是排查取整、角度约定、坐标与计时，确认无误后再决定是否调整阈值。因此，在误差界能够被正确估计并同步调整的前提下，方案对误差幅度变化总体不敏感。

对最小接收半径的灵敏度。 这是全文最关键的敏感参数，因为两问的覆盖保证都以 $\rho_{\min} = 1000$ m 为前提。第三问相当宽裕：由定理 6，七站布局可行当且仅当 $\rho_{\min} \geq R/2 = 900$ m，题设 1000 m 有 11.1% 余量。第四问则薄得多：22 站布局在 $\rho_{\min} \geq 994$ m 时仍通过、990 m 时失效（附录 F 表 18），容忍度约 6 ~ 10 m；表中 998 ~ 1000 m 的余量不变、994 m 突降是粗网格量化所致，题设下的真实几何余量以 $h = 1$ m 细网格的 1.7331 m 为准（第 7.3 节）。值得注意的是，优化搜索得到的 22 站布局不仅比解析的 25 站更快，对 $\rho_{\min}$ 的余量也更大（约 6 ~ 10 m 对不足 2 m）；两者余量都不宽，故程序保留回退开关：删除配置中的 `q4_station_list` 即恢复 25 站布局。

对光学格距与其他阈值的灵敏度。 光学兜底要求 $h_{\text{opt}} \leq 28.2843$ m（取 25 m），条带覆盖要求外接圆半径 $\leq r_c$ （取 19.9 m），认证阈值取 $20 \text{ m} - 10^{-6}$ 。这些阈值一旦被误改超限，

覆盖保证立即失效，因此程序启动时统一校验、不满足即拒绝运行。

9.3 稳健性（压力场景）

除随机案例外，本文设计了五类压力场景：源全部贴近区域边界且取最小接收半径、源在圆心聚集、全体取最小接收半径、第四问全部为朝外的定向源、固定 $\pm 1^\circ$ 偏差与光滑空间相关偏差。表 6 给出了第四问五类场景的完整结果：220 例全部清除，22 站在每一类上都优于 25 站。第三问的压力集（70 例、890 源）同样全部清除，只有 2 例轻微退步，最大退步 2.72 s/源。

9.4 模型的优点

核心贡献有四点。第一，“一定能清完”是一条可证明的性质：定理 5 与定理 9 把终止条件建立在连续域几何覆盖上，定理 10 把它化为可执行的有限检查，推论 1 与 (24)、(31) 保证清除动作确定性成功，整条链路上没有一步靠“试几次收不到就认为没有”。第二，七站达到距离覆盖的理论下界：定理 7 证明 6 站不可能、7 站可达。第三，凸包条件刻画定向源的方向覆盖，22 站取得连续域证书：该布局在 1018.68 万点细网格上最小余量 1.7331 m，所比较布局中 ≤ 21 站均无法通过。第四，时间诊断驱动布局优化：先由第 3.2 节认定固定成本主导，再把优化落到测站布局上。此外，题目只给出误差的界，本文据此只用凸多边形交与最小包围圆，并对所有近似取保守外包，使包含性不变量单调成立。

工程优化另有若干：no_signal 圆盘排除、认证条带覆盖、共享测站扫描、圆心补测精化、有界等待与即时清除等，贯穿第六、七节。附录 E 的表 13 与表 14 保留了 11 条被否决的路线与一次几何证书失效，它们既是消融实验，也是结论可信度的一部分；覆盖保证参数在程序启动时统一校验，不满足即拒绝运行。

9.5 模型的缺点与局限

- (1) 调度层是启发式的. 混合贪心只比较一步的移动代价，没有纳入未来的测量结果与光学路径；两步前瞻的尝试反而更慢（表 13），但这不说明贪心已最优。
- (2) 第四问的 no_signal 尚未被利用. 一次“收不到”蕴含“距离超限或背向”的析取信息，可在“位置 $\times$ 朝向”的联合状态空间上做保守排除；本文原型未能胜过现行方案，这是最有价值的未完成方向。
- (3) 覆盖余量偏薄. 22 站布局对 $\rho_{\min}$ 只有约 6 ~ 10 m 的余量（表 18）；若官方对 $\rho_{\min}$ 的保证比 1000 m 更弱，覆盖可能失效。
- (4) 本地分布未必与官方一致. 表 26 显示偏差随源数从 -0.4% 升到 $+8.4\%$ ，故本地配对实验的相对结论可谨慎外推、绝对数值不能。
- (5) 异常只能安全停止. 客户端会检测超时与协议异常、记录现场并停在原地退出，但没有自动恢复能力。
- (6) 结论依赖题设的理想化物理. 静止源、无遮挡直线行走、确定性阈值、误差有界，任一被打破，包含性不变量与覆盖证明都需重建。

9.6 模型的改进与推广

针对上述局限,最值得推进的是把 `no_signal` 的析取信息纳入“位置 $\times$ 朝向”的联合证书,并在多机协同下进一步摊薄固定成本。本文框架也可推广:区域改变时只需把定理 10 的网格证书套用到新区域重新搜索布局;定向覆盖角收窄为 2β 时把凸包条件换成更强的“任意 β -锥内至少有一站”;多机协同则化为测站集划分的 min-max k -TSP。水下声源搜寻、林区火点定位等“方位有界误差、作用距离有下界、处置有作用半径”的任务也可整体移植。

最后需要说明:本文的工程价值不仅在于给出一个可运行的方案,更在于区分了严格保证、数值验证与经验估计三类结论。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具,主要用于语言润色与表述打磨、 $\text{\LaTeX}$ 排版与公式规范化、以及程序语法层面的调试;问题的分析、建模、定理证明、算法设计与全部在线模拟实验均由参赛队员独立完成,详细使用情况见支撑材料。

参考文献

- [1] K. Bezdek. Über einige Kreisüberdeckungen [J]. *Beiträge zur Algebra und Geometrie*, 1983, 14: 7–13.
- [2] K. Bezdek. Über einige optimale Konfigurationen von Kreisen [J]. *Ann. Univ. Sci. Budapest. Eötvös Sect. Math.*, 1984, 27: 143–151.
- [3] H. W. E. Jung. Über die kleinste Kugel, die eine räumliche Figur einschließt [J]. *Journal für die reine und angewandte Mathematik*, 1901, 123: 241–257.
- [4] E. Welzl. Smallest enclosing disks (balls and ellipsoids) [M]. *New Results and New Trends in Computer Science*, LNCS 555, Springer, 1991: 359–370.
- [5] J. Beardwood, J. H. Halton, J. M. Hammersley. The shortest path through many points [J]. *Mathematical Proceedings of the Cambridge Philosophical Society*, 1959, 55(4): 299–327.
- [6] R. T. Rockafellar. *Convex Analysis* [M]. Princeton University Press, 1970.
- [7] G. Fejes Tóth. Thinnest covering of a circle by eight, nine, or ten congruent circles [J]. *Combinatorial and Computational Geometry*, MSRI Publications 52, 2005: 361–376.
- [8] D. J. Torrieri. Statistical theory of passive location systems [J]. *IEEE Transactions on Aerospace and Electronic Systems*, 1984, AES-20(2): 183–198.
- [9] L. P. Kaelbling, M. L. Littman, A. R. Cassandra. Planning and acting in partially observable stochastic domains [J]. *Artificial Intelligence*, 1998, 101(1–2): 99–134.
- [10] M. Milanese, A. Vicino. Optimal estimation theory for dynamic systems with set membership uncertainty: an overview [J]. *Automatica*, 1991, 27(6): 997–1009.

- [11] R. Kershner. The number of circles covering a set [J]. *American Journal of Mathematics*, 1939, 61(3): 665–671.

A 支撑材料清单

表 11 支撑材料目录说明

文件 / 目录	内容
program/	完整可运行源程序: model.py (有界误差几何)、optimized.py (在线策略主体)、q3_policy.py、q4_policy.py、q4_optical.py、practice_robot.py (官方协议客户端), 以及 22 站方向覆盖的网格证书脚本 layout_opt.py、certify22.py; 入口 practice_launcher.py、practice_q3.py/practice_q4.py 与启动第 3/4 问演练.cmd; config.json 为全部参数与 22 站坐标, requirements.txt 为依赖, README.md 为运行说明
official_practice/ 模拟测试导出数据/	官方演练与正式测试的导出数据 q3_runs.csv、q4_runs.csv: 第三、四问各 70 局的逐局明细 (含时间分项与动作数)
正式测试导出数据/	official_runs.json, 以及第三、四问各三次正式测试的目录 (actions.jsonl 逐动作日志、run_config.json、summary.json、localization_regions.json)
正式测试日志/	formal-p3-*.jlog、formal-p4-*.jlog: 第三、四问各三次正式测试的加密行为日志, 文件名未作任何修改
AI 工具使用详情.pdf	按 2026 年 AI 工具使用规定提交的详情说明

运行方式: 安装 Python 3.10 及以上与 numpy、scipy, 进入 program/ 目录后执行

```
python practice_robot.py --question 3 --robot-id < 队号 >
--case-code < 案例编码 > --connect
```

(第四问将 --question 改为 4)。不加 --connect 时程序不会发送任何请求, 便于离线检查。

B 关键参数配置

下表汇总当前默认路径使用的参数及其来源。

表 12 算法参数及其来源

参数	取值	作用	来源
region_radius_m	1800	目标区域半径	题目
receiver_radius_min_m	1000	接收半径下界，覆盖证明的基准	题目
receiver_radius_max_m	1500	接收半径上界，初始楔形长度	题目
bearing_bound_deg	1.005	保守误差界	假设 3
clear_m	20	认证清除半径	题目
near_m	5	近场阈值	题目
optical_grid_m	25	光学方格边长， $25/\sqrt{2} < 20$	(24)
q3_survey_layout	hexagon	第三问七站布局	定理 7
q3_ring_radius_m	1150	第三问环半径，可行区间 [1122.96, 1732.05]	推论 3
q3_policy	mixed	搜索与目标混合贪心调度	第 6.5 节
q3_second_forward/lateral_m	200 / 100	第二测点偏移	第五节
q3_refine_max_m	300	圆心补测的适用上限	第 6.4 节
q3_refine_offset_m	60	补测横移距离	第 6.4 节
q4_policy	mixed_ring	第四问联合调度	第 7.5 节
q4_station_list	22 点	认证布局，删除即回退 25 站	表 5
q4_selective_remeasure	true	选择性重测	(30)
q4_remeasure_min_sine	0.3	重测的最小交会角正弦	(30)
q4_remeasure_min_gap_m	300	重测的最小测点间距	(30)
q4_station_help_m	950	判定测站对目标是否有价值	(30)
q4_wait_for_survey / q4_wait_mode	true / inner	有界等待，仅限内环发现的目标	第 7.5 节
q4_wait_stale_m	1400	超过该距离不再等待	第 7.5 节
q4_refine_max_m	150	第四问补测的适用上限	第 7.5 节
q4_strip_cover	true	认证条带覆盖	(31)
q4_scan_cost_gate	true	扫描成本门控	(32)
q4_immediate_near	true	收到 near 立即清除	第 7.5 节

config.json 中另有若干历史参数（如通用的 second_forward_m=650、second_lateral_m=250 以及 q3_grid_m、q4_grid_m、q4_triangle_spacing_m）。它们不被当前默认路径读取；第三、四问的第二测点分别取 q3_、q4_ 前缀的同名参数。保留这些条目是为了能一键回退到历史对照版本（例如置 q3_policy=legacy 或删除 q4_station_list），复现旧轮次实验时仍需要它们，因此没有删除。

C 调试记录：一次边界顶点的浮点失稳

在第三问圆盘排除约束的首轮验证中，种子 2031091108 的案例报出定位证书异常 (excluded true source by 9.7374m)：该局虽然最终清除了全部 15 个源，但中间的几何证书已经失效。根因是重复施加圆盘排除时，一个理论上位于边界的交点被浮点判为落在圆盘内部，相应二次方程的根又略微超出线段参数范围，导致边界顶点被错误丢弃，多次裁剪后可行域出现不保守的收缩。修复后改用平方距离容差保留边界顶点、对接近端点的根做范围容许与夹取、点数不足或凸包退化时保留输入多边形，并补充了重复约束的回归测试。本例说明“结果全部清除”不能替代对中间几何不变量的检查（正文第 6.3 节）。

D 虚拟时间上界的推导

所有行动点都在以原点为心、2000 m 为半径的圆内，任意两点距离不超过 4000 m。7 个测站每站至多 20 次含切频的检测 (120 s)；至多 16 个目标，每个目标至多两段全局

跳转、一条保守长度 6500 m 的光学路线、若干次检测与至多 183 次清除尝试¹。于是

$$T \leq 7\left(\frac{4000}{5}+120\right)+16\left[\frac{2 \times 4000+6500}{5}+6 \times 6+183 \times 3+5\right] \approx 6.2 \times 10^4 \mathrm{~s} \ll 3.6 \times 10^5 \mathrm{~s}, \tag{34}$$

远小于虚拟时间上限，也说明算法不可能因死循环而被强制终止。实际演练中整局虚拟时间为 $3293 \pm 294 \mathrm{~s}$ ，第三问的现实程序运行时间仅 $0.4 \sim 2.2 \mathrm{~s}$ ，相对 $1200 \mathrm{~s}$ 的限制有三个数量级的余量。

E 被否决改进路线的完整记录

下列两表汇总了第三、四问中完成了完整配对实验、但最终未进入正式方案的全部改进路线。每一行都使用与开发阶段不重叠的留出案例、同一误差场做配对比较，清除率均为 100%。保留这些记录的目的有两个：它们构成本文的消融实验，说明最终算法的每一项选择都有对照；同时也提醒读者，平均值改善并不足以作为采用一项改动的充分理由。逐条完整数据与逐例记录随支撑材料提交。

表 13 第三问被否决的改进方案（均完成完整配对实验，清除率均为 100%）

方案	对照 s/源	候选 s/源	否决原因
两步前瞻调度：选动作时同时计入下一步的最优代价， $J(a)=C(p, a)+\lambda \min_b C(a, b)$	263.920	292.795	总移动距离由 12583 升至 14452 m， P_{95} 同步恶化；复杂度上升而性能下降
非凸可行域：把 no_signal 排除掉的圆盘真的“挖掉”，维护非凸单元集合	263.920	264.943	单元碎片化，平均光学格数由 100.1 升至 102.3，区域缩小没有转化为更短路径
自适应第二测点：在 5×4 个离散偏移中按收益选点	263.920	265.518	均值与最坏值都变差，详见第五节
主动搜索：按“新增覆盖收益除以移动代价”自行选择下一测站	251.70	260.95	提前减少搜索后，已发现目标拿到的方位不足，光学成本反而上升

F 灵敏度分析的两张数据表

本附录汇总正文中因篇幅而移出的数据表：表 15 为第三问六站布局的实验记录，表 16 为第四问布局搜索考察过的候选，另两张表是第九节灵敏度分析所依据的原始数据。

表 14 第四问被否决的改进方案（均完成完整实验，清除率都是 100%）

方案	实验结果与否决原因
绕行代价阈值调度	只有当“插入该目标所多走的路” $d(x, g) + d(g, s) - d(x, s)$ 低于阈值时才立即处理，否则推迟到收尾阶段。在 27 站与 25 站上各做 1050 次仿真，最好的阈值使平均时间恶化 8.54% 与 9.15%，100 个随机案例中 89 个退步。返回主路线的移动确实降了 16%，但光学扫描移动增加 49.7%、目标测量移动增加 88.0%。
按价值门控裁剪重复测量	测量次数减少 14.36%、平均时间改善 3.35%，但光学扫描距离增加 39%，最坏案例多花 296.49 s。裁剪过度会让少数困难源失去关键方位而被迫进入长距离搜索。
为上一条加装风险保护	先检查预计光学路径长度与尝试次数，超限则强制测量。最佳参数把光学距离增幅压到 2.17%、平均改善 1.72%，却出现新的最坏案例（多花 950.86 s）。 平均值改善抵不过尾部风险，整条路线放弃。
固定巡回路线与两交换改进	当前“每次去最近未访问站”的路线长 18233.1 m；最近邻加两交换、100 个多起点加两交换给出的都是同样的数值，现有路线已与这些方法找到的最好解持平。
低价值测站逐一删除	在 1120 个案例上统计每站的发现贡献与到达成本，再逐站做留一实验。虽有若干外环站平均价值偏低，但每个外环站都能构造出独占方向覆盖的反例，没有任何站拿得到连续域的安全删除证书。
运行时动态跳站	只在“剩余测站仍能对所有未发现频道提供完整覆盖证明”时才允许跳过。1000 个案例中安全证书授权的跳站次数为 0：证书足够安全但过于保守，没有实际收益。
主动搜索	用保守单元维护位置-方向联合证书，按覆盖收益自行选站。原型曾有“借助真实源数决定补测”与“用有限探针代替连续证明”两处研究有效性缺陷，修正后仍未胜过现行调度。

表 15 六站布局实验（独立入口，默认方案未采用）

布局	最坏未覆盖距离/m	单源漏检概率	120 例均值 s/源	与 7 站配对差	更快局数
7 站（默认）	988.5（有保证）	0	258.6	—	—
6 站环 960 m	1102.0	$\approx 4 \times 10^{-4}$	256.1	-2.5	69/120（漏 1 源）
6 站环 1000 m	1062.9	$\approx 2 \times 10^{-4}$	257.2	-1.4	61/120（无漏）
6 站环 1040 m	1043.5	$\approx 8 \times 10^{-5}$	257.9	-0.7	63/120（无漏）

表 16 布局搜索中考察过的候选（严格证书取 $h = 1\text{ m}$ 、 $\delta = 0.7071\text{ m}$ ；余量为负表示存在无法覆盖的点）

布局	站数	粗网格最小余量/m	严格证书
圆心 +12@999 + 12@1863.5（上一版）	25	0.0（解析三角剖分证明成立）	通过
圆心 +9@990 + 12@1866（采用）	22	2.42	通过，最小余量 1.73
圆心 +8@990 + 12@1866	21	-0.5	不通过
圆心 +7@990 + 12@1866	20	-29	不通过
三核 +6@1050 + 12@1866 等	20 ~ 22	-8 ~ -43	不通过
随机局部搜索所得布局	19 ~ 22	-42 ~ -92	不通过
圆心 +6@1200 + 8@2000 等稀疏布局	14 ~ 19	-176 ~ -596	不通过

本表是一次有限搜索的记录，覆盖“圆心 + 正内环 + 正外环”族及若干由该族出发的局部扰动，**不构成对任意 21 站布局的穷举**。搜索脚本与逐布局余量见支撑材料 layout_opt.py。

表 17 测向误差幅度的灵敏度（同一批种子，单位：虚拟秒/源，均 100% 清除）

实际误差幅度	算法误差界	问题三	问题四
$\pm 0.5^\circ$	0.505°	732.38	966.58
$\pm 1.0^\circ$	1.005°	741.00	987.94
$\pm 1.5^\circ$	1.505°	745.92	1015.95

表 18 方向覆盖余量对最小接收半径的灵敏度（20 m 粗网格； $-\infty$ 表示存在无法覆盖的点）

$\rho_{\min}/\text{m}$	1000	998	996	994	990	985	980
22 站（采用）余量 / m	2.417	2.417	2.417	0.482	$-\infty$	$-\infty$	$-\infty$
25 站（解析）余量 / m	0.000	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$

表 19 第四问历次结构性改进的配对实验（单位：虚拟秒/源，两版均 100% 清除）

改进	数据集	对照	候选	下降
49 站方格 $\rightarrow$ 27 站三角网格	100 例，1310 源	928.46	767.64	17.32%
（同上，压力集）	50 例，670 源	916.49	727.66	20.60%
27 站 $\rightarrow$ 25 站双环 + 混合调度	100 例随机	792.16	566.30	28.51%
选择性重测 + 有界等待 + 精化	220 例，2908 源	534.05	499.98	6.38%
认证条带 + near 即清 + 成本门控	450 例，5935 源	499.45	493.55	1.18%

表 20 第三问历次结构性改进的配对实验（单位：虚拟秒/源，两版均 100% 清除）

改进	数据集	对照	候选	下降
单次测向 $\rightarrow$ 两次测向	40 例，498 源	977.71	764.71	21.79%
动态处理顺序 + 共享测站	100 例，1310 源	723.82	653.11	9.77%
25 站网格 $\rightarrow$ 7 站解析覆盖	100 例，1308 源	654.53	346.62	47.04%
环半径 1150 + 混合贪心调度	100 例，1315 源	343.73	263.86	23.24%
（同上，压力集）	70 例，890 源	336.31	281.83	16.20%
圆心补测精化	100 例留出	262.6	260.0	1.0%

各行使用的案例集合不同，**不能把各行降幅连乘**得到一个“总提升”。

表 21 官方演练中虚拟时间的构成（累计占比与每源分摊）

问题	移动	检测	切频	光学定位失败	激光清除
问题三（7 站，70 局，917 源）	74.5%	19.5%	3.6%	1.6%	0.8%
折合每源耗时/s	187.3	48.9	9.1	4.1	2.0
问题四（22 站，70 局，887 源）	74.6%	20.2%	3.8%	1.0%	0.4%
折合每源耗时/s	373.3	101.2	18.8	4.9	2.0

G 时间构成与固定成本的补充数据

平均定位清除时间对源数的双曲线拟合为

$$\bar{T} \approx \frac{T_0}{n} + k, \quad (T_0, k) \approx (2355, 72) \text{ 或 } (6848, -40), \quad (35)$$

第三问 $n = 10$ 与 $n = 16$ 的官方实测为 302.3 与 218.3 s/源，第四问为 639.3 与 375.2 s/源；第四问整局时间基本为常数 6338 ± 398 s（70 局），源多时略快是因为清满 16 个即提前结束（70 局中有 3 局）。

第三问的成本基准由三项相加：走遍七站（最短哈密顿路 6900 m，目标处理段用 $0.7124\sqrt{nA}$ 估计额外行程）、全站排除空频道（未占用频道数 $\times$ 站数 $\times 6$ s，已占用按扫过半数站计）、每源一次第二测点往返与一次清除，结果见表 22。

表 22 第三问的成本基准与官方演练实测（单位：虚拟秒）

源数 n	路线	扫描	每源附加	合计	基准 s/源	官方演练 s/源	局数	超出
10	1874	750	310	2934	293	302.3	7	+3.2%
13	2030	700	403	3133	241	258.0	14	+7.0%
16	2180	696	496	3372	211	218.3	13	+3.5%

第四问成本基准。对 n 个源，

$$\hat{T}_{\text{base}}(n) = \frac{17940.2}{5} + (20 - n) \cdot 22 \cdot 6 + n \cdot 11 \cdot 6 + n \cdot 31 \quad (\text{s}), \quad (36)$$

第一项为 22 站最近邻巡回，第二项为频道扫描（空频道全站各测一次，已占用按扫过半数站计），第三项为到达与清除，结果见表 23；放弃方向覆盖保证的收益与风险见表 24。

表 23 第四问的成本基准与官方演练实测（虚拟秒/源）

源数 n	巡站	扫描	到达与清除	基准 s/源	官方演练 s/源（超出）
10	3588	1980	310	587.8	639.3 (+8.8%)
12	3588	1848	372	484.0	537.1 (+11.0%)
13	3588	1782	403	444.1	494.7 (+11.4%)
14	3588	1716	434	409.9	445.3 (+8.6%)
16	3588	1584	496	354.3	375.2 (+5.9%)

¹2000 m：最外环测站半径 1150 m（第三问）与 1866 m（第四问）均不超过 1866 m，而所有源都在 $R = 1800$ m 内，第二测点与光学格点也仅在其附近，故所有行动点到原点的距离不超过 2000 m。6500 m：对任意“交会后凸子集”，按 25 m 方格的列式蛇形路线，每列内部移动不超过 50 m，相邻非空列间距不超过 $\sqrt{25^2 + 50^2}$ m，且连续凸域不会跳过内部列，故单目标光学路径以 6500 m 为上界。183：在 $\varepsilon = 1.005^\circ$ 、格距 25 m 下，单个可行域的候选格中心至多 61 个横向列 $\times$ 3 个纵向行 = 183 格。三者均按最坏情形取保守上界，只用于确认 T 远小于虚拟时间上限，不参与任何决策。

表 24 放弃方向覆盖保证的收益与风险 (40 万随机定向源蒙特卡洛, 13 源、半数定向)

布局	站数	单定向源漏检率	每局期望漏检	估计 s/源
圆心 +9 + 12 (采用)	22	0	0	≈ 490
圆心 +9 + 8 外环	18	0.4%	0.03	≈ 445
圆心 +9 + 6 外环	16	2.1%	0.14	≈ 410
圆心 +12 内环	13	17.8%	1.16	≈ 330

表 25 共享测站的筛选条件 (第三问)

条件	阈值	条件	阈值
与已有采样点的最小距离	$\geq 100\text{ m}$	每目标额外检测次数	≤ 4
估计中心到当前位置	$\leq 1300\text{ m}$	已有有效方位数	< 3
估计交会角的正弦	≥ 0.12	已清除或已 near	跳过

表 26 本地留出预测与官方演练的分层对照 (虚拟秒/源)

源数 n	问题三			问题四		
	本地留出	官方实测	官方局数	本地留出	官方实测	官方局数
10	318.9	302.3	7	642	639.3	10
13	255.9	258.0	14	475	494.7	12
16	223.5	218.3	13	346	375.2	4

H 问题一的实现细节与补充证明

前向分支引理的证明. 令 $x - s_k = r \mathbf{u}(\varphi)$ ($r > 0$)。两条约束分别化为 $\sin(\varphi - \theta_k + \varepsilon) \geq 0$ 与 $\sin(\varphi - \theta_k - \varepsilon) \leq 0$, 即 $\varphi - \theta_k + \varepsilon \in [0, \pi]$ 且 $\varphi - \theta_k - \varepsilon \in [-\pi, 0] \pmod{2\pi}$, 两者之交为 $\varphi - \theta_k \in [-\varepsilon, \varepsilon]$ 。取 $\varphi = \theta_k + 180^\circ$ 时第一式给出 $-\sin \varepsilon < 0$, 故背向射线被排除。

直径在顶点取到的证明. $\|\cdot\|$ 是凸函数, 故对固定的 y , $x \mapsto \|x - y\|$ 在紧凸集 P 上的最大值必在极点处取到; 对 y 再用一次同样的论证即可。由 Minkowski 定理, 有界闭凸多边形的极点恰为其顶点。

五次线性规划各自的用途. 算法中的线性规划只用来判别 P 的类型, 不用来求顶点。可行性规划取零目标, 只问“约束组是否相容”; 随后四次规划分别沿 $\pm x$ 、 $\pm y$ 方向求极小, 只要有一次报告无下界, P 就在该方向延伸至无穷。四个坐标方向足以判定: 二维多面体无界当且仅当其回收锥非平凡, 而非平凡锥必与某个坐标半轴张成正夹角。判完类型后, 有界情形的顶点由“边界线两两求交、再逐个检验是否满足全部约束”得到。

复杂度. 共 5 次二维线性规划 (可视为常数次); 顶点枚举 $O(m^2)$ 对交点、每个交点 $O(m)$ 次检验, 合计 $O(m^3)$; 直径枚举 $O(k^2)$ 。若检测点数很大, 可换用半平面交 $O(m \log m)$ 加旋转卡壳 $O(k)$, 总复杂度降至 $O(m \log m)$; 本文提交的代码用的是前一种实现。

四类退化的处理. P 为空意味着读数与误差界不相容, 此时必须报错并排查角度约定、取整与坐标系, 不能静默取近似交点; P 无界出现在各楔形方向锥未能互相封闭时 (如只有一个检测点, 或多点近似共线且同侧), 此时直径为 $+\infty$, 不得用人为大框截断后报一个有限值; P 退化为线段或单点时算法 1 给出的 D 自然正确。

平行四边形引理的证明. 不妨设 $\mathbf{u}^\top \mathbf{v} \geq 0$ (否则把 $\mathbf{v}$ 换成 $-\mathbf{v}$ 并平移顶点标号)。两条对角线长为 $\|\mathbf{u} \pm \mathbf{v}\|$, 且 $\|\mathbf{u} + \mathbf{v}\|^2 - \|\mathbf{u} - \mathbf{v}\|^2 = 4\mathbf{u}^\top \mathbf{v} \geq 0$, 故 $\mathbf{u} + \mathbf{v}$ 是较长对角线; 又 $\|\mathbf{u} + \mathbf{v}\|^2 \geq \max(\|\mathbf{u}\|^2, \|\mathbf{v}\|^2)$, 故它也不短于任何边, 即 $\text{diam}(P) = \|\mathbf{u} + \mathbf{v}\|$ 。取圆心 $c = p + (\mathbf{u} + \mathbf{v})/2$ 、半径 $\|\mathbf{u} + \mathbf{v}\|/2$, 该圆过 p 与 $p + \mathbf{u} + \mathbf{v}$, 另两个顶点到 c 的距离均为 $\|\mathbf{u} - \mathbf{v}\|/2 \leq \|\mathbf{u} + \mathbf{v}\|/2$, 故四顶点都在圆内或圆上, 凸性给出覆盖, 结合定理 2 左端即得 $R_*(P) = \text{diam}(P)/2$ 。

两站交会的数值算例. 取真源 $g = (600, 400)$ 、 $\varepsilon = 1^\circ$ 、读数含 $+0.6^\circ$ 与 -0.4° 的偏差, 结果见表 27; 两例中真源均满足全部线性约束, 符合假设 3 下的包含性要求。

表 27 两站交会的数值算例 (真源 $g = (600, 400)$, $\varepsilon = 1^\circ$)

s_1	s_2	交会角 ψ	顶点数	D/m	R_*/m	$D/2$	可否一次认证清除
(0, 0)	(800, 0)	97.13°	4	32.0756	16.0378	16.0378	可 ($R_* \leq 20$)
(0, 0)	(1400, 0)	60.26°	4	55.7265	27.8632	27.8632	否, 需光学兜底

I 问题二的补充推导与完整枚举

保证接收区域定理的证明. 因 $\rho \geq \max(\rho_{\min}, r)$, 只需证 $\|q - g\| \leq \max(\rho_{\min}, r)$ 。当 $r \leq \rho_{\min}$ 时, $\|q - r\mathbf{u}(\delta)\|^2$ 关于 r 是凸二次函数, 在 $[0, \rho_{\min}]$ 上的最大值在端点取得, 两端分别给出 (11) 的两式 (其中 $q^\top \mathbf{u}(\delta) = a \cos \delta + b \sin \delta \geq a \cos \varepsilon - |b| \sin \varepsilon$); 当 $r > \rho_{\min}$ 时只需证 $\|q - g\| \leq r$, 其右端关于 r 单调增, 最紧情形仍为 $r = \rho_{\min}$ 。又第二式等价于 $\|q - \rho_{\min} \mathbf{u}(\pm \varepsilon)\| \leq \rho_{\min}$, 故得 (12)。

交会角引理的证明. 在三角形 $s_1 g q$ 中, g 处的内角为 ψ (或其补角, 正弦相同)。由正弦定理 $\|q - s_1\| / \sin \psi = \|q - g\| / \sin \angle(s_1)$, 而 $\text{dist}(q, L) = \|q - s_1\| \sin \angle(s_1)$, 整理即得。第二式由 L 与局部 x 轴夹角为 δ 、 $|\delta| \leq \varepsilon$ 直接算出。

一阶定位尺度估计. 在小角近似下, 两条楔形在 g 附近可视为宽 $w_1 = 2r_1 \tan \bar{\varepsilon}$ 、 $w_2 = 2r_2 \tan \bar{\varepsilon}$ 的等宽条带, 其交为平行四边形, 故

$$R_* \approx \frac{1}{2} \cdot \frac{\sqrt{w_1^2 + w_2^2 + 2w_1 w_2 |\cos \psi|}}{\sin \psi}, \quad \sin \psi = \frac{\text{dist}(q, L)}{r_2}, \quad (37)$$

用 $\sqrt{w_1^2 + w_2^2 + 2w_1 w_2 |\cos \psi|} \leq w_1 + w_2$ 放大, 得 $R_* \lesssim (r_1 + r_2) r_2 \tan \bar{\varepsilon} / \text{dist}(q, L)$ 。该式把楔形当成等宽条带, 只在交会角不特别尖锐时准确: 与精确枚举对照, $b \geq 450 \text{ m}$ 时相差约 15%, 而 $b = 100 \text{ m}$ 时会高估近一倍, 故仅用于解释趋势。

极值点的证明. $B(\mathbf{0}, \rho_{\min})$ 与 $B(\rho_{\min}\mathbf{u}(-\varepsilon), \rho_{\min})$ 是半径相等、圆心相距 $\rho_{\min}$ 的圆盘，其边界交点在 $\rho_{\min}\mathbf{u}(-\varepsilon \pm 60^\circ)$ ，上方交点即 $\rho_{\min}\mathbf{u}(60^\circ - \varepsilon)$ ，代入即得 $|b|_{\max} = \rho_{\min} \sin 59^\circ$ 。

候选第二点的一阶估计. 表 28 给出代表性候选点的一阶估计与移动代价；所有候选点均在 $\mathcal{C}_{\text{safe}}$ 内。

表 28 不同候选第二点的定位效果与移动代价（ R_* 为按 (14) 算出的一阶估计， $\bar{\varepsilon} = 1.005^\circ$ ，单位 **m**）

候选点 (a, b)	移动/m	移动/s	$r_1 = 500$	$r_1 = 1000$	$r_1 = 1500$	可认证的最大 r_1
(200, 100)	223.6	44.7	44.7	255.0	640.8	377 m
(500, 250)	559.0	111.8	9.8	59.7	181.7	—
(750, 450)	874.6	174.9	17.6	26.7	78.2	—
(515, 857) [†]	999.8	200.0	17.5	34.6	70.3	—

[†] 为推论 2 给出的最大横向偏移点；末列为使一阶 R_* 降到 20 m 以内的最大首测距离，后四个点的 R_* 随 r_1 非单调，无单一阈值，故不填。

12 个候选第二点的精确枚举. 源距取 21 个值、两测偏差各取 3 个方向，共 189 个组合，逐一按算法 1 计算精确交会多边形的直径并取样本内最坏值，结果见表 29。

表 29 12 个候选第二点的精确枚举结果（每点 189 个组合， $\varepsilon = 1^\circ$ ）

左栏（6 个候选点）				右栏（6 个候选点）			
前进 a/m	侧移 b/m	移动/s	最坏直径/m	前进 a/m	侧移 b/m	移动/s	最坏直径/m
250	100	53.85	631.03	650	100	131.53	414.19
250	250	70.71	361.97	650	250	139.28	230.30
250	450	102.96	240.56	650	450	158.11	156.35
500	100	101.98	494.05	750	100	151.33	362.11
500	250	111.80	277.46	750	250	158.11	200.52
500	450	134.54	185.97	750	450	174.93	138.06

左右两栏各含 6 个候选点。

数据印证横向偏移主导: 固定 a 时最坏直径随 b 增大迅速下降（ $b: 250 \rightarrow 450$ 可使直径减半），固定 b 时增大 a 收益明显更小；表现最好的 (750, 450) 最坏直径仍有 138.06 m（对应 $R_* \approx 69.0$ m），远大于 $r_c = 20$ m，与定理 4 一致。

三类数字的口径. 表 28 是一阶估计的半径，表 29 是精确枚举的直径（除以二才与前者可比），定理 4 的 41 m 是不可分辨源的间距。三者结论一致，但只有定理 4 是严格的。

自适应第二点的配对实验. 在 $\{100, 150, 200, 250, 300\} \times \{50, 100, 150, 200\}$ 上按信息增益与移动代价自适应选点，与固定 (200, 100) 相比：第三问 $263.920 \rightarrow 265.518$ s/ 源、第四问 $505.344 \rightarrow 508.205$ s/ 源，均值与最坏值均无改善（清除率均为 100%）。

J 第四问减站边界的三条引理

引理 4：正外环至少需要 12 站。若外圈测站构成以原点为心的正 n 边形，则为使目标圆边界上每一点满足 (28)，必有 $n \geq 12$ 。

证明。 设 g 在目标圆边界上。由定理 9， g 必须落在 $\mathcal{N}_{\rho_{\min}}(g)$ 的凸包内；对边界上所有 g 同时成立，要求外圈站所成多边形包含整个目标圆，即其内切圆半径不小于 $R = 1800\text{ m}$ ，此时正 n 边形边长为 $2R \tan(\pi/n)$ 。 $n = 11$ 时边长 $1057.1\text{ m} > \rho_{\min}$ ：相邻两外环站之间的圆弧上存在与两站距离都超过 $\rho_{\min}$ 的点，其外侧没有可用站，凸包不含它； $n = 12$ 时边长 $964.6\text{ m} < \rho_{\min}$ ，障碍消失。□

引理 5：圆心附近必须有站。在内外两个正环构成的布局中去掉圆心站，则距圆心约 300 m 、朝外发射的源不满足 (28)：此时 $\mathcal{N}_{\rho_{\min}}(g)$ 只剩近侧若干内环站，它们落在过 g 的某条直线同侧，故 $g \notin \text{conv } \mathcal{N}_{\rho_{\min}}(g)$ 。

引理 6：内环取 8 站时证书失败。在外环固定为 $12@1866\text{ m}$ 时，内环取 8 站（半径 990 m ）的相邻站距为 757.7 m ，在 $(-100, 100)$ 附近存在只有 3 个站落在其 1000 m 内、且位于这三站凸包之外的点，网格证书余量为 -0.5 到 -5.9 m ；内环改为 9 站后余量转正为 2.42 m 。继续增大内环半径会使内外环间距超过 1000 m 而在别处失效，故这一维也无余地。

需要重申：后两条引理是特定布局上的数值验证，其负余量依赖具体环半径、网格步长与浮点计算，不能推广为“任意 21 站布局都不可行”。

K 核心源程序

以下为完整的可运行源程序。为便于核对，文件内容与支撑材料中的副本逐字节一致。

K.1 program/model.py：有界误差几何与测站布局

```
1 """有界误差交会定位的几何工具与在线策略基线。
2
3 本模块的每一步决策只使用机器狗可观测的读数（方位、near、no_signal），
4 不读取源的真实坐标，与题设“部分可观测”一致。示向度统一处理成前向楔形：
5 多站交会时背向区域被排除，交集只剩真实位置所在的一块（见 bearing_halfplanes
6 与 clip）。观测带界，因此可行域由误差范围对应的半平面交集描述，认证判据
7 只依赖最小包围圆。
8 """
9 import itertools, math
10 import numpy as np
11 from scipy.optimize import linprog
12
13 def direction(deg):
14     a=np.deg2rad(deg);return np.array([np.cos(a),np.sin(a)])
15
16 def bearing_halfplanes(point, deg, error=1.005):
```

```

17     """以方位角为中心、两侧各展宽 error 度，给出两条前向半平面的约束。"""
18     lo,hi=direction(deg-error),direction(deg+error)
19     A=np.array([[lo[1],-lo[0]],[-hi[1],hi[0]]])
20     return A,A@np.asarray(point)
21
22 def clip(poly,A,b,tol=1e-8):
23     """用半平面组  $A x \leq b$  逐边裁剪多边形：tol 为落在边界上的容差。"""
24     # FIXME: 逐边裁剪在顶点很多时较慢，尚未替换为排序增量式半平面交。
25     poly=np.asarray(poly,dtype=float).reshape(-1,2)
26     for a,c in zip(A,b):
27         if not len(poly):return poly
28         new=[]
29         for p,q in zip(poly,np.roll(poly,-1,axis=0)):
30             fp,fq=a@p-c,a@q-c
31             if fp<=tol:new.append(p)
32             if (fp<=tol) != (fq<=tol):new.append(p+(q-p)*fp/(fp-fq))
33         poly=np.array(new).reshape(-1,2)
34     return poly
35
36 def polygon_from_halfplanes(A,b):
37     """判定半平面交的类型（空/无界/有界）并返回顶点与直径。
38
39     先用线性规划沿  $\pm x$ 、 $\pm y$  四个方向探测目标函数是否有下界，任一方向无下界
40     即判 unbounded，避免把无界区域当成有界而低估直径。有界时枚举两两约束
41     直线的交点作为候选顶点，过滤后按极角排序。
42     """
43     A,b=np.asarray(A,float),np.asarray(b,float)
44     feasible=linprog([0,0],A_ub=A,b_ub=b,bounds=[(None,None)]*2,method='highs')
45     if feasible.status==2:return {'status':'empty','vertices':np.empty((0,2)),
46     'diameter':None}
47     if not feasible.success:raise RuntimeError(feasible.message)
48     for objective in ([1,0],[-1,0],[0,1],[0,-1]):
49         r=linprog(objective,A_ub=A,b_ub=b,bounds=[(None,None)]*2,method='highs')
50         if r.status==3:return {'status':'unbounded','vertices':np.empty((0,2)),
51         'diameter':math.inf}
52         if not r.success:raise RuntimeError(r.message)
53     vertices=[]
54     for i,j in itertools.combinations(range(len(A)),2):
55         M=A[[i,j]]
56         if abs(np.linalg.det(M))<1e-12:continue
57         x=np.linalg.solve(M,b[[i,j]])
58         if np.all(A@x<=b+1e-7) and not any(np.linalg.norm(x-v)<1e-7 for v in
59         vertices):vertices.append(x)
60     v=np.array(vertices).reshape(-1,2)
61     if not len(v):raise RuntimeError('Numerically degenerate feasible polyhedron')
62     c=v.mean(axis=0);v=v[np.argsort(np.arctan2(v[:,1]-c[1],v[:,0]-c[0]))]
63     return {'status':'bounded','vertices':v,'diameter':diameter(v)}
64
65 def diameter(v):
66     """顶点集合的直径：两两欧氏距离的最大值。"""
67     v=np.asarray(v);return float(np.sqrt(np.max(np.sum((v[:,None]-v[None,:])**2,
68     axis=2))))

```

```

66 def enclosing_circle(v):
67     """最小包围圆：枚举全部候选圆心，二维情形最多由 3 个支撑顶点确定。
68
69     候选包括各顶点、各顶点对的中点，以及任意不共线三点确定的外接圆心，
70     逐一代入取到最远顶点的距离作为半径并保留最小者。这样得到的半径是可行
71     域的真实外接圆半径，直接供清除判据使用；三角形一类可行域若用外接矩形
72     中心近似会被低估，从而误判为可一次清除。
73     """
74     v=np.asarray(v);best=(None,math.inf)
75     candidates=list(v)
76     candidates.extend((p+q)/2 for p,q in itertools.combinations(v,2))
77     for p,q,r in itertools.combinations(v,3):
78         M=2*np.array([q-p,r-p])
79         if abs(np.linalg.det(M))>1e-10:
80             candidates.append(np.linalg.solve(M,np.array([q@q-p@p,r@r-p@p])))
81     for c in candidates:
82         rad=float(np.max(np.linalg.norm(v-c,axis=1)))
83         if rad<best[1]:best=(c,rad)
84     return best
85
86 def initial_polygon(p,deg,cfg):
87     """由首个方位读数给出初始前向楔形：沿 deg 前进到接收距离，按角度界张开。"""
88     u=direction(deg);v=np.array([-u[1],u[0]])
89     r=cfg['receiver_radius_max_m'];w=r*np.tan(np.deg2rad(cfg['bearing_bound_deg']))
90     return np.array([p,p+r*u-w*v,p+r*u+w*v])
91
92 def optical_cells(poly,origin,deg,h):
93     """返回与多边形相交的每个方格（输出格心坐标）；方格外接圆半径  $h/\sqrt{2}$ 。"""
94     u=direction(deg);v=np.array([-u[1],u[0]]);B=np.stack([u,v],axis=1)
95     local=(poly-origin)@B
96     lows=np.floor(local.min(axis=0)/h-.5).astype(int)
97     highs=np.ceil(local.max(axis=0)/h+.5).astype(int)
98     cells=[]
99     A=np.array([[1,0],[-1,0],[0,1],[0,-1]])
100     for i in range(lows[0],highs[0]+1):
101         js=list(range(lows[1],highs[1]+1))
102         if i%2:js.reverse()
103         for j in js:
104             c=np.array([i*h,j*h]);b=np.array([c[0]+h/2,-c[0]+h/2,c[1]+h/2,-c[1]+h
105 /2])
106             if len(clip(local,A,b)):cells.append(origin+B@c)
107     return cells
108
109 def triangular_survey_points(cfg):
110     """目标圆内所有闭合三角网格单元的顶点，用于第四问早期的三角布局。
111
112     单元直径小于最小接收距离时，圆内任一点都落在某个单元顶点的接收范围内，
113     由此得到方向覆盖的充分条件；它要求相邻站距都足够小，站数因此压不下去，
114     后来改用凸包充要条件。质心单元落在原点，环半径的取值见
115     docs/experiment2_report.md。圆外顶点一并保留：把测站裁剪进圆内会破坏
116     方向覆盖。
117     """
118     radius=cfg['region_radius_m']

```

```

118 spacing=cfg.get('q4_triangle_spacing_m',950)
119 reception=cfg['receiver_radius_min_m']
120 if not all(math.isfinite(v) and v>0 for v in (radius,spacing,reception)):
121     raise ValueError('Triangle survey requires positive finite distances')
122 if spacing>=reception:
123     raise ValueError('Triangle spacing must be below minimum reception radius')
124 height=spacing*math.sqrt(3)/2
125 offset=np.array([spacing/2,height/3])
126 def vertex(i,j):
127     return np.array([spacing*(i+j/2),height*j])-offset
128 # 每个方向都多取一整格，避免边界处的单元被漏掉。
129 rows=math.ceil((radius+height/3)/height)+1
130 columns=math.ceil(radius/spacing+rows/2+0.5)+1
131 tolerance=1e-9*max(1.,radius,spacing)
132 selected=set()
133 for j in range(-rows,rows+1):
134     for i in range(-columns,columns+1):
135         for ids in ((i,j),(i+1,j),(i,j+1)),
136                     ((i+1,j),(i+1,j+1),(i,j+1))):
137             a=np.array([vertex(*ij) for ij in ids])
138             edge=np.roll(a,-1,axis=0)-a
139             cross=edge[:,0]*(-a[:,1])-edge[:,1]*(-a[:,0])
140             inside=bool(np.all(cross>=0))
141             fraction=np.clip(-np.sum(a*edge,axis=1)/np.sum(edge*edge,axis=1)
142                               ,0,1)
143             closest=a+fraction[:,None]*edge
144             if inside or np.any(np.sum(closest*closest,axis=1)<=(radius+
145 tolerance)**2):
146                 selected.update(ids)
147     return sorted((vertex(*ij) for ij in selected),key=lambda p:(p[0],p[1]))
148
149 def worst_uncovered_distance(stations,region,spacing=5.):
150     """任一圆盘点到自己最近测站的最大距离（粗网格 + 圆周边界采样）。
151     粗网格会把最坏点漏在格间，故沿目标圆周按 spacing 再补一圈采样；返回值
152     再按格对角线的一半  $h/\sqrt{2}$  外扩，得到连续域上的保守上界。
153     """
154     stations=np.asarray(stations,float)
155     xs=np.arange(-region,region+spacing,spacing);X,Y=np.meshgrid(xs,xs)
156     points=np.column_stack((X.ravel(),Y.ravel()));points=points[np.hypot(points
157[:,0],points[:,1])<=region]
158     angles=np.linspace(0,2*np.pi,int(2*np.pi*region/spacing)+1,endpoint=False)
159     points=np.vstack((points,region*np.column_stack((np.cos(angles),np.sin(angles))
160)))
161     worst=0.
162     for start in range(0,len(points),100000):
163         block=points[start:start+100000]
164         worst=max(worst,float(np.min(np.linalg.norm(block[:,None,:]-stations[None
165, :, :],axis=2),axis=1).max()))
166     return worst+spacing/math.sqrt(2)
167
168 def survey_points(question, cfg):
169     if question==3:

```

```

166     layout=cfg.get('q3_survey_layout','grid')
167     if layout=='hexagon':
168         # 圆心站加六个环站；连续域覆盖上界在下方即时校验。
169         # 环半径的取值区间见 docs/experiment3_report.md。
170         radius=cfg['region_radius_m']
171         if not math.isfinite(radius) or radius<=0 or radius/2>cfg['
receiver_radius_min_m']:
172             raise ValueError('Hexagon survey requires 0 < region radius <= 2 *
minimum reception radius')
173         ring=cfg.get('q3_ring_radius_m',radius*math.cos(math.pi/6))
174         if not math.isfinite(ring) or not 0<ring<=math.sqrt(3)*radius:
175             raise ValueError('Q3 ring radius must be positive, finite, and
within the coverage geometry')
176         # 极值点: Voronoi 三重点, 以及目标圆上两环点角平分方向的点。
177         worst=max(ring/math.sqrt(3),math.sqrt(radius**2+ring**2-math.sqrt(3)*
radius*ring))
178         if worst>cfg['receiver_radius_min_m']-1e-6:
179             raise ValueError('Q3 ring does not guarantee minimum-radius
coverage')
180         return [np.zeros(2)]+[ring*direction(60*k) for k in range(6)]
181     if layout=='six':
182         # 实验性、无覆盖保证: 半径比为 1.8 的目标圆, 六个测站覆盖不了,
183         # 边界处总有一圈薄带超过最小接收半径, 因此存在极小漏检概率。
184         # 调用方须显式接受这一风险并给出 q3_uncovered_tolerance_m;
185         # 详见 docs/q3_six_station_trial.md。
186         stations=np.asarray(cfg.get('q3_station_list'),float)
187         if stations.ndim!=2 or stations.shape[1]!=2 or not np.all(np.isfinite(
stations)) or len(stations)<3:
188             raise ValueError('Q3 six-station layout requires q3_station_list
with finite planar points')
189         allowed=cfg.get('q3_uncovered_tolerance_m')
190         if not (isinstance(allowed,(int,float)) and math.isfinite(allowed) and
0<allowed<=100):
191             raise ValueError('Q3 six-station layout requires an explicit
q3_uncovered_tolerance_m in (0,100]')
192         worst=worst_uncovered_distance(stations,cfg['region_radius_m'])
193         if worst>cfg['receiver_radius_min_m']+allowed:
194             raise ValueError(f'Q3 station list leaves points {worst:.1f} m from
every station, beyond the declared tolerance')
195         return list(stations)
196         if layout!='grid':raise ValueError(f'Unknown Q3 survey layout: {layout}')
197     if question==4:
198         layout=cfg.get('q4_survey_layout','grid')
199         if layout=='triangular':return triangular_survey_points(cfg)
200         if layout!='grid':raise ValueError(f'Unknown Q4 survey layout: {layout}')
201         h=cfg[f'q{question}_grid_m'];n=math.ceil(cfg['region_radius_m']/h)
202         return [np.array([i*h,j*h],float) for i in range(-n,n+1) for j in range(-n,n+1)
]
203
204 class Policy:
205     """基线策略: 逐站扫描, 用观测半平面逐步收缩目标可行域并认证清除。"""
206     def __init__(self,transport,cfg,question,strategy='triangulate'):
207         self.io=transport;self.cfg=cfg;self.question=question;self.strategy=

```

```

strategy
    self.pos=np.zeros(2);self.channel=1;self.cleared=set();self.observations
208 ={};self.certificates=[]
209 def measure(self,p,ch):
210     r=self.io.measure(p,ch);self.pos=np.array(p);self.channel=ch;return r
211 def clear(self,p,ch):
212     r=self.io.clear(p,ch);self.pos=np.array(p)
213     if r['clear_result']=='success':self.cleared.add(ch);return True
214     return False
215 def locate(self,ch,p,r):
216     """在测得方位的一站上建立并收缩可行域，随后认证或光学清除该频道。"""
217     if r['measure_result']=='near':
218         if not self.clear(p,ch):raise RuntimeError('near contradicts clear')
219         return
220     deg=r['svd_deg'];poly=initial_polygon(p,deg,self.cfg)
221     if self.strategy=='triangulate':
222         u=direction(deg);v=np.array([-u[1],u[0]])
223         q=p+self.cfg['second_forward_m']*u+self.cfg['second_lateral_m']*v
224         r2=self.measure(q,ch)
225         if r2['measure_result']=='near':
226             if not self.clear(q,ch):raise RuntimeError('near contradicts clear')
227         )
228         return
229         if r2['measure_result']=='direction':
230             A,b=bearing_halfplanes(q,r2['svd_deg'],self.cfg['bearing_bound_deg']
231             );poly=clip(poly,A,b)
232             if not len(poly):raise RuntimeError('Inconsistent bearing bounds;
233             stop rather than fabricate estimate')
234             center,radius=enclosing_circle(poly)
235             self.certificates.append({'channel':ch,'center':center.tolist(),'radius_m':
236             radius,'diameter_m':diameter(poly),'vertices':poly.tolist()})
237             if radius<=self.cfg['clear_m']-1e-6:
238                 if not self.clear(center,ch):raise RuntimeError('Certified disk clear
239                 failed')
240             return
241             cells=optical_cells(poly,p,deg,self.cfg['optical_grid_m'])
242             if cells and np.linalg.norm(cells[-1]-self.pos)<np.linalg.norm(cells[0]-
243             self.pos):cells.reverse()
244             for cell in cells:
245                 if self.clear(cell,ch):return
246             raise RuntimeError('Optical cover exhausted without success')
247 def run(self):
248     """按最近邻遍历测站，命中源后交 locate 处理，直到清满或走完全部测站。"""
249     points=survey_points(self.question,self.cfg);visited=0
250     while points:
251         k=min(range(len(points)),key=lambda i:float(np.sum((points[i]-self.pos)
252         **2)))
253         p=points.pop(k);visited+=1
254         channels=[c for c in range(1,21) if c not in self.cleared]
255         channels.sort(key=lambda c:(c!=self.channel,c))
256         hits=[]
257         for ch in channels:
258             r=self.measure(p,ch)

```

```

252         if r['measure_result']!='no_signal':hits.append((ch,p.copy(),r))
253     for ch,s,r in hits:
254         if ch not in self.cleared:self.locate(ch,s,r)
255         if len(self.cleared)==16:break
256     return {'cleared_count':len(self.cleared),'visited_survey_points':visited,'
termination':'count_upper_bound' if len(self.cleared)==16 else 'full_coverage'}

```

K.2 program/optimized.py: 在线策略主体

```

1  """路径排序与共享测站：只让可观测的响应进入决策。
2
3  搜索任务与目标任务放进同一循环，每步按“谁更近”决定先做哪一个，减少基线
4  逐站扫描带来的折返。第二测点取 (200, ±100)：远源终归要进入光学覆盖，把
5  第二点放近一些即可，继续外移换来的扫描节省有限。共享测站只比较一步移动
6  代价，属于纯启发式；试过的两步前瞻会把总移动距离拉长，故保留一步贪心。
7  variant='legacy' 保留旧路径，用于配对消融。
8  """
9  import numpy as np
10 from model import Policy,direction,initial_polygon,clip,bearing_halfplanes,
    enclosing_circle,diameter,optical_cells,survey_points
11
12 class OptimizedPolicy(Policy):
13     """在基线之上加入路径排序、共享测站，以及按问挂载的调度器。"""
14     def __init__(self,transport,cfg,question,variant='shared'):
15         super().__init__(transport,cfg,question,'triangulate')
16         self.variant=variant;self.tracks={};self.shared_attempts=0;self.shared_hits
17         =0
18         # 第三/四问的调度器按配置挂载：未启用时回退到基线的逐站流程。
19         self.q3_planner=None
20         if question==3 and variant=='shared' and cfg.get('q3_policy','legacy')==
21         'mixed':
22             from q3_policy import Q3Planner
23             self.q3_planner=Q3Planner(self)
24             self.planner=self.q3_planner
25             # 第四问默认 mixed_ring: 选择性重测 + 有界等待 + 认证条带覆盖。
26             if question==4 and variant=='shared' and cfg.get('q4_policy','legacy')==
27             'mixed_ring':
28                 from q4_policy import Q4Planner
29                 self.planner=Q4Planner(self)
30         def measure(self,p,ch):
31             response=super().measure(p,ch)
32             if self.planner is not None:self.planner.observe(p,ch,response)
33             return response
34         def make_track(self,ch,p,r):
35             deg=r.get('svd_deg',0.)
36             self.tracks[ch]={'origin':p.copy(),'deg':deg,'poly':initial_polygon(p,deg,
37             self.cfg),'count':1,'near':p.copy() if r['measure_result']=='near' else None,'
38             extra':0,'sample_points':[p.copy()]}
39             if self.planner is not None:self.planner.constrain(ch)
40         def update(self,ch,p,r):
41             t=self.tracks[ch];t['sample_points'].append(p.copy())
42             if r['measure_result']=='near':t['near']=p.copy();return

```

```

38     if r['measure_result']=='direction':
39         A,b=bearing_halfplanes(p,r['svd_deg'],self.cfg['bearing_bound_deg']);t[
'poly']=clip(t['poly'],A,b)
40         if not len(t['poly']):raise RuntimeError('Inconsistent observed
bearings')
41         t['count']+=1
42         if self.planner is not None:self.planner.constrain(ch)
43     def destination(self,ch):
44         if self.planner is not None:return self.planner.destination(ch)
45         t=self.tracks[ch]
46         if t['near'] is not None:return t['near'],False
47         c,rad=enclosing_circle(t['poly'])
48         if self.variant=='shared' and (rad<=20-1e-6 or t['count']>=2):return c,
False
49         # legacy 路径: 沿方位方向前移 650 m、再左右各偏 250 m 取较近的一侧。
50         u=direction(t['deg']);v=np.array([-u[1],u[0]])
51         candidates=[t['origin']+650*u+sign*250*v for sign in [-1,1]]
52         q=min(candidates,key=lambda x:np.linalg.norm(x-self.pos))
53         return q,True
54     def shared_scan(self,exclude=None):
55         # 到达一个测点/清除点后,顺带看其他待处理频道是否也值得测一次。
56         # TODO: 这里只比较一步移动代价,两步前瞻尚未纳入调度。
57         if self.variant!='shared':return
58         station=self.pos.copy()
59         for ch in sorted(self.tracks):
60             if ch in self.cleared or ch==exclude:continue
61             t=self.tracks[ch]
62             if t['near'] is not None or t['count']>=3 or t['extra']>=4:continue
63             if min(np.linalg.norm(station-p) for p in t['sample_points'])<100:
continue
64                 c,rad=enclosing_circle(t['poly'])
65                 if rad<=20 or np.linalg.norm(c-station)>1300:continue
66                 a=c-t['origin'];b=c-station
67                 sine=abs(a[0]*b[1]-a[1]*b[0])/max(np.linalg.norm(a)*np.linalg.norm(b),1
e-9)
68                 if sine<.12:continue
69                 t['extra']+=1;self.shared_attempts+=1
70                 r=self.measure(station,ch);self.update(ch,station,r)
71                 if r['measure_result']!='no_signal':self.shared_hits+=1
72                 if r['measure_result']=='near':
73                     if not self.clear(station,ch):raise RuntimeError('near clear failed
')
74     def finish(self,ch):
75         t=self.tracks[ch]
76         if t['near'] is not None:
77             if not self.clear(t['near'],ch):raise RuntimeError('near clear failed')
78             return
79         poly=t['poly'];c,rad=enclosing_circle(poly)
80         self.certificates.append(dict(channel=ch,center=c.tolist(),radius_m=rad,
diameter_m=diameter(poly),vertices=poly.tolist()))
81         if rad<=20-1e-6:
82             if not self.clear(c,ch):raise RuntimeError('Certified clear failed')
83             return

```

```

84     cells=optical_cells(poly,t['origin'],t['deg'],self.cfg['optical_grid_m'])
85     if np.linalg.norm(cells[-1]-self.pos)<np.linalg.norm(cells[0]-self.pos):
        cells.reverse()
86     for p in cells:
87         if self.clear(p,ch):return
88         raise RuntimeError('Optical cover exhausted')
89     def run(self):
90         if self.planner is not None:return self.planner.run()
91         points=survey_points(self.question,self.cfg);visited=0
92         while points:
93             idx=min(range(len(points)),key=lambda i:np.linalg.norm(points[i]-self.
pos));p=points.pop(idx);visited+=1
94             self.tracks={}
95             channels=sorted((c for c in range(1,21) if c not in self.cleared),key=
lambda c:(c!=self.channel,c))
96             for ch in channels:
97                 r=self.measure(p,ch)
98                 if r['measure_result']!='no_signal':self.make_track(ch,p,r)
99                 while any(ch not in self.cleared for ch in self.tracks):
100                     remaining=[ch for ch in self.tracks if ch not in self.cleared]
101                     ch=min(remaining,key=lambda ch:np.linalg.norm(self.destination(ch)
[0]-self.pos))
102                     q,needs_measure=self.destination(ch)
103                     if needs_measure:
104                         r=self.measure(q,ch);self.update(ch,q,r)
105                         self.shared_scan(exclude=ch)
106                         self.finish(ch)
107                         self.shared_scan()
108                     if len(self.cleared)==16:break
109         return dict(cleared_count=len(self.cleared),visited_survey_points=visited,
termination='count_upper_bound' if len(self.cleared)==16 else 'full_coverage',
shared_attempts=self.shared_attempts,shared_hits=self.shared_hits)

```

K.3 program/q3_policy.py: 第三问专用策略

```

1  """第三问专用：共享测站下的混合路由，以及对观测的保守约束。
2
3  第三问的源都是全向源，“在某站收不到”蕴含“距离超过  $\rho_{min}$ ”，可以对该站
4  邻域做圆盘排除；第四问的源可能定向，这条信息用不上。圆盘排除的边界点处理
5  过一次证书失效（浮点把边界交点误判到圆盘内），此后保留边界顶点、对接近
6  端点的根做夹取，并补了回归测试。可行域落在 20~300 m 时先到圆心补测精化，
7  明显减少了后续的失败清除。
8  """
9  import math
10 import numpy as np
11 from scipy.spatial import ConvexHull,QhullError
12 from model import clip,direction,enclosing_circle,survey_points
13
14 def outside_disk_hull(poly,center,radius):
15     """凸多边形“挖去”一个开圆盘后的凸包外近似。
16
17     极值点只可能来自圆盘外的多边形顶点或边与圆的交点；凸包取外近似，

```

宁可多留一小块已排除区域，也要保证真源一定还在区域内，否则包含性不变量会被破坏。数值退化时（点数不足、凸包失败）直接返回输入多边形，同样保持保守。

```

"""
points=[]
squared_tolerance=1e-7*max(1.,radius*radius)
for a,b in zip(poly,np.roll(poly,-1,axis=0)):
    delta=a-center;edge=b-a
    if delta@delta>=radius*radius-squared_tolerance:points.append(a)
    aa=edge@edge;bb=2*delta@edge;cc=delta@delta-radius*radius
    discriminant=bb*bb-4*aa*cc
    if aa>1e-16 and discriminant>=0:
        for fraction in ((-bb-math.sqrt(discriminant))/(2*aa),
                           (-bb+math.sqrt(discriminant))/(2*aa)):
            if -1e-9<=fraction<=1+1e-9:points.append(a+np.clip(fraction,0.,1.)*
edge)
    if len(points)<3:return poly
points=np.unique(np.asarray(points),axis=0)
    if len(points)<3:return poly
    try:return points[ConvexHull(points).vertices]
    except QhullError:return poly

class Q3Planner:
    """第三问调度：全向源的圆盘排除、最近优先路由与圆心补测精化。"""
    def __init__(self,policy):
        self.p=policy
        self.silent={ch:[] for ch in range(1,21)}
        angles=np.arange(48)*2*np.pi/48
        self.region_normals=np.column_stack([np.cos(angles),np.sin(angles)])
        self.forward=policy.cfg.get('q3_second_forward_m',200.)
        self.lateral=policy.cfg.get('q3_second_lateral_m',100.)
        # 对不超过该半径的可行域，先到圆心（必要时再横向偏移）补测一次，
        # 仍不够才退回光学网格。
        self.refine_max=policy.cfg.get('q3_refine_max_m',300.)
        self.refine_offset=policy.cfg.get('q3_refine_offset_m',60.)
        if not all(math.isfinite(x) and x>0 for x in (self.forward,self.lateral,
self.refine_offset)):
            raise ValueError('Q3 second-point offsets must be positive and finite')
        if not (math.isfinite(self.refine_max) and self.refine_max>=0):
            raise ValueError('Q3 refinement radius must be non-negative and finite')
    )
    def observe(self,position,ch,response):
        if response['measure_result']=='no_signal':
            self.silent[ch].append(np.asarray(position,float).copy())
    def constrain(self,ch):
        p=self.p;t=p.tracks[ch]
        if t['near'] is not None:return
        # 支撑半平面取外切：圆边界附近的一圈真实位置必须保留，内接会把它切掉。
        poly=clip(t['poly'],self.region_normals,
                np.full(48,p.cfg['region_radius_m']+1e-7))
        if not len(poly):raise RuntimeError('Bearings contradict the target region')
    )
    for station in self.silent[ch]:

```

```

67     # 第三问均为全向源且接收半径  $\geq R_{\min}$ ; 排除盘略取小一点留浮点余量。
68     poly=outside_disk_hull(poly,station,p.cfg['receiver_radius_min_m']-1e
-4)
69     t['poly']=poly
70     def destination(self,ch):
71         p=self.p;t=p.tracks[ch]
72         if t['near'] is not None:return t['near'],False
73         center,radius=enclosing_circle(t['poly'])
74         if radius<=20-1e-6 or t['count']>=2:return center,False
75         u=direction(t['deg']);v=np.array([-u[1],u[0]])
76         candidates=[t['origin']+self.forward*u+sign*self.lateral*v for sign in
(-1,1)]
77         point=min(candidates,key=lambda q:np.linalg.norm(q-p.pos)+np.linalg.norm(q-
center))
78         return point,True
79     def refine(self,ch):
80         """对 20~300 m 的可行域用一到两次近距离测向收缩; 认证清除仍然精确。"""
81         p=self.p;t=p.tracks[ch]
82         if t['near'] is not None or self.refine_max<=0:return
83         center,radius=enclosing_circle(t['poly'])
84         if not (p.cfg['clear_m']<radius<=self.refine_max):return
85         response=p.measure(center,ch);p.update(ch,center,response)
86         if response['measure_result']!='direction' or t['near'] is not None:return
87         center2,radius2=enclosing_circle(t['poly'])
88         if radius2<=p.cfg['clear_m']-1e-6:return
89         u=direction(response['svd_deg']);v=np.array([-u[1],u[0]])
90         point=min((center+self.refine_offset*v,center-self.refine_offset*v),
91                 key=lambda q:np.linalg.norm(q-center2))
92         response=p.measure(point,ch);p.update(ch,point,response)
93     def cost(self,ch):
94         point,needs_measure=self.destination(ch)
95         center,_=enclosing_circle(self.p.tracks[ch]['poly'])
96         return np.linalg.norm(point-self.p.pos)+(np.linalg.norm(point-center) if
needs_measure else 0.)
97     def run(self):
98         p=self.p;points=survey_points(3,p.cfg);visited=0
99         while points or any(ch not in p.cleared for ch in p.tracks):
100             pending=[ch for ch in p.tracks if ch not in p.cleared]
101             ch=min(pending,key=self.cost) if pending else None
102             index=min(range(len(points)),key=lambda i:np.linalg.norm(points[i]-p.
pos)) if points else None
103             if index is not None and (ch is None or np.linalg.norm(points[index]-p.
pos)<self.cost(ch)):
104                 station=points.pop(index);visited+=1
105                 channels=sorted((c for c in range(1,21) if c not in p.cleared),key=
lambda c:(c!=p.channel,c))
106                 for c in channels:
107                     response=p.measure(station,c)
108                     if c in p.tracks:p.update(c,station,response)
109                     elif response['measure_result']!='no_signal':p.make_track(c,
station,response)
110                 else:
111                     point,needs_measure=self.destination(ch)

```

```

112         if needs_measure:
113             response=p.measure(point,ch);p.update(ch,point,response)
114             p.shared_scan(exclude=ch)
115             # 先补测精化，再走原有的认证清除与完整光学兜底。
116             self.refine(ch)
117             p.finish(ch);p.shared_scan()
118             if len(p.cleared)==16:break
119         return dict(cleared_count=len(p.cleared),visited_survey_points=visited,
120                    termination='count_upper_bound' if len(p.cleared)==16 else '
full_coverage',
121                    shared_attempts=p.shared_attempts,shared_hits=p.shared_hits)

```

K.4 program/q4_policy.py: 第四问专用策略

```

1  """第四问：选择性重测、有界等待与精化，全部建立在认证覆盖之上。
2
3  第四问的源可能定向，`no_signal` 带不了距离信息，故不做圆盘排除；判定改用
4  “凸包方向覆盖”充要条件。选择性重测只在新测向确有缩小可行域的价值（交会角
5  与间距达标）时才发起，减少无效测量；有界等待只对“内环发现、且离首测点
6  不远”的目标生效。精化阶段带扫描成本门控：光学覆盖更省时跳过圆心补测。
7  认证条带覆盖与方格路线都用 route_seconds 折算成时间，取更省者。
8  """
9  import math
10 import numpy as np
11 from model import clip, direction, enclosing_circle, diameter
12 from q4_optical import clearing_route, route_seconds
13
14
15 def ring_stations(cfg):
16     """用边长小于接收距离的正十二边形拆出内外双环（外加圆心），共 25 站。
17
18     每个源都能写成三个接收范围内测站的凸组合，于是过该源的任一闭半平面
19     都含一个测站，对任意未知天线朝向都能被发现。该解析布局作为认证布局的
20     保守备用，配置中省略测站列表时回退到它。
21     """
22     radius=cfg['region_radius_m'];reception=cfg['receiver_radius_min_m']
23     inner=999.*radius/1800.
24     outer=radius/math.cos(math.pi/12)
25     bound=max(inner,2*radius*math.tan(math.pi/12),
26               math.sqrt(inner**2+outer**2-2*inner*radius))
27     if not all(math.isfinite(v) and v>0 for v in (radius,reception)) or bound>=
reception:
28         raise ValueError('Q4 ring stations do not satisfy the reception certificate
')
29     angles=np.arange(12)*2*np.pi/12
30     inner_points=inner*np.column_stack((np.cos(angles),np.sin(angles)))
31     angles=angles+math.pi/12
32     outer_points=outer*np.column_stack((np.cos(angles),np.sin(angles)))
33     return list(np.vstack([[0.,0.],inner_points,outer_points]))
34
35
36 def hull_coverage_slack(stations,cfg,spacing=20.,margin=0.):

```

"""在圆盘网格上取“方向覆盖余量”的最小值（精确凸包，负值表示落在凸包外）。

源 g 对任意朝向都能被发现，当且仅当 g 落在其接收范围内测站的凸包内，余量就是 g 到该凸包边界的距离。运行时用 20 m 粗网格复核配置，余量为负即拒绝运行；连续域的严格证书要求最小余量超过网格步长对应量，见 experiments/q4_layout_search。

```
"""
from scipy.spatial import ConvexHull
S=np.asarray(stations,float);R=cfg['receiver_radius_min_m'];region=cfg['region_radius_m']
xs=np.arange(-region,region+spacing,spacing);X,Y=np.meshgrid(xs,xs)
G=np.column_stack((X.ravel(),Y.ravel()));G=G[np.hypot(G[:,0],G[:,1])<=region]
angles=np.linspace(0,2*np.pi,int(2*np.pi*region/spacing)+1,endpoint=False)
G=np.vstack((G,region*np.column_stack((np.cos(angles),np.sin(angles)))))
D=np.linalg.norm(G[:,None,:]-S[None,:,:],axis=2);mask=D<=R-margin
worst=math.inf;groups={}
for i,row in enumerate(map(bytes,np.packbits(mask,axis=1))):groups.setdefault(
row,[]).append(i)
for idx in groups.values():
    T=S[mask[idx[0]]]
    if len(T)<3: return -math.inf
    eq=ConvexHull(T).equations;g=G[idx]
    worst=min(worst,float((-g@eq[:,2].T+eq[:,2]).max(axis=1)).min())
return worst
```

```
def survey_stations(cfg):
    """测站布局：解析的 25 站双环，或配置中显式给出的认证布局（22 站）。"""
    explicit=cfg.get('q4_station_list')
    if explicit is None: return ring_stations(cfg)
    S=np.asarray(explicit,float)
    if S.ndim!=2 or S.shape[1]!=2 or not np.all(np.isfinite(S)) or len(S)<3:
        raise ValueError('q4_station_list must be a finite list of planar points')
    if hull_coverage_slack(S,cfg)<0:
        raise ValueError('q4_station_list fails the directional coverage check')
    return list(S)
```

```
def polygon_distance(poly,point):
    """点到非空凸多边形的欧氏距离，含线段/单点等退化情形。"""
    poly=np.asarray(poly,float).reshape(-1,2);point=np.asarray(point,float)
    if not len(poly): raise ValueError('Empty target region')
    edges=np.roll(poly,-1,axis=0)-poly;delta=point-poly
    cross=edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]
    area2=np.sum(poly[:,0]*np.roll(poly[:,1],-1)-poly[:,1]*np.roll(poly[:,0],-1))
    if abs(area2)>1e-12 and (np.all(cross>=0) or np.all(cross<=0)): return 0.
    length2=np.sum(edges*edges,axis=1)
    fraction=np.divide(np.sum(delta*edges,axis=1),length2,out=np.zeros(len(poly)),
    where=length2>0)
    closest=poly+np.clip(fraction,0,1)[:,None]*edges
    return float(np.linalg.norm(closest-point,axis=1).min())
```

```

87 class Q4Planner:
88     """第四问调度：方向覆盖校验、选择性重测、有界等待、精化与条带覆盖。"""
89     def __init__(self, policy):
90         self.p=policy
91         angles=np.arange(48)*2*np.pi/48
92         self.normals=np.column_stack([np.cos(angles),np.sin(angles)])
93         self.forward=policy.cfg.get('q4_second_forward_m',200.)
94         self.lateral=policy.cfg.get('q4_second_lateral_m',100.)
95         if not all(math.isfinite(v) and v>0 for v in (self.forward,self.lateral)):
96             raise ValueError('Q4 second-point offsets must be positive and finite')
97         cfg=policy.cfg
98         self.clear_m=cfg['clear_m']
99         self.selective=cfg.get('q4_selective_remeasure',True)
100        self.wait=cfg.get('q4_wait_for_survey',True)
101        self.shared=cfg.get('q4_shared_scan',True)
102        self.help_m=cfg.get('q4_station_help_m',950.)
103        self.help_distance=cfg.get('q4_help_distance','polygon')
104        self.min_sine=cfg.get('q4_remeasure_min_sine',0.3)
105        self.min_gap=cfg.get('q4_remeasure_min_gap_m',300.)
106        self.stale_m=cfg.get('q4_wait_stale_m',1400.)
107        self.refine_max=cfg.get('q4_refine_max_m',150.)
108        self.wait_mode=cfg.get('q4_wait_mode','inner')
109        self.strip_cover=cfg.get('q4_strip_cover',False)
110        self.immediate_near=cfg.get('q4_immediate_near',False)
111        self.scan_cost_gate=cfg.get('q4_scan_cost_gate',False)
112        if not all(type(value) is bool for value in (self.selective,self.wait,self.
shared)):
113            raise ValueError('Q4 feature switches must be booleans')
114        if not all(math.isfinite(v) and v>0 for v in (self.clear_m,self.help_m,self
.min_gap,self.stale_m)):
115            raise ValueError('Q4 distances must be positive and finite')
116        if not math.isfinite(self.refine_max) or self.refine_max<0:
117            raise ValueError('Q4 refinement radius must be non-negative and finite'
)
118        if not math.isfinite(self.min_sine) or not 0<=self.min_sine<=1:
119            raise ValueError('Q4 intersection sine must be between zero and one')
120        if self.help_m>=cfg['receiver_radius_min_m'] or self.help_distance not in (
'polygon','center'):
121            raise ValueError('Invalid Q4 station help criterion')
122        if self.wait_mode not in ('stale','inner','same_side','all_sides'):
123            raise ValueError('Invalid Q4 waiting rule')
124        if not all(type(value) is bool for value in (self.strip_cover,self.
immediate_near,self.scan_cost_gate)):
125            raise ValueError('Q4 optical feature switches must be booleans')
126
127        def observe(self, position, ch, response):
128            # 收不到信号与“附近存在定向源”相容，第四问不能用它排除距离。
129            # TODO: 暂未利用 no_signal 的析取信息（源可能落在另一些判据之外），
130            # 后续可把它并入可行域。
131            pass
132
133        def constrain(self, ch):
134            t=self.p.tracks[ch]

```

```

135     if t['near'] is not None: return
136     poly=clip(t['poly'],self.normals,
137              np.full(48,self.p.cfg['region_radius_m']+1e-7))
138     if not len(poly): raise RuntimeError('Bearings contradict the target region'
139 )
140     t['poly']=poly
141
142     def destination(self,ch):
143         p=self.p;t=p.tracks[ch]
144         if t['near'] is not None: return t['near'],False
145         center,radius=enclosing_circle(t['poly'])
146         if radius<=self.clear_m-1e-6 or t['count']>=2: return center,False
147         u=direction(t['deg']);v=np.array([-u[1],u[0]])
148         candidates=[t['origin']+self.forward*u+sign*self.lateral*v for sign in
149 (-1,1)]
150         return min(candidates,key=lambda q:np.linalg.norm(q-p.pos)+np.linalg.norm(q
151 -center)),True
152
153     def cost(self,ch):
154         point,measure=self.destination(ch)
155         center,_=enclosing_circle(self.p.tracks[ch]['poly'])
156         return np.linalg.norm(point-self.p.pos)+(np.linalg.norm(point-center) if
157 measure else 0.)
158
159     def close_enough(self,station,ch):
160         t=self.p.tracks[ch]
161         if self.help_distance=='center':
162             return np.linalg.norm(enclosing_circle(t['poly'])[0]-station)<=self.
163 help_m
164         return polygon_distance(t['poly'],station)<=self.help_m
165
166     def useful(self,station,ch):
167         if not self.selective: return True
168         t=self.p.tracks[ch]
169         if t['near'] is not None: return False
170         center,radius=enclosing_circle(t['poly'])
171         if radius<=self.clear_m-1e-6 or not self.close_enough(station,ch): return
172 False
173         for previous in t['sample_points']:
174             a=center-previous;b=center-station
175             sine=abs(a[0]*b[1]-a[1]*b[0])/max(np.linalg.norm(a)*np.linalg.norm(b),1
176 e-9)
177             if sine<self.min_sine and np.linalg.norm(station-previous)<self.min_gap
178 : return False
179         return True
180
181     def ready(self,ch):
182         t=self.p.tracks[ch]
183         if t['near'] is not None: return True
184         radius=enclosing_circle(t['poly'])[1]
185         return radius<=max(self.clear_m-1e-6,self.refine_max)
186
187     def target_cost(self,ch,stations):

```

```

180     t=self.p.tracks[ch]
181     if self.ready(ch):
182         center=t['near'] if t['near'] is not None else enclosing_circle(t['poly
183         '])[0]
184         return np.linalg.norm(center-self.p.pos)
185         # 有界等待: 只在剩余巡站有限、且离首次发现不太远时才允许等。
186         if self.wait and np.linalg.norm(t['origin']-self.p.pos)<self.stale_m:
187             if self.wait_mode=='inner' and np.linalg.norm(t['origin'])>=self.p.cfg[
188             'region_radius_m']:
189                 return self.cost(ch)
190                 for station in stations:
191                     if not self.close_enough(station,ch):continue
192                     if self.wait_mode in ('same_side','all_sides'):
193                         probes=t['poly'] if self.wait_mode=='all_sides' else
194                         enclosing_circle(t['poly'])[0][None,:])
195                         a=t['origin']-probes;b=station-probes
196                         cosine=np.sum(a*b,axis=1)/np.maximum(np.linalg.norm(a,axis=1)*
197                         np.linalg.norm(b,axis=1),1e-9)
198                         if np.any(cosine<0.3):continue
199                         return math.inf
200                     return self.cost(ch)
201
202 def refine(self,ch):
203     t=self.p.tracks[ch];p=self.p
204     if t['near'] is not None:return
205     center,radius=enclosing_circle(t['poly'])
206     if not self.clear_m<radius<=self.refine_max:return
207     if self.scan_cost_gate:
208         points,_=clearing_route(t['poly'],t['origin'],t['deg'],p.pos,p.cfg)
209         budget=route_seconds(points,p.pos,p.cfg)
210         travel=np.linalg.norm(center-p.pos)/p.cfg['speed_m_s']
211         if budget<=travel+p.cfg['measure_s']+p.cfg['clear_failure_s']+10:return
212         response=p.measure(center,ch);p.update(ch,center,response)
213         if response['measure_result']!='direction':return
214         new_center,new_radius=enclosing_circle(t['poly'])
215         if new_radius<=self.clear_m-1e-6:return
216         u=direction(response['svd_deg']);side=np.array([-u[1],u[0]])
217         offset=3*self.clear_m
218         candidates=(center+offset*side,center-offset*side)
219         point=min(candidates,key=lambda q:np.linalg.norm(q-new_center))
220         response=p.measure(point,ch);p.update(ch,point,response)
221
222 def finish(self,ch):
223     p=self.p;t=p.tracks[ch]
224     if ch in p.cleared:return
225     if t['near'] is not None:return p.finish(ch)
226     center,radius=enclosing_circle(t['poly'])
227     if radius<=self.clear_m-1e-6 or not self.strip_cover:return p.finish(ch)
228     points,_=clearing_route(t['poly'],t['origin'],t['deg'],p.pos,p.cfg)
229     p.certificates.append(dict(channel=ch,center=center.tolist(),radius_m=
230     radius,
231
232                                     diameter_m=diameter(t['poly']),vertices=t['poly'
233                                     ].tolist()))

```

```

227     for point in points:
228         if p.clear(point,ch):return
229     raise RuntimeError('Certified optical strip cover exhausted')
230
231     def run(self):
232         p=self.p;points=survey_stations(p.cfg);visited=0
233         while points or any(ch not in p.cleared for ch in p.tracks):
234             pending=[ch for ch in p.tracks if ch not in p.cleared]
235             costs={ch:self.target_cost(ch,points) for ch in pending}
236             ch=min(pending,key=costs.get) if pending else None
237             index=min(range(len(points)),key=lambda i:np.linalg.norm(points[i]-p.
pos)) if points else None
238             if index is not None and (ch is None or np.linalg.norm(points[index]-p.
pos)<costs[ch]):
239                 station=points.pop(index);visited+=1
240                 channels=sorted((c for c in range(1,21) if c not in p.cleared and
(c not in p.tracks or self.useful(station,c))),key
241 =lambda c:(c!=p.channel,c))
242                 for c in channels:
243                     response=p.measure(station,c)
244                     if c in p.tracks:p.update(c,station,response)
245                     elif response['measure_result']!='no_signal':p.make_track(c,
station,response)
246                     if self.immediate_near and response['measure_result']=='near':
247                         if not p.clear(station,c):raise RuntimeError('near clear
failed')
248                         if len(p.cleared)==16:break
249                 else:
250                     if not self.ready(ch):
251                         point,measure=self.destination(ch)
252                         if measure:
253                             response=p.measure(point,ch);p.update(ch,point,response)
254                             if self.shared:p.shared_scan(exclude=ch)
255                         self.refine(ch)
256                         self.finish(ch)
257                         if self.shared:p.shared_scan()
258                     if len(p.cleared)==16:break
259             return dict(cleared_count=len(p.cleared),visited_survey_points=visited,
260                 termination='count_upper_bound' if len(p.cleared)==16 else '
full_coverage',
261                 shared_attempts=p.shared_attempts,shared_hits=p.shared_hits)

```

K.5 program/q4_optical.py: 认证条带覆盖

```

1  """第四问光学清除的连续域覆盖，独立于射频可见性。
2
3  把多边形按首测方向切成窄条，条内再切矩形：矩形四角落在同一个清除盘内时，
4  由凸性整个矩形都在盘内，沿矩形中心布点即可覆盖该块区域。条带越宽列数越少、
5  单列却越长，路径未必更短，因此条带路线与方格路线都用 route_seconds 折算成
6  时间，取更省者（见 clearing_route）。
7  """
8  import math

```

```

9 import numpy as np
10 from model import clip, direction, optical_cells
11
12
13 def route_seconds(points, start, cfg):
14     """把一条清除路线的移动与清除时间折算成虚拟秒。"""
15     points = np.asarray(points, float)
16     length = float(np.linalg.norm(points[0] - start) + np.linalg.norm(np.diff(points, axis=0), axis=1).sum())
17     return length / cfg['speed_m_s'] + len(points) * cfg['clear_failure_s']
18
19
20 def strip_cover(poly, origin, deg, width, radius):
21     """把多边形按条带切分、条内再切行，使每个矩形四角都落在同一个清除盘内。"""
22     u = direction(deg); basis = np.stack([u, [-u[1], u[0]]], axis=1)
23     local = (np.asarray(poly) - origin) @ basis
24     lo, hi = local[:, 0].min(), local[:, 0].max()
25     count = max(1, math.ceil((hi - lo) / width))
26     cuts = np.linspace(lo, hi, count + 1); columns = []; certificates = []
27     for left, right in zip(cuts[:-1], cuts[1:]):
28         section = clip(local, [[1., 0.], [-1., 0.]], [right, -left])
29         if not len(section): continue
30         bottom, top = section[:, 1].min(), section[:, 1].max()
31         half_x = (right - left) / 2
32         max_height = 2 * math.sqrt(max(0., radius * radius - half_x * half_x))
33         if max_height <= 0: raise ValueError('Strip wider than clear diameter')
34         rows = max(1, math.ceil((top - bottom) / max_height))
35         ycuts = np.linspace(bottom, top, rows + 1)
36         centers = []
37         for lower, upper in zip(ycuts[:-1], ycuts[1:]):
38             center = np.array([(left + right) / 2, (lower + upper) / 2])
39             corners = np.array([[left, lower], [right, lower], [right, upper], [left, upper]])
40
41             assert np.max(np.linalg.norm(corners - center, axis=1)) <= radius + 1e-7
42             centers.append(origin + basis @ center)
43             certificates.append(dict(center=(origin + basis @ center).tolist(),
44                                     corners=(origin + corners @ basis.T).tolist()))
45
46         columns.append(centers)
47     return columns, certificates
48
49 def order_columns(columns, start):
50     """逐条选择较近的一端进入，把各列拼成一条连续路线。"""
51     path = []; position = np.asarray(start)
52     for column in columns:
53         if np.linalg.norm(column[-1] - position) < np.linalg.norm(column[0] - position):
54             column = column[::-1]
55         path.extend(column); position = column[-1]
56     return path
57
58 def clearing_route(poly, origin, deg, start, cfg):
59     """方格路线与四种条带宽度的条带路线比较，返回时间更短者及其矩形证书。"""

```

```

59 old=optical_cells(poly,origin,deg,cfg['optical_grid_m'])
60 if np.linalg.norm(old[-1]-start)<np.linalg.norm(old[0]-start):old.reverse()
61 best=old;cost=route_seconds(old,start,cfg);certificate=None
62 radius=cfg['clear_m']-0.1
63 if radius<=0:return best,certificate
64 for fraction in (1.25,1.5,1.7,1.9):
65     columns,rectangles=strip_cover(poly,origin,deg,fraction*radius,radius)
66     for ordered in (columns,columns[::-1]):
67         candidate=order_columns(ordered,start)
68         candidate_cost=route_seconds(candidate,start,cfg)
69         if candidate_cost<cost-1e-7:
70             best,cost,certificate=candidate,candidate_cost,rectangles
71 return best,certificate

```

K.6 program/layout_opt.py、program/certify22.py: 方向覆盖余量与网格证书

```

1 """在“方向覆盖”的带余量证书下搜索第四问测站布局。
2
3 证书: 对 1800 m 圆盘上每一点 g, 存在测站子集 T, 使每个 s 属于 T 时满足
4 |s-g| ≤ R - m, 且 g 落在 conv(T) 内、到其边界距离 ≥ m。于是过 g 附近任意点
5 的任一闭半平面内都含一个距其 R 内的测站, 即 g 处任意天线朝向都能被看到。
6 当最小余量超过网格步长时, 离散网格证书即可推广到连续域 (h/√2)。
7
8 搜索以最小余量 slack 为目标 (objective 取网格上的最小 slack), 从环形种子
9 出发做局部随机扰动, 再用 2-opt 收紧巡站长度; 减站过程按此逐档比较。
10 """
11 import sys,math,json,time
12 import numpy as np
13 from scipy.spatial import ConvexHull
14
15 R=1000.;REGION=1800.
16
17 def grid(h):
18     xs=np.arange(-REGION,REGION+h,h);X,Y=np.meshgrid(xs,xs);G=np.column_stack((X.
19     ravel(),Y.ravel()))
20     G=G[np.hypot(G[:,0],G[:,1])<=REGION]
21     ring=np.linspace(0,2*np.pi,int(2*np.pi*REGION/h)+1,endpoint=False)
22     return np.vstack((G,REGION*np.column_stack((np.cos(ring),np.sin(ring)))))
23
24 def slack(S,G,m):
25     """Min over grid of hull slack using stations within R-m; -inf if fewer than 3.
26     """
27     S=np.asarray(S);D=np.linalg.norm(G[:,None,:]-S[None,:,:],axis=2);mask=D<=R-m
28     packed=np.packbits(mask,axis=1);keys={}
29     for i,row in enumerate(map(bytes,packed)):keys.setdefault(row,[]).append(i)
30     third=np.sort(D,axis=1)[:,2] if S.shape[0]>=3 else np.full(len(G),np.inf)
31     out=(R-m)-third # negative when fewer than three stations are in range
32     for row,idx in keys.items():
33         T=S[mask[idx[0]]]
34         if len(T)<3:continue

```

```

33     try: eq=ConvexHull(T).equations
34     except Exception: out[idx]=np.minimum(out[idx],-1.); continue
35     g=G[idx]; out[idx]=np.minimum(out[idx],-(g@eq[:, :2].T+eq[:, 2]).max(axis=1))
36     return out
37
38 def objective(S,G,m):
39     s=slack(S,G,m); k=int(np.argmin(s)); return float(s[k]),G[k]
40
41 def tour(S):
42     """从原点出发的最近邻巡回，再用 2-opt 收紧，返回总长度。"""
43     S=[np.zeros(2)]+[np.asarray(p) for p in S]; n=len(S); order=[0]; left=set(range(1,
44     n))
45     while left:
46         k=min(left, key=lambda i: np.linalg.norm(S[i]-S[order[-1]])); order.append(k);
47         left.remove(k)
48         L=lambda o: sum(np.linalg.norm(S[o[i]]-S[o[i+1]])) for i in range(len(o)-1)
49         improved=True
50         while improved:
51             improved=False
52             for i in range(1,n-1):
53                 for j in range(i+1,n):
54                     new=order[:i]+order[i:j+1][::-1]+order[j+1:]
55                     if L(new)<L(order)-1e-9: order=new; improved=True
56     return L(order)
57
58 def ring(n,r,phase=0.):
59     a=np.arange(n)*2*np.pi/n+phase; return list(np.column_stack((r*np.cos(a),r*np.
60     sin(a))))
61
62 def seeds(N,rng):
63     out=[]
64     for n_out in (9,10,11,12):
65         for center in (0,1):
66             n_in=N-n_out-center
67             if n_in<3: continue
68             for r_in in (700,850,950):
69                 for r_out in (1850,1900,1950):
70                     out.append((np.zeros(2) if center else [])+ring(n_in,r_in,rng
71                     .uniform(0,2*np.pi))+ring(n_out,r_out,rng.uniform(0,2*np.pi)))
72     return out
73
74 def optimize(N,budget_s,seed=0,m=20.,h=40.):
75     """在预算内从多个环形种子出发，用随机扰动最大化网格上的最小余量。"""
76     rng=np.random.default_rng(seed); G=grid(h); best=None; t0=time.time()
77     cands=seeds(N,rng)
78     scored=sorted(((objective(S,G,m)[0],i) for i,S in enumerate(cands)),reverse=
79     True)[:6]
80     pool=[np.array(cands[i]) for _,i in scored]
81     while time.time()-t0<budget_s:
82         for S in pool:
83             cur,_=objective(S,G,m)
84             for _ in range(40):
85                 step=rng.choice([15.,40.,100.]); T=S.copy()

```

```

81         k=rng.integers(len(S)) if rng.random()<0.7 else slice(None)
82         T[k]+=rng.normal(0,step,size=np.shape(T[k]))
83         val,_=objective(T,G,m)
84         if val>cur:S[:]=T;cur=val
85         if best is None or cur>best[0]:best=(cur,S.copy())
86         if time.time()-t0>budget_s:break
87     return best
88
89 if __name__=='__main__':
90     N=int(sys.argv[1]);budget=float(sys.argv[2]);seed=int(sys.argv[3]) if len(sys.
91     argv)>3 else 0
92     val,S=optimize(N,budget,seed)
93     fine,worst=objective(S,grid(6.),20.)
94     print(json.dumps(dict(N=N,coarse_slack=val,fine_slack=fine,worst=worst.tolist(),
95     tour_m=tour(list(S)),stations=S.tolist())))

```

```

1  """22 站布局的严格网格证书（见 docs/q4_layout_search.md）。
2
3  先用 20 m 粗网格快速筛选，再用 h=1 m 细网格做最终认证。判据是最小余量
4  不小于  $\delta=h/\sqrt{2}$ ，且全部网格点通过；此时离散网格证书可推广到连续域。网格
5  按横坐标分块处理，逐块调用 slack 并记录最坏点。
6  """
7  import sys,json,math,time;sys.path.insert(0,'experiments/q4_layout_search')
8  import numpy as np
9  from layout_opt import slack
10 S=np.array(json.load(open('experiments/q4_layout_search/ring22_stations.json')));
    REGION=1800.
11 h=float(sys.argv[1]) if len(sys.argv)>1 else 1.;delta=h/math.sqrt(2);t=time.time()
12 xs=np.arange(-REGION-2*h,REGION+3*h,h);worst=math.inf;wpt=None;count=0
13 # 每次取 200 列切片，控制单次 slack 的内存。
14 for x0 in xs[::200]:
15     X,Y=np.meshgrid(xs[(xs>=x0)&(xs<x0+200*h)],xs);G=np.column_stack((X.ravel(),Y.
16     ravel()));G=G[np.hypot(G[:,0],G[:,1])<=REGION+delta]
17     if not len(G):continue
18     count+=len(G)
19     for i in range(0,len(G),80000):
20         s=slack(S,G[i:i+80000],delta);k=int(np.argmin(s))
21         if s[k]<worst:worst=float(s[k]);wpt=G[i+k]
22 # 全部网格点通过且最小余量不低于  $\delta$ ，连续域证书成立。
23 ok=worst>=delta+1e-6
24 res=dict(stations=S.tolist(),grid_h=h,delta=delta,grid_points=count,min_slack=worst,
25     worst_point=wpt.tolist(),certified=bool(ok),seconds=time.time()-t)
26 print(json.dumps({k:v for k,v in res.items() if k!='stations'}))
27 json.dump(res,open('experiments/q4_layout_search/ring22_certificate.json','w'),
    indent=1)

```

K.7 program/practice_robot.py: 模拟器通信客户端

```

1  """第 3、4 问的官方协议演练运行器。
2
3  演练模块要在模拟器里手动选择，协议本身区分不了“演练/正式”会话。程序严格

```

```

4 按附件 2 串行发送、逐次等待响应；网络中断后用原请求内容与原 request_id
5 重试，结果未知的动作标记为 uncertain 并禁止再发不同请求，避免重复计时或
6 状态错位。每次发送前先写日志，防止中断掩盖一个可能已执行的动作；关键源码
7 的 sha256 与配置一起写入 run_config.json，用于事后筛选同版本成绩。
8 """
9 import argparse
10 import hashlib
11 import http.client
12 import json
13 import math
14 import socket
15 import sys
16 import time
17 import unicodedata
18 import uuid
19 from datetime import datetime
20 from pathlib import Path
21 from urllib.error import HTTPError, URLError
22 from urllib.parse import urlsplit
23 from urllib.request import ProxyHandler, Request, build_opener
24
25 from model import Policy
26 from optimized import OptimizedPolicy
27
28 class BudgetStop(RuntimeError):
29     pass
30
31 class ProtocolFailure(RuntimeError):
32     pass
33
34 class UncertainAction(RuntimeError):
35     """请求可能已经执行过：禁止再发送不同的下一个动作。"""
36     pass
37
38 class PracticeHTTP:
39     """官方 HTTP 协议的客户端：请求计数、重试、虚拟时间对账与动作日志。"""
40     def __init__(self, robot_id, base_url, log_path, timeout=5., attempts=3,
41                  progress=True):
42         self.robot_id=robot_id;self.base_url=base_url.rstrip('/')
43         self.timeout=timeout;self.attempts=attempts;self.progress=progress
44         self.log=Path(log_path).open('x',encoding='utf-8',buffering=1)
45         self.opener=build_opener(ProxyHandler({}))
46         self.session=uuid.uuid4().hex;self.counter=0;self.virtual_time=0.
47         self.deadline=None;self.enter_started=None;self.started=False;self.ended=
48         False
49         self.uncertain=False;self.position=[0.,0.];self.channel=1;self.cleared=set
50         ()
51         self.max_virtual=360000.;self.parts=dict(move=0.,switch=0.,measure=0.,
52         optical=0.,laser=0.)
53         self.accepted_actions=0;self.attempted_requests=0;self.latest_remaining=
54         None
55
56     def record(self,event,**data):

```

```

52     self.log.write(json.dumps(dict(event=event, local_time=datetime.now().
isoformat(), **data), ensure_ascii=False, allow_nan=False)+'\n')
53     self.log.flush()
54
55     @staticmethod
56     def number(value, name):
57         if isinstance(value, bool) or not isinstance(value, (int, float)) or not math.
isfinite(value):
58             raise ProtocolFailure(f'Invalid {name}: {value!r}')
59         return float(value)
60
61     def post(self, path, **fields):
62         if self.uncertain: raise UncertainAction('Previous action unresolved; stop
this run.')
63         self.counter+=1
64         payload=dict(arena_id='default', robot_id=self.robot_id, request_id=f'{self.
session}-{self.counter}', **fields)
65         raw=json.dumps(payload, ensure_ascii=False, allow_nan=False, separators=(',', '
:')).encode('utf-8')
66         if len(raw)>65536: raise ProtocolFailure('Request too large')
67         # 发送前先记录日志, 避免中断时掩盖一个可能已执行的动作。
68         self.record('request', path=path, payload=payload)
69         last=None
70         for attempt in range(1, self.attempts+1):
71             remaining=self.timeout if self.deadline is None else self.deadline-time
.monotonic()
72             reserve=0. if path=='/exit' else 8.
73             if self.deadline is not None and remaining<=reserve:
74                 if attempt>1:
75                     self.uncertain=True; raise UncertainAction('Deadline reached
before retry could resolve action')
76                     raise BudgetStop('Actual remaining runtime is nearly exhausted')
77             request=Request(self.base_url+path, data=raw, headers={'Content-Type': '
application/json'}, method='POST')
78             self.attempted_requests+=1
79             self.uncertain=True
80             try:
81                 with self.opener.open(request, timeout=min(self.timeout, max(.05,
remaining-reserve)) if self.deadline else self.timeout) as response:
82                     status=response.status; content=response.read()
83             except HTTPError as error:
84                 content=error.read().decode('utf-8', errors='replace')
85                 self.record('http_error', path=path, request_id=payload['request_id'
], status=error.code, body=content)
86                 # 服务器返回含糊错误后, 绝不再发起不同动作。
87                 raise ProtocolFailure(f'HTTP {error.code}: {content[:400]}') from
error
88             except (URLError, OSError, socket.timeout, http.client.HTTPException) as
error:
89                 last=error; self.record('transport_error', path=path, request_id=
payload['request_id'], attempt=attempt, error=str(error))
90                 if attempt<self.attempts:
91                     time.sleep(.15)

```

```

92         continue
93         raise UncertainAction(f'Connection failed after {attempt} attempts;
action status unknown: {last}') from error
94     try:
95         body=json.loads(content.decode('utf-8'))
96         if not isinstance(body,dict):raise ValueError('Response must be an
object')
97     except (ValueError,UnicodeError) as error:
98         self.record('invalid_response',path=path,body=content[:2000].decode
('utf-8',errors='replace'))
99         raise ProtocolFailure('Response is not valid JSON; action status
unknown') from error
100         self.record('response',path=path,request_id=payload['request_id'],
status=status,body=body)
101         if status!=200 or body.get('accepted') is not True:
102             raise ProtocolFailure(f'Request rejected; stop and check simulator:
HTTP {status}, {body}')
103             vt=self.number(body.get('virtual_time_s'),'virtual_time_s')
104             if vt<0 or vt+1e-6<self.virtual_time:raise ProtocolFailure('Virtual
clock went backwards')
105             self.virtual_time=vt;self.accepted_actions+=1
106             # 由调用方在放行前校验该动作特有的字段。
107             return body
108         raise UncertainAction(str(last))
109
110     def enter(self):
111         """进入会话，读取剩余现实时间与虚拟时间上限并设置截止时刻。"""
112         self.enter_started=time.monotonic();r=self.post('/enter')
113         remaining=self.number(r.get('remaining_real_duration_s'),'
remaining_real_duration_s')
114         limit=self.number(r.get('max_virtual_duration_s'),'max_virtual_duration_s')
115         if not 0<=remaining<=1200 or limit<=0:raise ProtocolFailure('Invalid
session limits')
116         self.deadline=self.enter_started+remaining;self.latest_remaining=remaining;
self.max_virtual=limit
117         self.started=True;self.uncertain=False
118         if self.progress:print(f'已进入测试，实际可用现实时间 {remaining:.0f} 秒。',flush
=True)
119         return r
120
121     def check_action(self,p,ch,worst_cost):
122         if not self.started or self.ended:raise ProtocolFailure('Session is not
active')
123         if isinstance(ch,bool) or int(ch)!=ch or not 1<=ch<=20:raise
ProtocolFailure('Channel must be an integer in 1..20')
124         p=[self.number(float(v),'coordinate') for v in p]
125         if len(p)!=2 or any(abs(v)>2000000 for v in p):raise ProtocolFailure('
Invalid position')
126         move=math.dist(p,self.position)/5
127         if self.virtual_time+move+worst_cost>=self.max_virtual-1:raise BudgetStop('
Virtual time budget nearly exhausted')
128         return p,move
129

```

```

130 def account(self,before,delta):
131     # 官方时钟按微秒取整, 允许累计的极小取整误差。
132     if abs(self.virtual_time-before-delta)>0.002:raise ProtocolFailure('
Official clock disagrees with action accounting')
133     self.uncertain=False
134     if self.progress and self.accepted_actions%50==0:
135         print(f'动作 {self.accepted_actions} | 已清除 {len(self.cleared)} | 虚拟时
间 {self.virtual_time:.1f} 秒',flush=True)
136
137 def measure(self,p,ch):
138     """移动到 p 后测频道 ch, 按协议计入移动、切频与检测时间。"""
139     p,move=self.check_action(p,ch,6);before=self.virtual_time;switch=int(self.
channel!=ch)
140     r=self.post('/measure',position=dict(x=p[0],y=p[1]),channel=int(ch))
141     kind=r.get('measure_result')
142     if kind not in ('direction','near','no_signal'):raise ProtocolFailure('
Unknown measure_result')
143     if kind=='direction':
144         angle=self.number(r.get('svd_deg'),'svd_deg')
145         if not 0<=angle<360:raise ProtocolFailure('Bearing outside [0,360)')
146     self.account(before,move+switch+5);self.position=p;self.channel=int(ch)
147     self.parts['move']+=move;self.parts['switch']+=switch;self.parts['measure'
] +=5
148     return r
149
150 def clear(self,p,ch):
151     """移动到 p 后对频道 ch 发起清除, 成功则记入已清除频道。"""
152     p,move=self.check_action(p,ch,5);before=self.virtual_time
153     r=self.post('/clear',position=dict(x=p[0],y=p[1]),channel=int(ch))
154     kind=r.get('clear_result')
155     if kind not in ('success','no_target_in_range'):raise ProtocolFailure('
Unknown clear_result')
156     success=kind=='success'
157     if success and ch in self.cleared:raise ProtocolFailure('Duplicate
successful clear')
158     self.account(before,move+3+2*success);self.position=p
159     self.parts['move']+=move;self.parts['optical']+=3;self.parts['laser']+=2*
success
160     if success:
161         self.cleared.add(int(ch))
162         if self.progress:print(f'清除频道 {ch}, 累计 {len(self.cleared)} 个, 虚拟时
间 {self.virtual_time:.1f} 秒.',flush=True)
163     return r
164
165 def exit(self):
166     """主动退出会话, 要求 exit_reason 为 user_exit。"""
167     r=self.post('/exit')
168     if r.get('exit_reason')!='user_exit':raise ProtocolFailure('Unexpected exit
reason')
169     self.uncertain=False;self.ended=True;return r
170
171 def arguments(argv=None):
172     """解析命令行参数, 并校验队号、回环地址与时间参数的取值范围。"""

```

```

173 parser=argparse.ArgumentParser(description='第3/4问演练程序：覆盖搜索、路线排序与多目
    标共用测站。')
174 parser.add_argument('--question', type=int, choices=[3, 4], required=True)
175 parser.add_argument('--robot-id', required=True, help='必须与模拟器登录队号一致')
176 parser.add_argument('--base-url', default='http://127.0.0.1:2026')
177 parser.add_argument('--strategy', choices=['shared', 'route', 'triangulate', 'sweep'],
    default='shared')
178 parser.add_argument('--output-dir', default=str(Path(__file__).resolve().parent/
    'practice_logs'))
179 parser.add_argument('--timeout', type=float, default=5.)
180 parser.add_argument('--attempts', type=int, default=3)
181 parser.add_argument('--case-code', default='', help='可选：从模拟器界面抄录，用于关联日
    志')
182 parser.add_argument('--connect', action='store_true', help='连接已经启动且接口就绪的演
    练测试')
183 parser.add_argument('--config', default='config.json', help='配置文件名（位于程序目
    录）；实验入口使用 config_q3_six.json')
184 args=parser.parse_args(argv)
185 if not args.connect: parser.error('未发送任何请求。请先在模拟器启动对应演练，接口就绪后加
    --connect。')
186 if not args.robot_id or len(args.robot_id.encode('utf-8'))>64 or any(
    unicodedata.category(ch).startswith('C') for ch in args.robot_id): parser.error('
    队号为空、过长或包含不可见字符')
187 url=urlsplit(args.base_url)
188 if url.scheme!='http' or url.hostname not in ('127.0.0.1', 'localhost', ':::1') or
    url.path not in ('', '/') or url.query or url.fragment or url.username or url.
    password: parser.error('接口地址必须是本机回环HTTP地址，例如 http://127.0.0.1:2026')
189 if not math.isfinite(args.timeout) or not .1<=args.timeout<=30 or not 1<=args.
    attempts<=5: parser.error('timeout必须在0.1至30秒之间，attempts必须在1至5之间')
190 return args
191
192 def main(argv=None):
193     """运行一次演练，并落盘 summary、run_config 与定位区域记录。"""
194     args=arguments(argv); base=Path(__file__).resolve().parent
195     if Path(args.config).name!=args.config or not (base/args.config).is_file():
196         raise SystemExit(f'配置文件必须位于程序目录：{args.config}')
197     cfg=json.loads((base/args.config).read_text(encoding='utf-8'))
198     out=Path(args.output_dir); out.mkdir(parents=True, exist_ok=True)
199     run_dir=out/f'q{args.question}_{datetime.now():%Y%m%d_%H%M%S}_{uuid.uuid4().hex
    [:8]}'
200     run_dir.mkdir(); io=PracticeHTTP(args.robot_id, args.base_url, run_dir/'actions.
    jsonl', args.timeout, args.attempts)
201     meta=dict(question=args.question, strategy=args.strategy, declared_mode='practice',
    mode_verified_by_protocol=False, case_code=args.case_code, config=cfg, python=sys
    .version, code_sha256={name:hashlib.sha256((base/name).read_bytes()).hexdigest()
    for name in ['practice_robot.py', 'model.py', 'optimized.py', 'q3_policy.py', '
    q4_policy.py', 'q4_optical.py', args.config]}, config_file=args.config)
202     (run_dir/'run_config.json').write_text(json.dumps(meta, ensure_ascii=False,
    indent=2), encoding='utf-8')
203     print(f'第{args.question}问，策略 {args.strategy}，日志：{run_dir}', flush=True)
204     if args.config!='config.json': print(f'警告：使用实验配置 {args.config}，不保证发现全
    部干扰源，请勿用于正式测试。', flush=True)
    print('接口无法识别演练/正式模式；当前应为模拟器中已启动的对应演练。', flush=True)

```

```

205 policy=None;result={};status='failed';error=None;code=1
206 try:
207     io.enter()
208     policy=OptimizedPolicy(io,cfg,args.question,args.strategy) if args.strategy
209     in ('shared','route') else Policy(io,cfg,args.question,args.strategy)
210     result=policy.run();io.exit();status='completed';code=0
211 except (Exception,KeyboardInterrupt) as exc:
212     error=f'{type(exc).__name__}: {exc}'
213     io.record('run_error',error=error)
214     # 只有上一个动作已确定、且会话仍有效时才退出，避免状态错位。
215     if io.started and not io.ended and not io.uncertain and io.deadline and
time.monotonic()<io.deadline-1:
216         try:io.exit()
217         except Exception as exit_error:io.record('exit_error',error=str(
exit_error))
218         if isinstance(exc,KeyboardInterrupt):status='interrupted';code=130
219         elif isinstance(exc,BudgetStop):status='budget_stop';code=2
220         print(f'本次未确认完成: {error}',file=sys.stderr,flush=True)
221 finally:
222     summary=dict(status=status,error=error,**result,cleared_channels=sorted(io.
cleared),successful_clear_count=len(io.cleared),total_virtual_s=io.virtual_time,
average_per_cleared_s=io.virtual_time/len(io.cleared) if io.cleared else None,
clearance_ratio=None,true_source_count=None,program_elapsed_s=time.monotonic()-
io.enter_started if io.enter_started else None,ended_by_exit=io.ended,
unresolved_action=io.uncertain,accepted_actions=io.accepted_actions,
http_attempts=io.attempted_requests,time_components_s=io.parts,case_code=args.
case_code)
223     (run_dir/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,
indent=2),encoding='utf-8')
224     if policy is not None:(run_dir/'localization_regions.json').write_text(json
.dumps(policy.certificates,ensure_ascii=False,indent=2),encoding='utf-8')
225     io.record('summary',**summary);io.log.close()
226     if code==0:print(f'任务完成, 清除 {len(io.cleared)} 个, 平均 {summary["
average_per_cleared_s"]:.2f} 虚拟秒/源。',flush=True)
227     print(f'结果已保存: {run_dir / "summary.json"}',flush=True)
228     return code
229 if __name__=='__main__':sys.exit(main())

```

K.8 program/config.json: 参数配置

完整配置文件随支撑材料提交；关键参数、取值与来源已汇总于附录 B 的表 12，此处不再逐行列出。